Three-Tier Architecture

Building a Three-Tier Architecture on AWS with Deploying Application

Achieving High Scalability, High Availability, and Fault Tolerance

- **Prerequisites**
  - AWS Account
  - Basic knowledge of Linux
- **List of AWS services**
  - Amazon Route 53
  - Amazon EC2
  - Amazon Auto scaling
  - Amazon Certificate Manager
  - Amazon RDS
  - Amazon VPC
  - Amazon Cloud Watch

Plan of Execution

- What is three-tier architecture
- The architecture of the project
- A step-by-step guide with screenshots
- Testing
- Resource cleanup

917396627149-Madhukiran
devopstraininghub@gmail.com

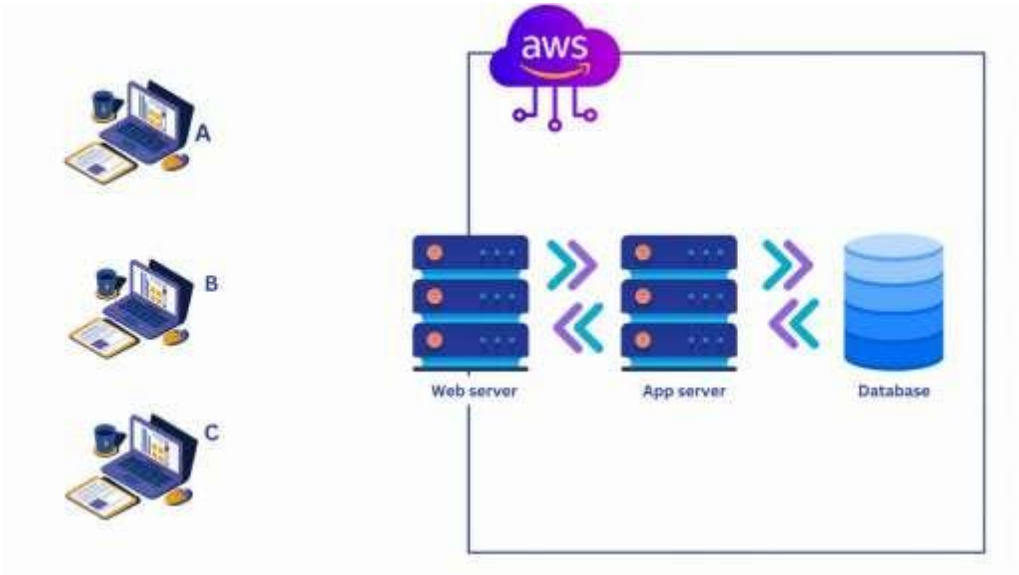
What is Three-tier architecture?

Three-tier architecture is a software architecture pattern that separates an application into three layers.

Presentation layer ➡ handles user interaction

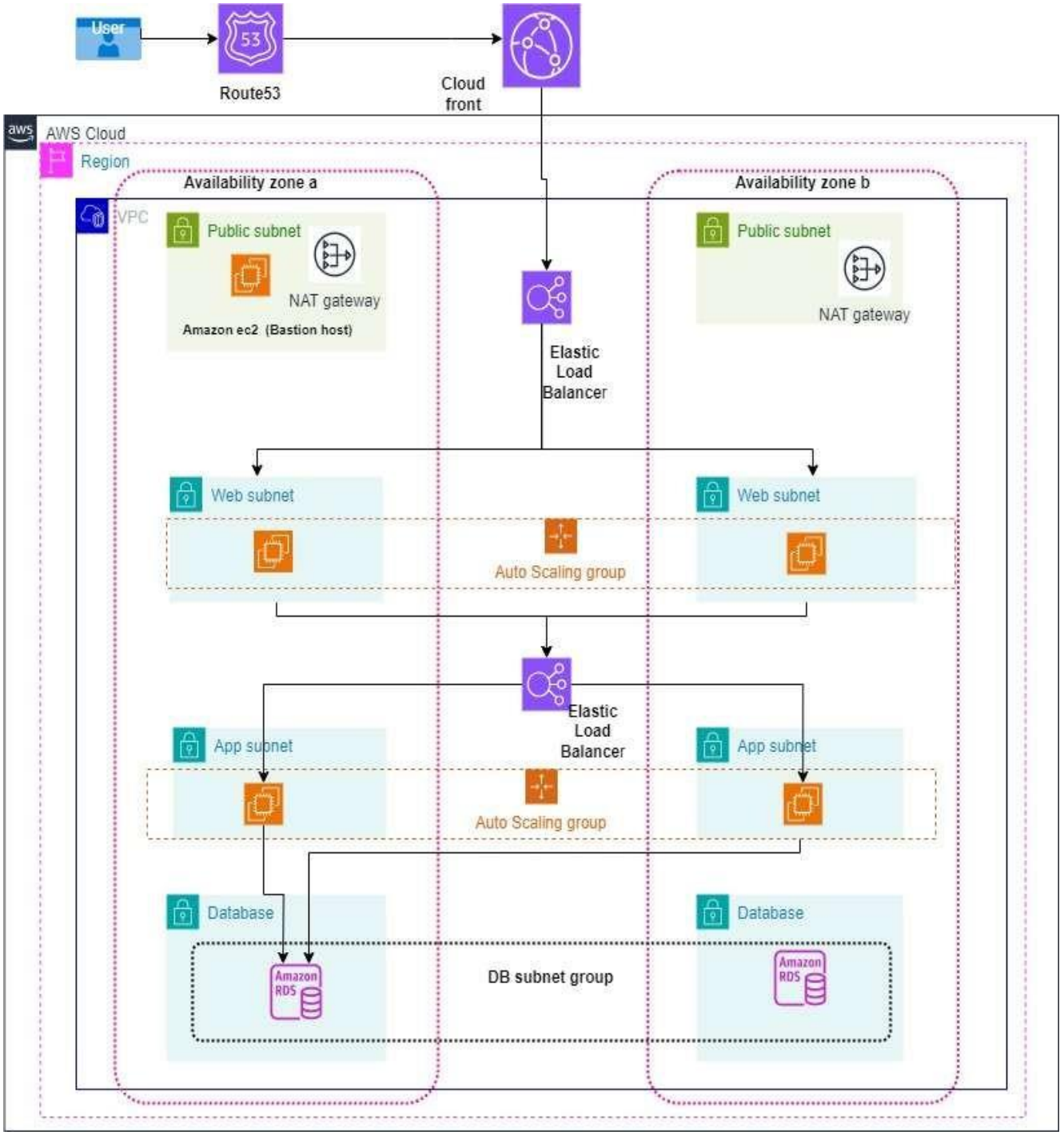
Application layer (backend logic) ➡ processes business logic and data processing

Data layer (database) ➡ manages data storage and retrieval



Each layer has distinct responsibilities, allowing for modularity, scalability, and maintainability. This architecture promotes the separation of concerns and facilitates easy updates or modifications to specific layers without impacting others.

Architecture of the Project



917396627149-Madhukiran
devopstraininghub@gmail.com

Architecture

The image describes three-tier architecture on AWS, which is commonly used to build highly available and scalable web applications.

1. Route 53 (DNS Service)

- **User Requests**: A user initiates a request, which is routed through AWS Route 53, a DNS service that translates domain names to IP addresses.
- **Load Balancing**: Route 53 directs the traffic to an Elastic Load Balancer (ELB) in the appropriate region.

2. CloudFront Distribution:

- CloudFront will cache the static content (images, CSS, JavaScript, etc.) and provide faster access to users from the edge locations around the world.
- CloudFront receives the user's request before passing it to the ELB.
- The CloudFront distribution is connected to the ELB for dynamic content delivery from the web and app servers.

3. Elastic Load Balancer (ELB)

- **Traffic Distribution**: The ELB distributes incoming application traffic across multiple targets, such as EC2 instances in different Availability Zones (AZs), ensuring high availability and fault tolerance.
- **Scaling**: ELBs also work with Auto Scaling groups to dynamically adjust the number of EC2 instances based on traffic demand.

4. VPC (Virtual Private Cloud)

- **Network Segmentation**: The architecture is hosted within a VPC, providing network isolation and segmentation. The VPC spans multiple Availability Zones for high availability.

5. Public Subnets

- **NAT Gateway and Bastion Host**: Each Availability Zone has a public subnet containing a NAT Gateway and possibly a Bastion Host (Amazon EC2 instance). The NAT Gateway allows instances in private subnets to connect to the internet for updates or other external communications without exposing them to inbound internet traffic.

6. Web Tier (Web Subnets)

- **Web Servers**: The web subnet in each Availability Zone hosts EC2 instances (web servers) running the front-end application. These instances are part of an Auto Scaling group that adjusts the number of servers based on demand.
- **ELB Connection**: The Elastic Load Balancer routes incoming traffic to these web servers.

7. Application Tier (App Subnets)

- **App Servers**: The app subnets contain EC2 instances that handle business logic and processing. Like the web tier, these instances are part of an Auto Scaling group for scalability.
- **Internal Load Balancing**: A second ELB could be used here to balance traffic among app servers.

8. Private Subnets

- **Database Tier (DB Subnet Group)**: The private subnet houses Amazon RDS instances (Relational Database Service) within a DB Subnet Group. This ensures the database is only accessible from within the VPC, enhancing security.
- **Network Isolation**: Since this subnet is private, it does not have direct internet access, further protecting sensitive data.

9. Security

- **Subnets and Security Groups**: Each tier (web, app, and database) is in separate subnets with different security groups, ensuring proper access control. Web servers might only accept traffic from the internet, app servers from the web servers, and the database only from the app servers.

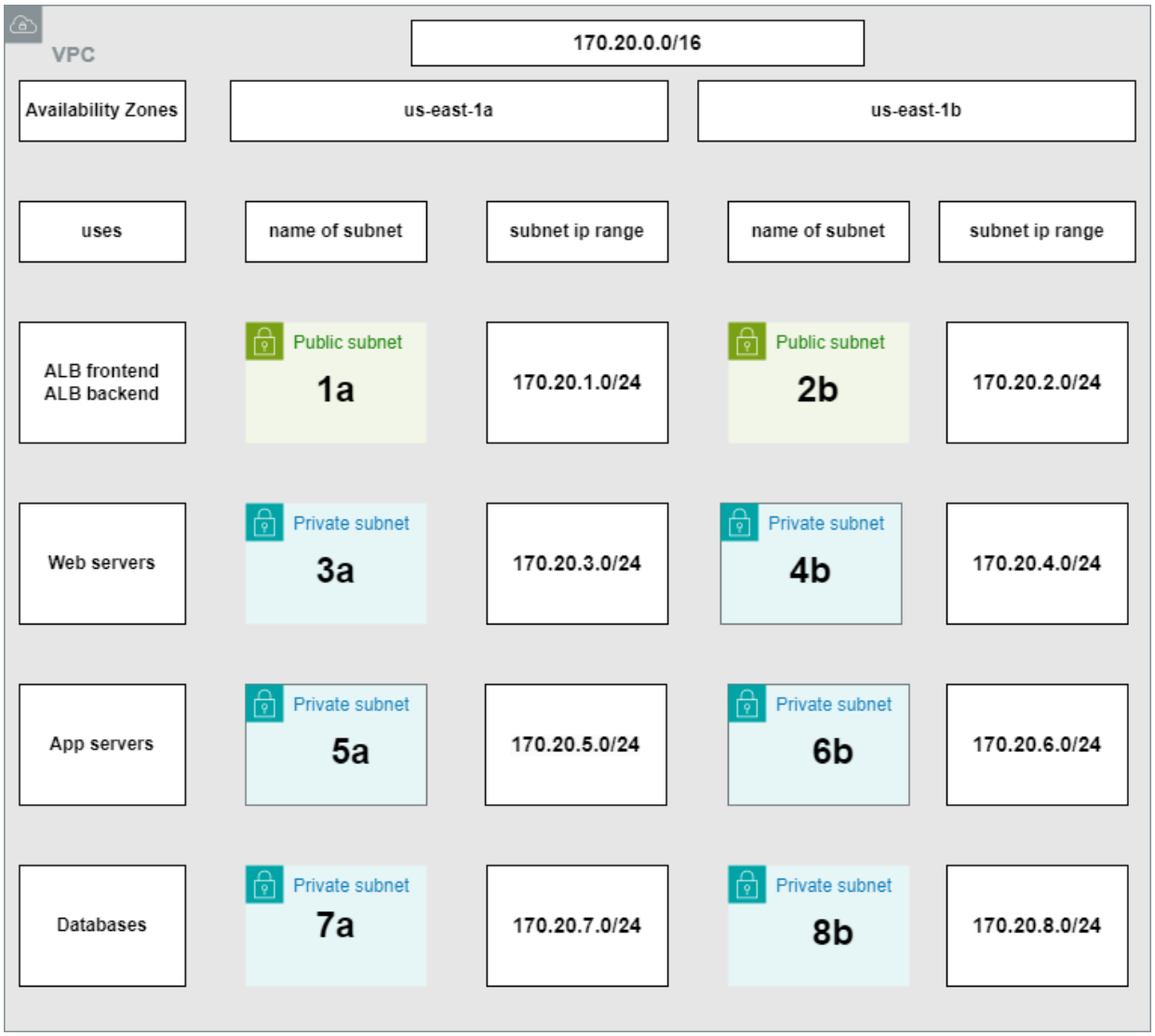
Workflow Overview

1. **User Request:** A user sends a request, which is routed through **Route 53**.
2. **CloudFront Caching:** CloudFront checks if the requested content is cached at an edge location.
    - If cached, the content is served directly from the edge location.
    - If not cached, CloudFront forwards the request to the **Elastic Load Balancer**.
3. **Load Balancing:** The ELB distributes the request to an available **web server** in the web subnet.

4. **Web Server Processing:** The web server processes the request or forwards it to the app server.
5. **App Server Logic:** The app server performs business logic, querying the **Amazon RDS database** if needed.
6. **Response Delivery:** The data flows back from the app server to the web server, through CloudFront, and then to the user.

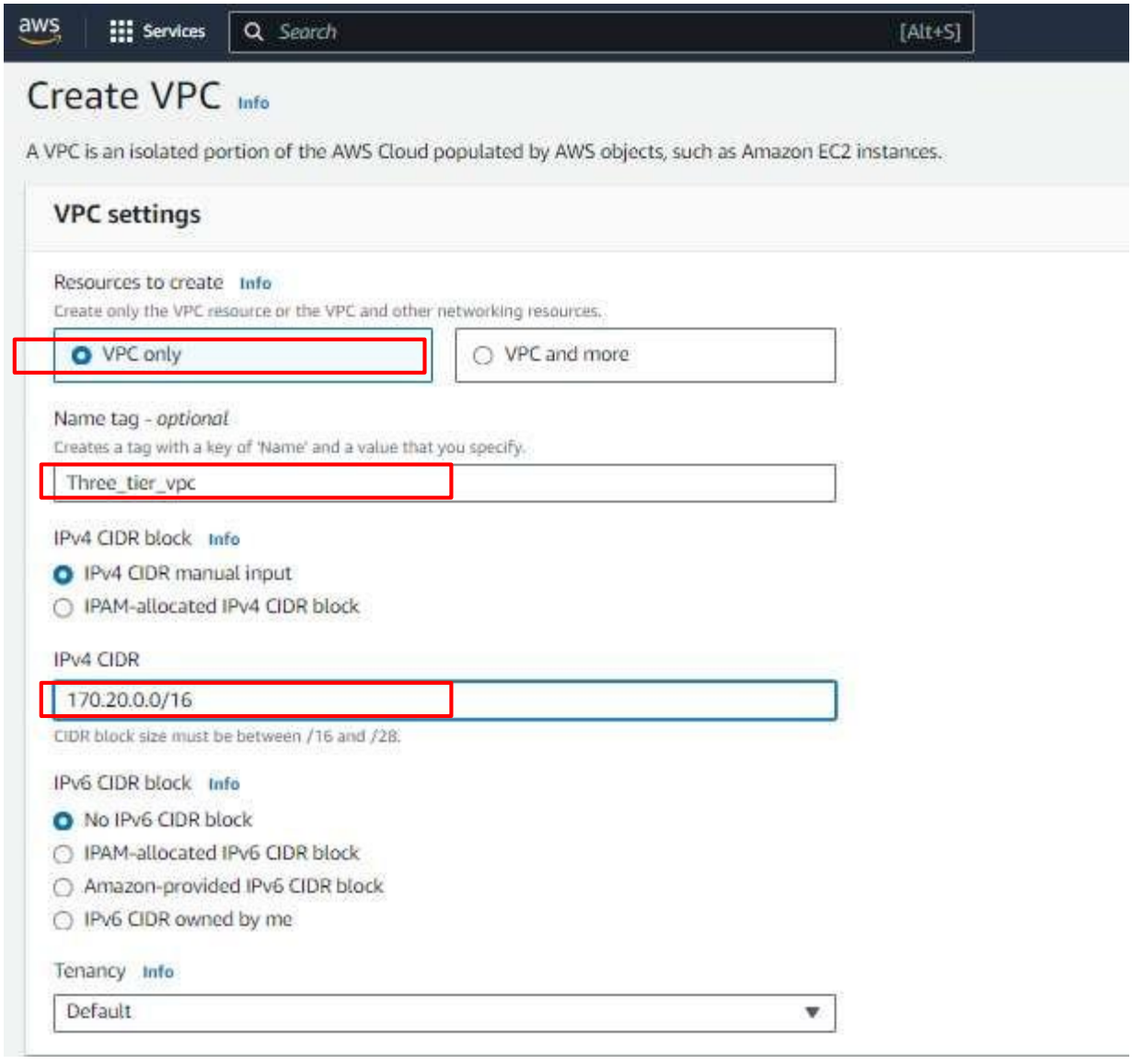
Creation of VPC, subnets, Rouetables, IGW, Natgateway

Setting up VPC in us-east-1 region to isolate our resources from the internet. The below image contained all the subnets, their IP range, and their uses.



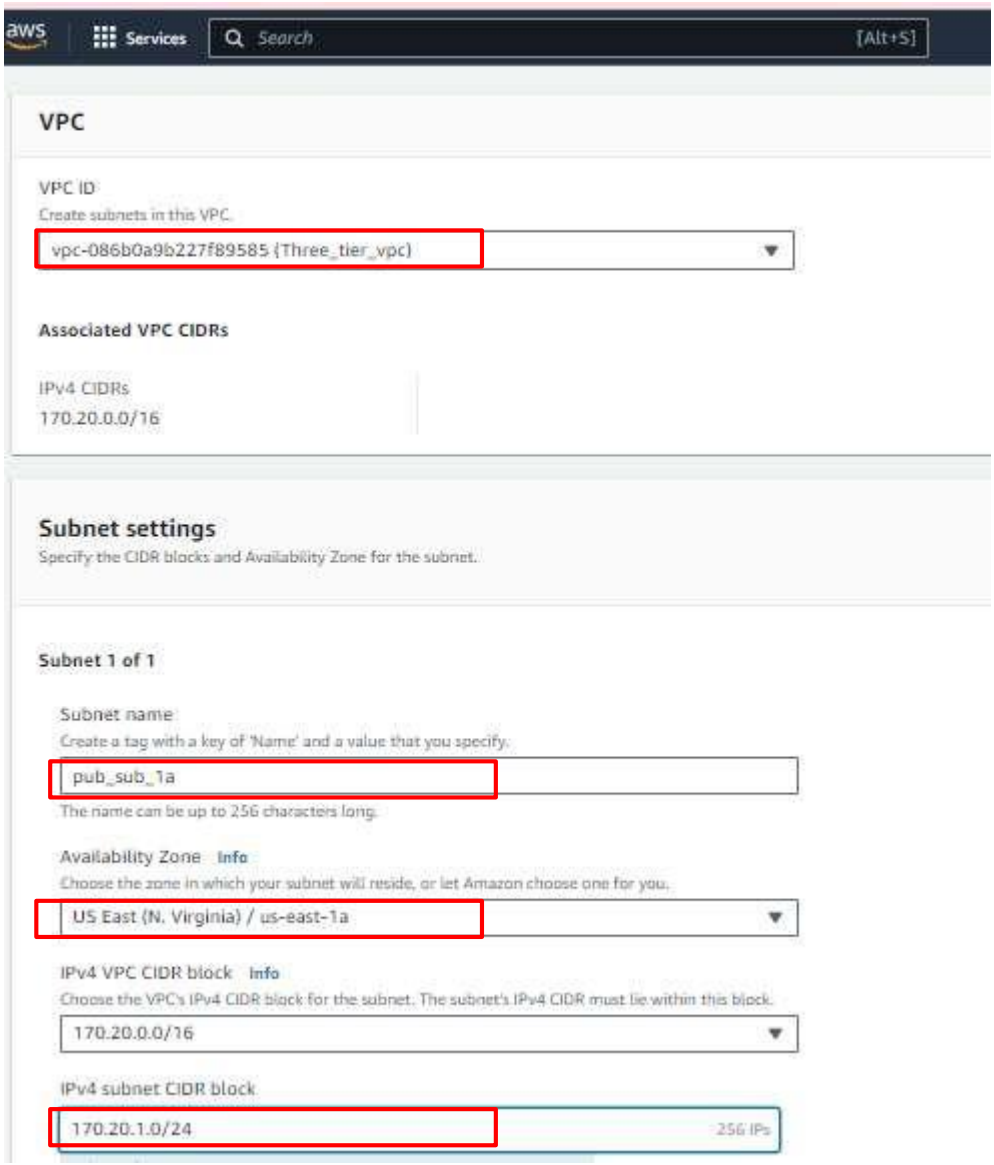
VPC:

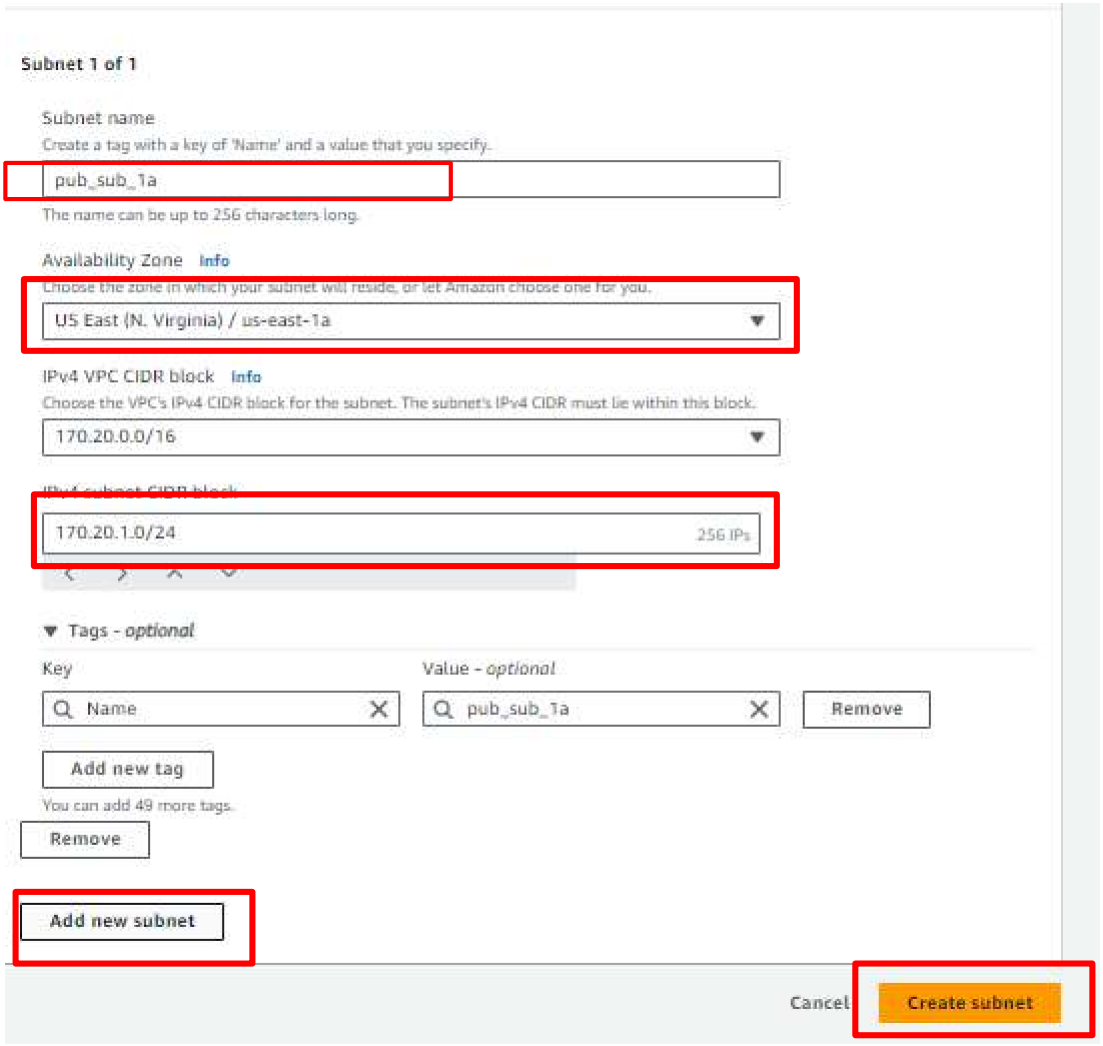
1. Please log in to your AWS Account and type VPC in the AWS console. And click on VPC service.
2. Click on Your VPC's button on the left and then click on Create VPC the button on the top right corner of the page
3. Here we can see the form where we can fill the configuration of VPC. Please enter the name that you want to keep and the IPV4 CIDR block. in my case CIDE block is 170.20.0.0/16.



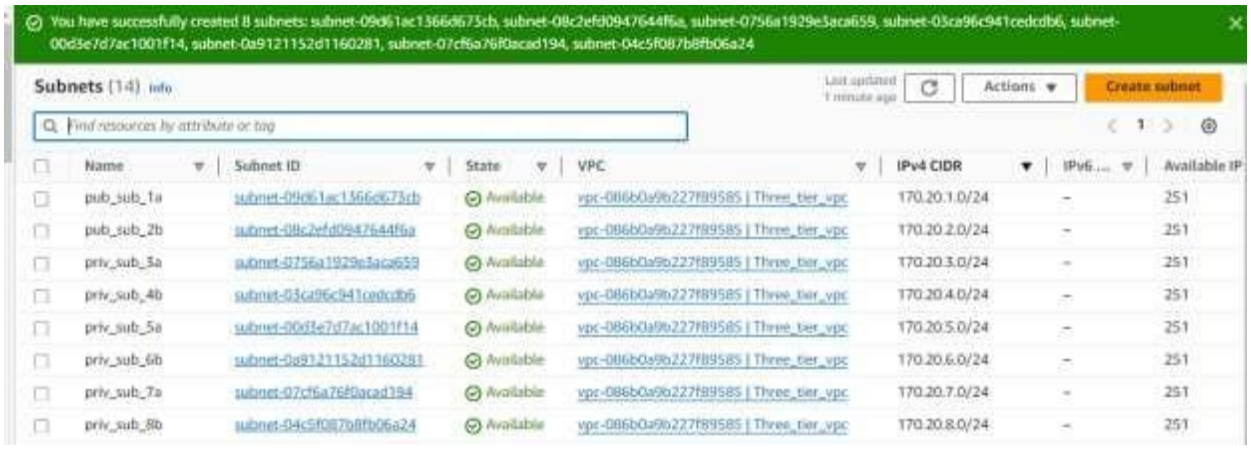
Subnets:

1. Now click on the subnet button which is located on the left side and then click on the Create subnet button on the top right corner of the page.
2. Please remove the default VPC ID and choose the VPC ID that we have just created in the VPC ID field. And click on the Add Subnet button at the bottom.
3. Now we need to configure our subnets. We are going to create a total of 8 subnets of which 2 of them are public and the rest of 6 subnets are private. After adding all the subnets click on Create subnet button.





After the successful creation of all 8 subnets, they look like this. you can verify with my subnets.

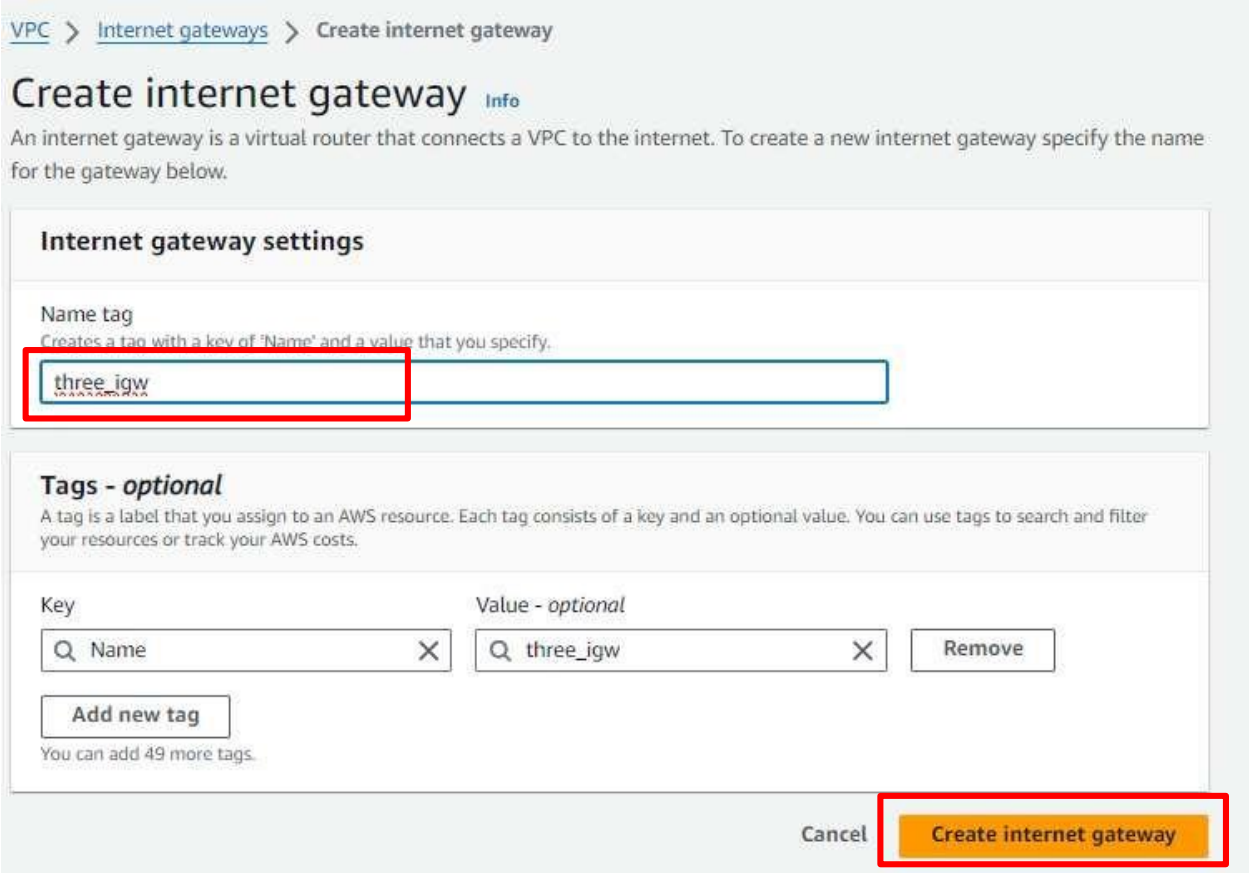


917396627149-Madhukiran
devopstraininghub@gmail.com

Internet Gateway:

Now we are going to create Internet Gateway also known as **IGW.** It is responsible for communication between VPC, VPC's public subnet with the Internet. Without IGW we won't be able to communicate with the Internet. Click on the internet gateways button at the left panel. and then click on the Create Internet gateways button on the top right corner of the page.

Give any name to IGW. And click on Create Internet gateway button.



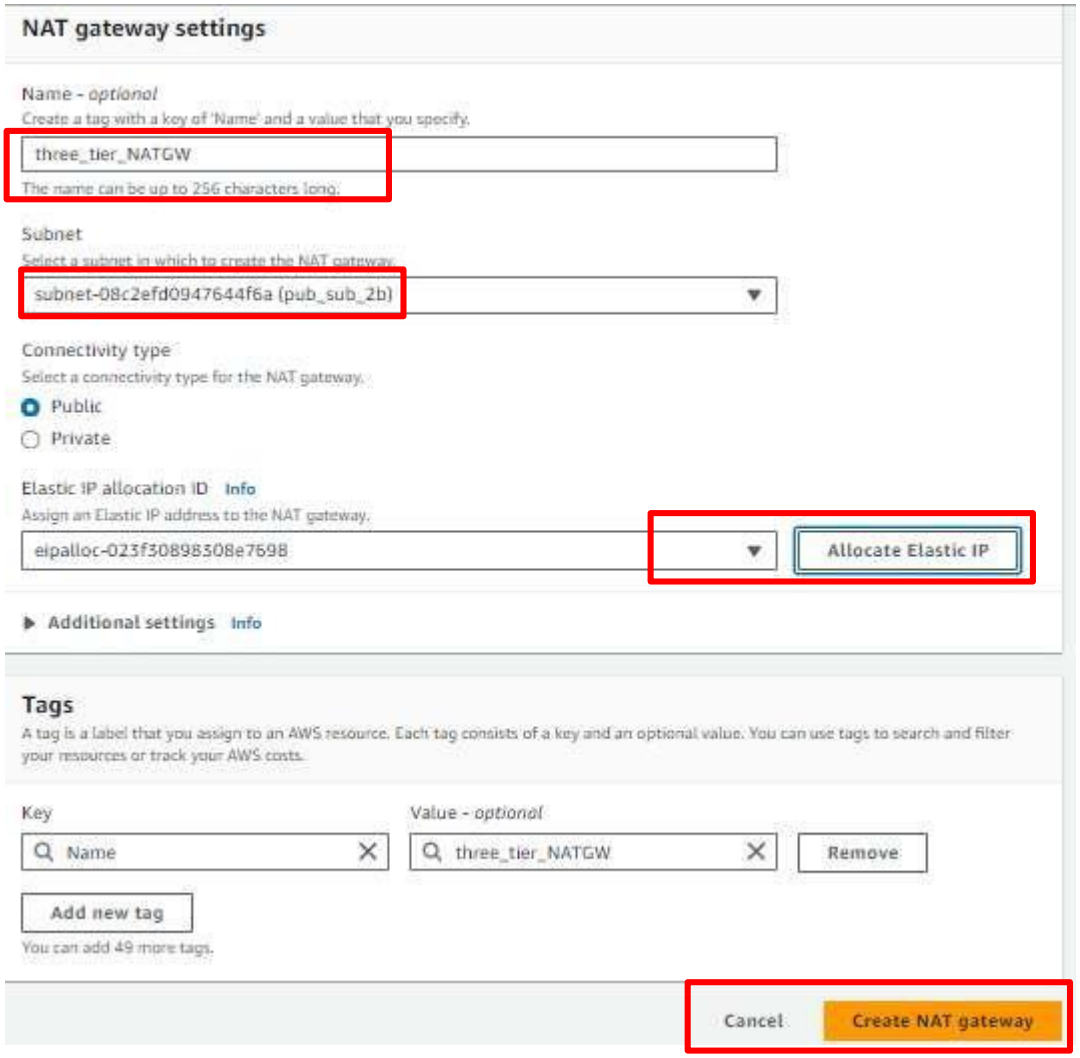
After creating an internet gateway we need to attach it with VPC to use it. For that click on the Action button. Here you can see the drop-down list. Please select the option Attach to VPC.

Please select VPC that we have created just now from the Available VPC list. And then click on the Attach Internet gateway button.

NAT gateway

NAT gateway is responsible to connect resources that are in the private subnet to communicate with the internet. All the resources which will be there in a private subnet will communicate to the internet through the NAT gateway. We will keep the NAT gateway in the public subnet so that it can access the internet. NAT gateway is a chargeable resource. So you will be charged by AWS as long as you keep it up. Now to create a NAT gateway click on the NAT gateways button on the left panel of the web page. and then click on the Create NAT gateways button in the top right corner of the page.

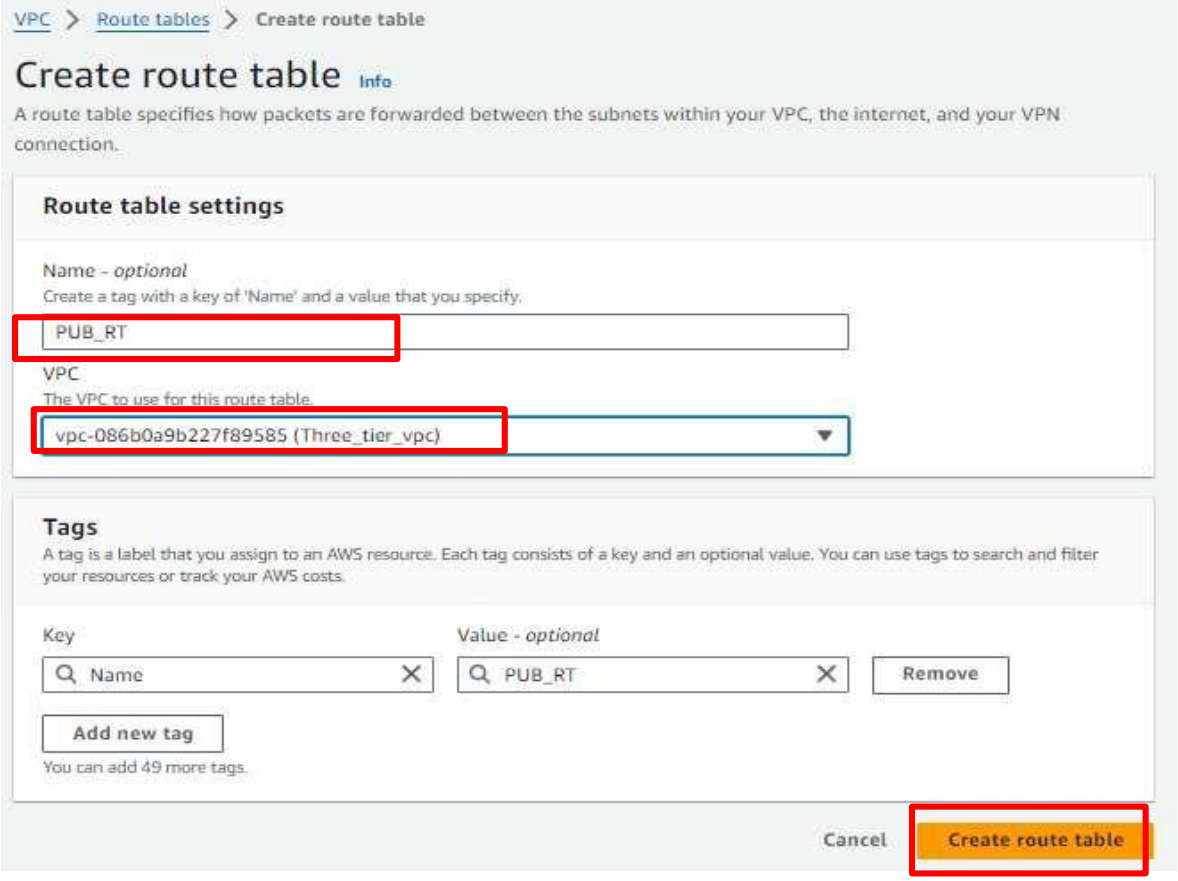
Give any name to the NAT gateway. But be cautious with selecting a subnet. ***You have to select one of the Public subnets among the two. either pub-sub-1a or pub-sub-2b***. Then click on the Allocate Elastic IP button to allocate Elastic IP. And then click on the Create NAT gateways button. NAT gateways creation takes 2-4 minutes.



Route table:

A Route Table to handle traffic for public subnet and private subnet and for that, we need to create a Route table. We are going to create two route tables one for the public subnet and another one for the private subnet. First, we are going to create RT for the public subnet. So click on the Route table button which you can see on the left panel. and click on the Create Route table button on the top corner of the page.

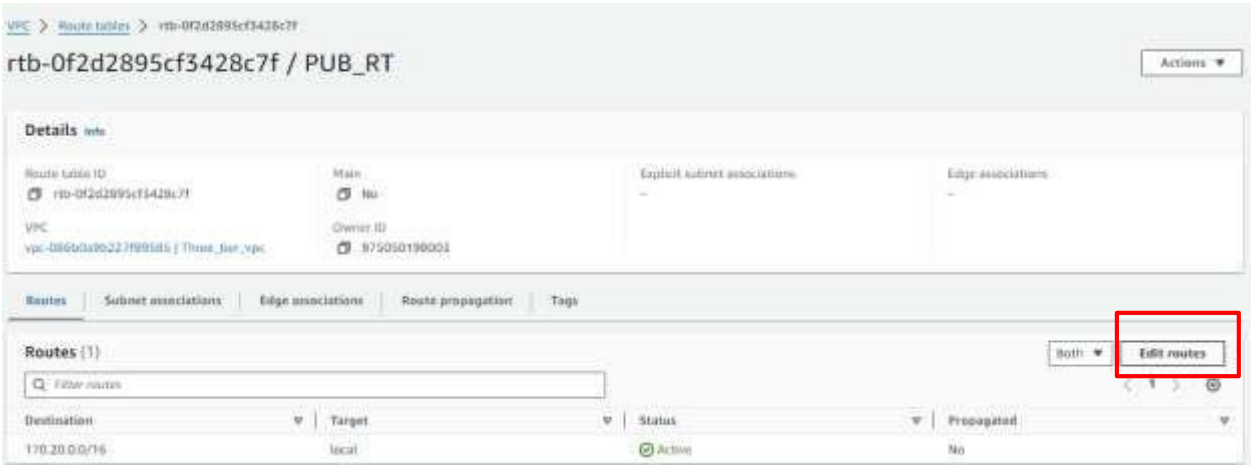
Give a name to your RT such as Pub-RT. Please give a name that is appropriate for resources then it will be easy to organize the things. Make sure you select the correct VPC. And then click on the create route table.



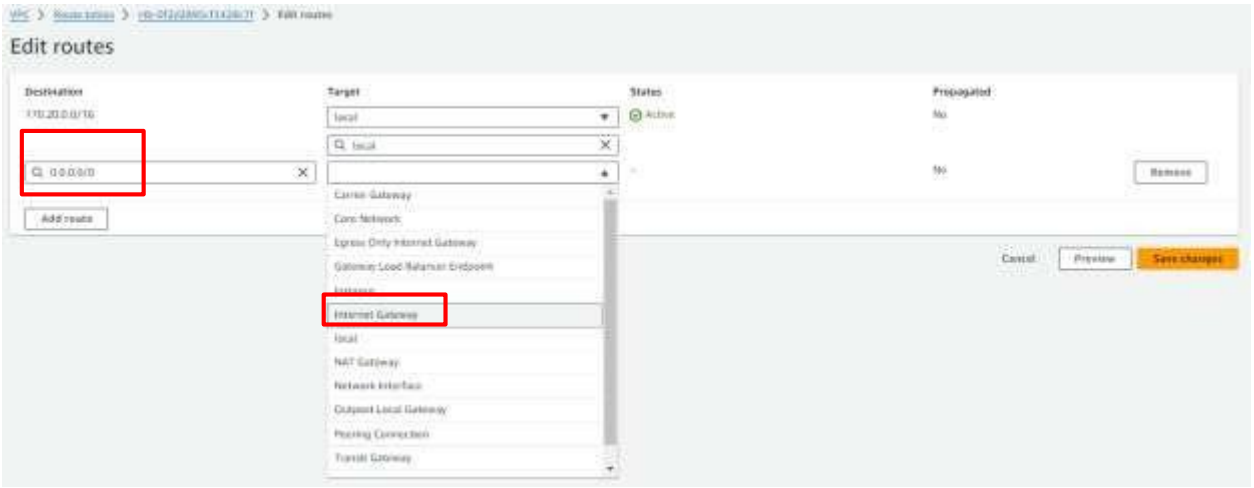
Create RT for the private subnet.

917396627149-Madhukiran
devopstraininghub@gmail.com

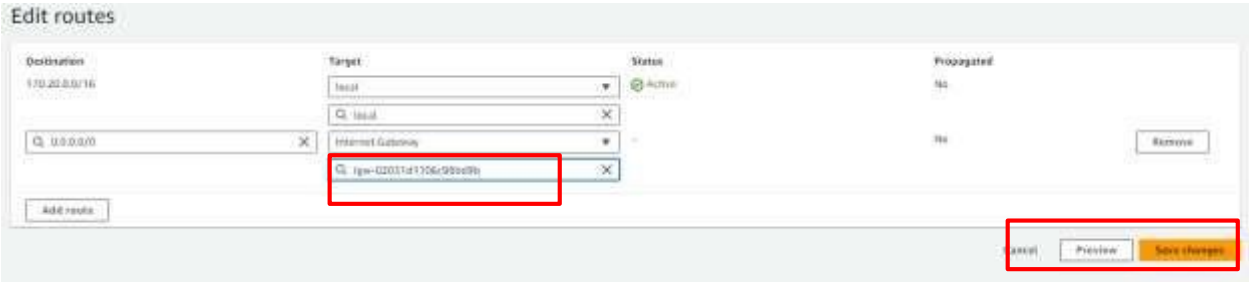
Now, we need to do some association with both RTs so select **Pub-RT** and click on the Routes tab at the bottom and then click on the edit route button.



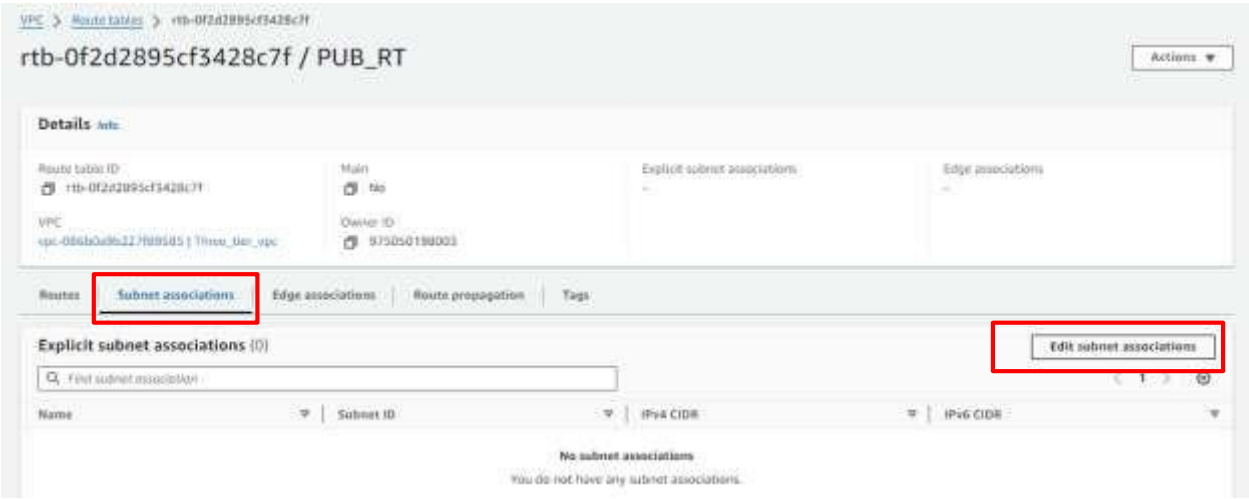
Click on the Add Route button. And select 0.0.0.0/0 in the destination field. And then click on the Target field. As soon as you click on the Target field one drop-down will open and here you have to select Internet gateway, shown in the below image.



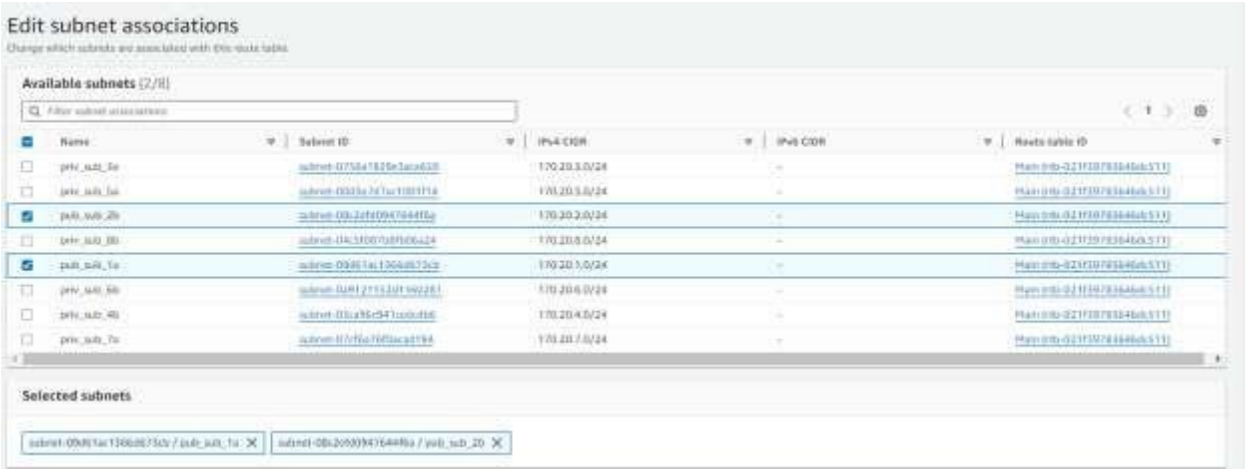
Here you can see the IGW that we created earlier. Select that IGW and click the save changes button.



Keep Pub-RT selected and click on the Subnet associations tab next to the Routes tab. and then click on the Edit subnet associations.



Now select both public subnets. **pub-sub-1a** and **pub-sub-2b** and click on the save associations button.

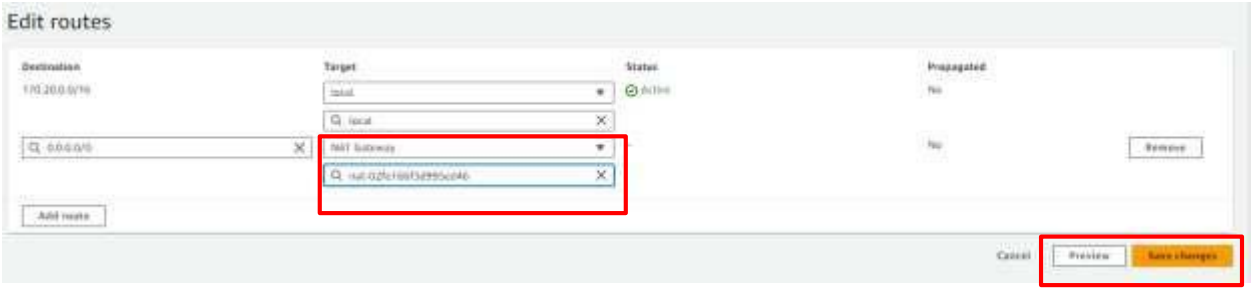


Now please select Pri-RT and click on the Routes tab at the bottom of the page.

Here please select 0.0.0.0/0 in the destination field and click on the target. As soon as you click on the target you will see the drop-down list.

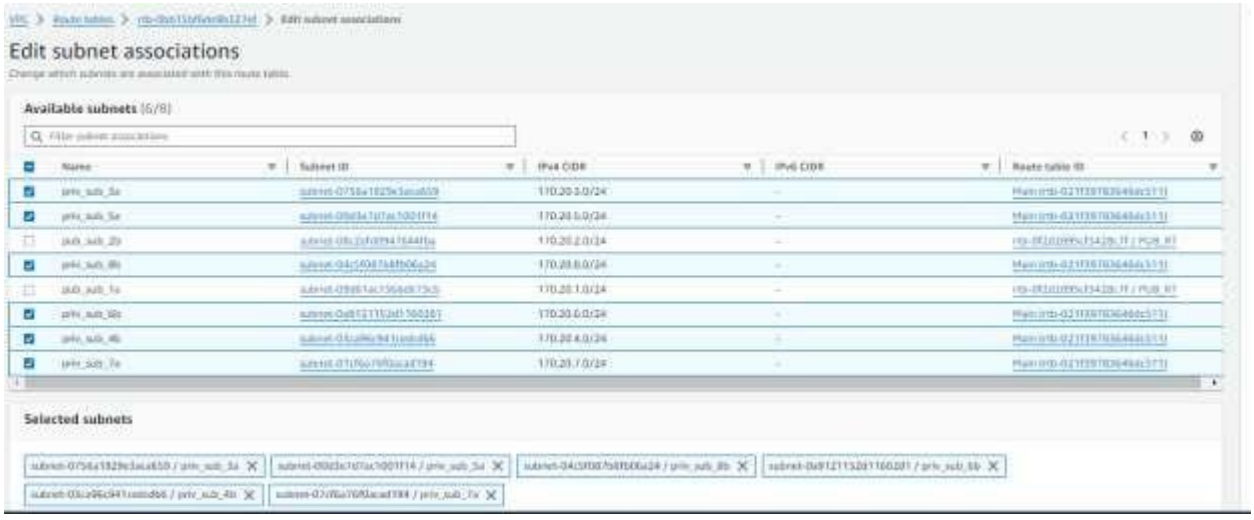
Please select NAT gateway from the drop-down list. As shown in the below image.

Select the NAT gateway that we have just created. And click on the save changes button.



Keep Pri-RT selected and click on the subnet associations tab at the bottom next to the Routes tab. And then click on the Edit route association's button.

Select all the 6 private subnets. And then click on the save association button.

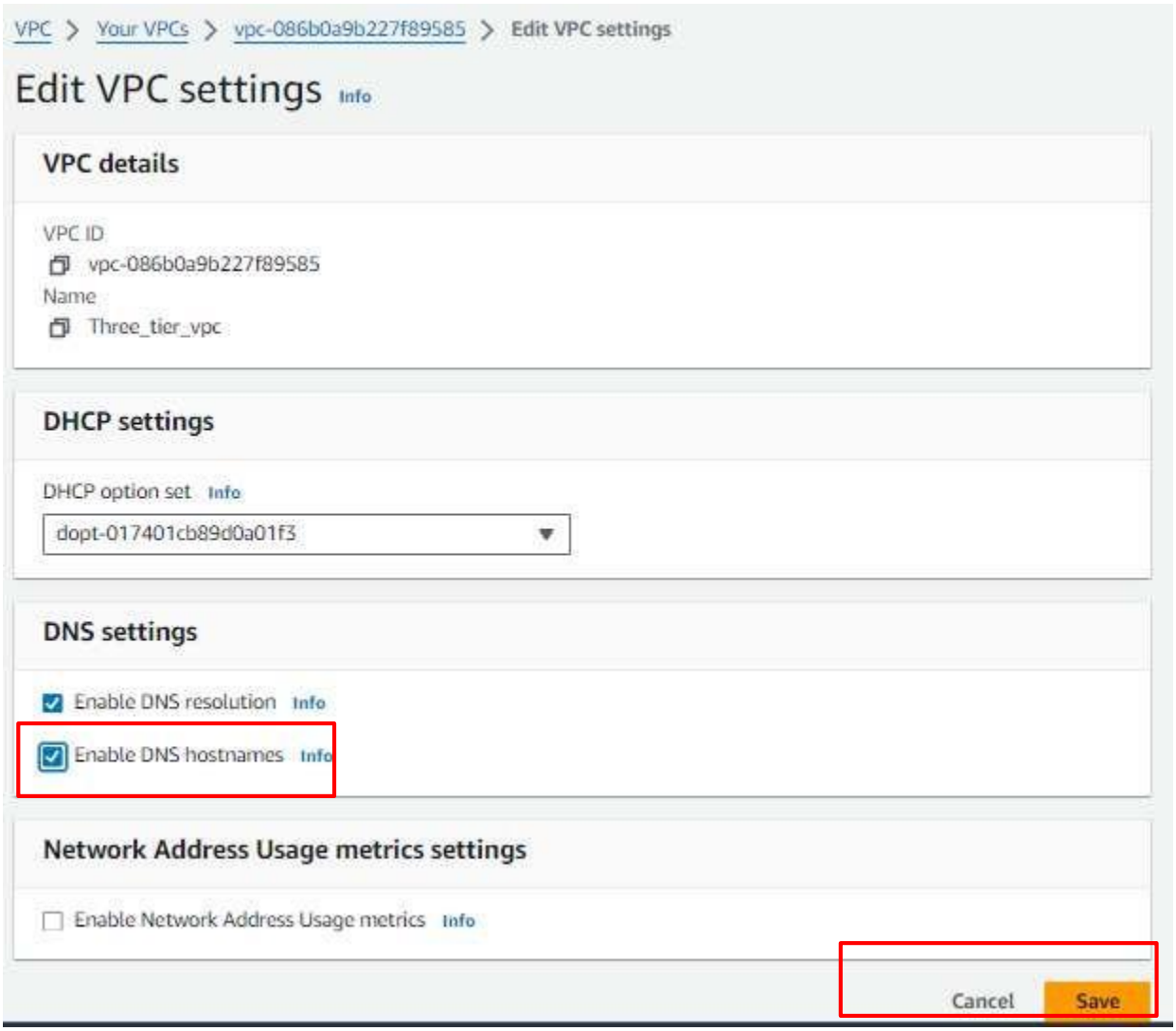


Change the settings of VPC and two public subnets. So just click on the VPC button on the left panel and select VPC that we have created and click on the action button and there you will see

917396627149-Madhukiran
devopstraininghub@gmail.com

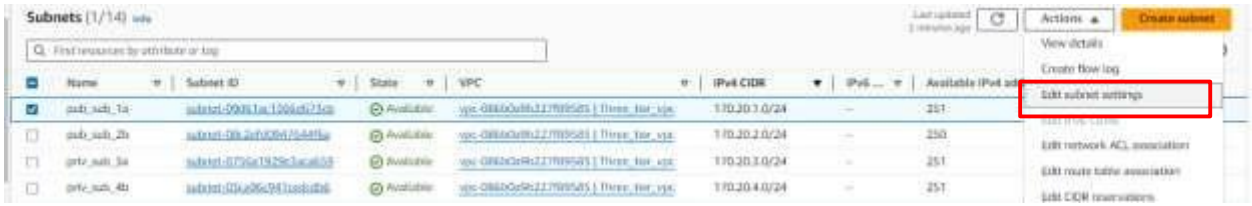
the drop-down menu. Select the Edit VPC setting button.

And here please **Enable DNS hostname** checkbox by clicking on it and then click on the Save button

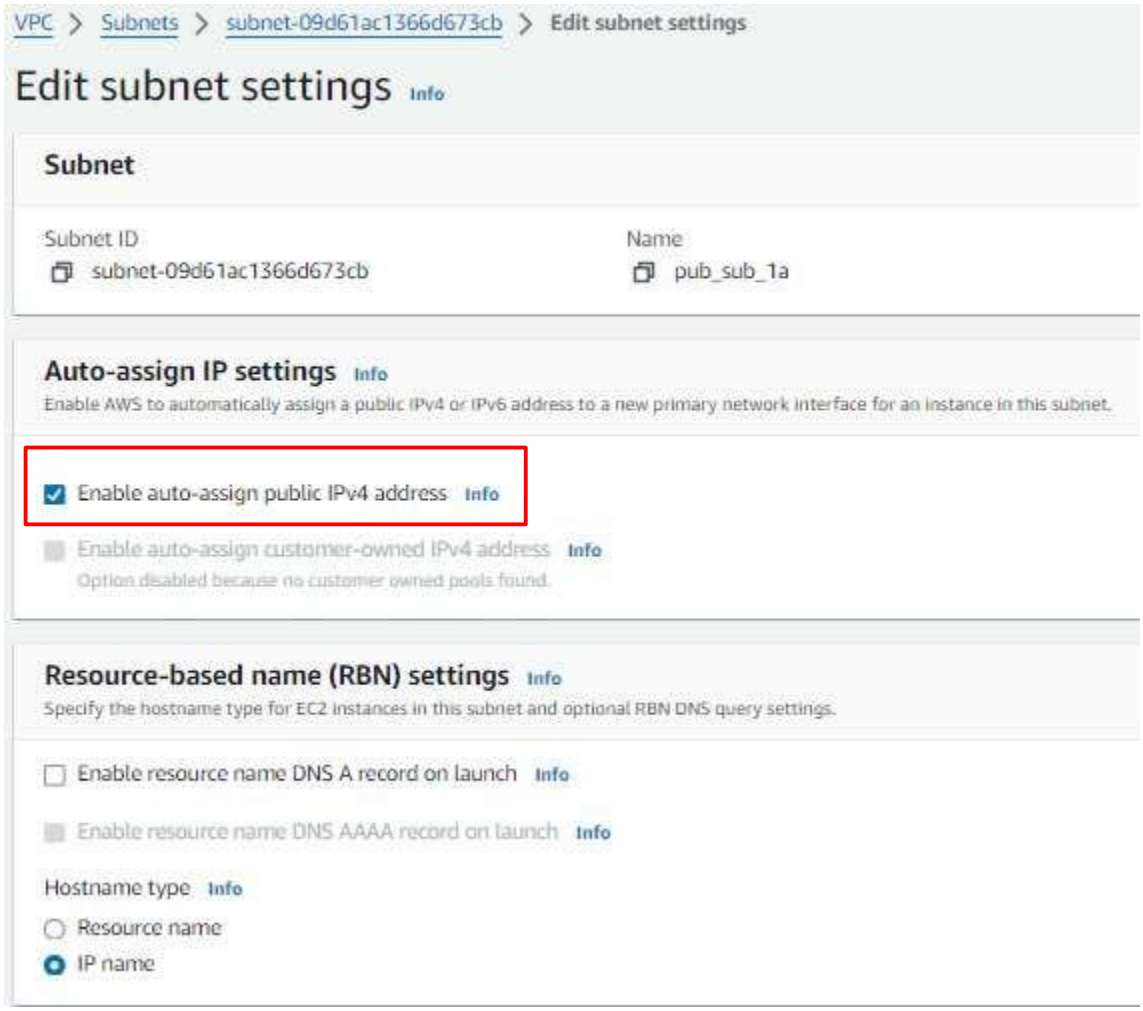


917396627149-Madhukiran
devopstraininghub@gmail.com

Please go to the subnet page and select the public subnet and click on the action button and then choose the Edit subnet setting button from the drop-down list.



Here you have to mark right on **Enable public assign public IPV4 address**. And then click on the save button

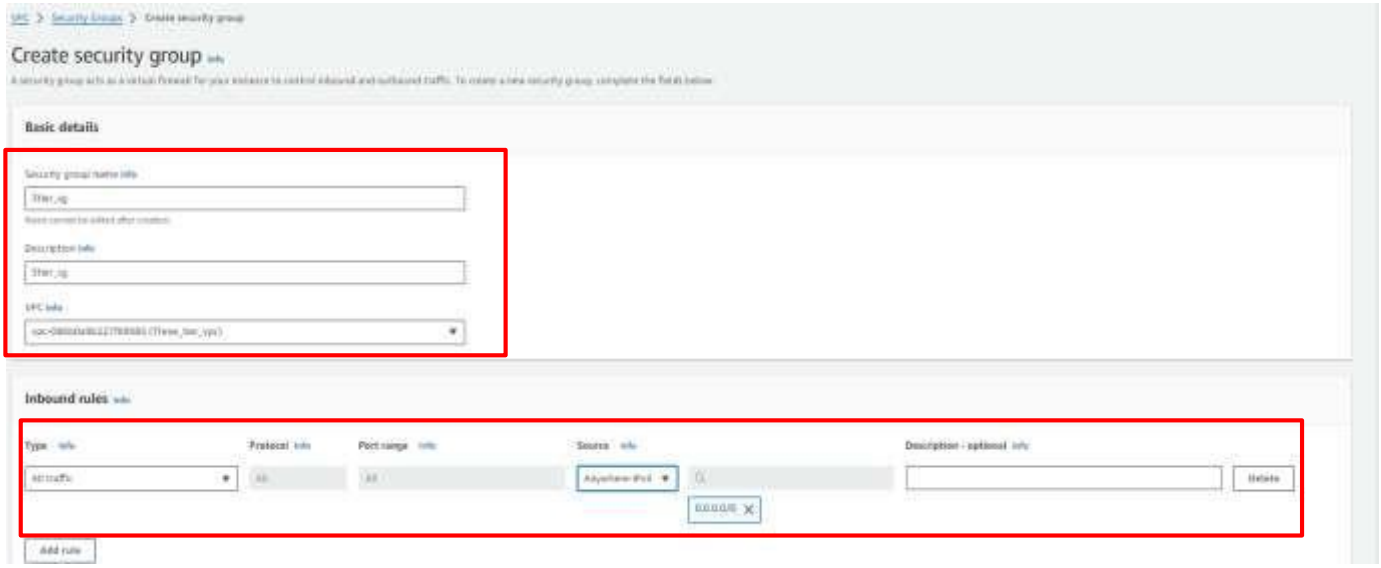


Same for pub_sub_2b, **Enable public assign public IPV4 address**.

Security Groups (SG):

Security groups are very essential part of the infrastructure. Because it can secure the resources in the cloud. SGs are a kind of firewall that allow or block incoming and outgoing traffic. SGs are applied to the resources like ALB, ec2, rds, etc. One resource can have more than one SG.

To create SG, click on the security groups tab on the left panel and here you will see the Security Groups button. Note that SGs are specific with VPC. So we can't use SG which is created in a different VPC. So when you create SG please make sure that you choose the right VPC. Click on the create security button on the top right corner.



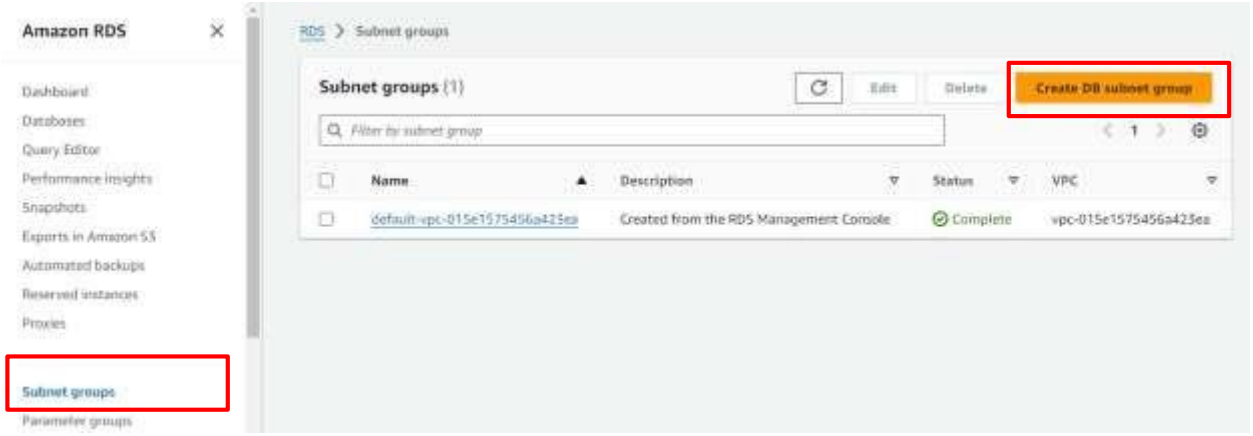
And here our SG setups are complete now.

- Same setup for the secondary region. Any region, it is **Oregon (us-west-2).**

RDS and Route 53:

Now we are going to set up a database for our application. And for that, we are going to utilize the RDS service of AWS. So let's head over to the RDS dashboard. Just search RDS in the AWS console. And click on the service.

Now first we need to set up a subnet group. It specifies in which subnet and Availability zone out database instance will be created. So click on the subnet group button on the left panel. And click on the button Create database subnet group which is in the middle of the web page.



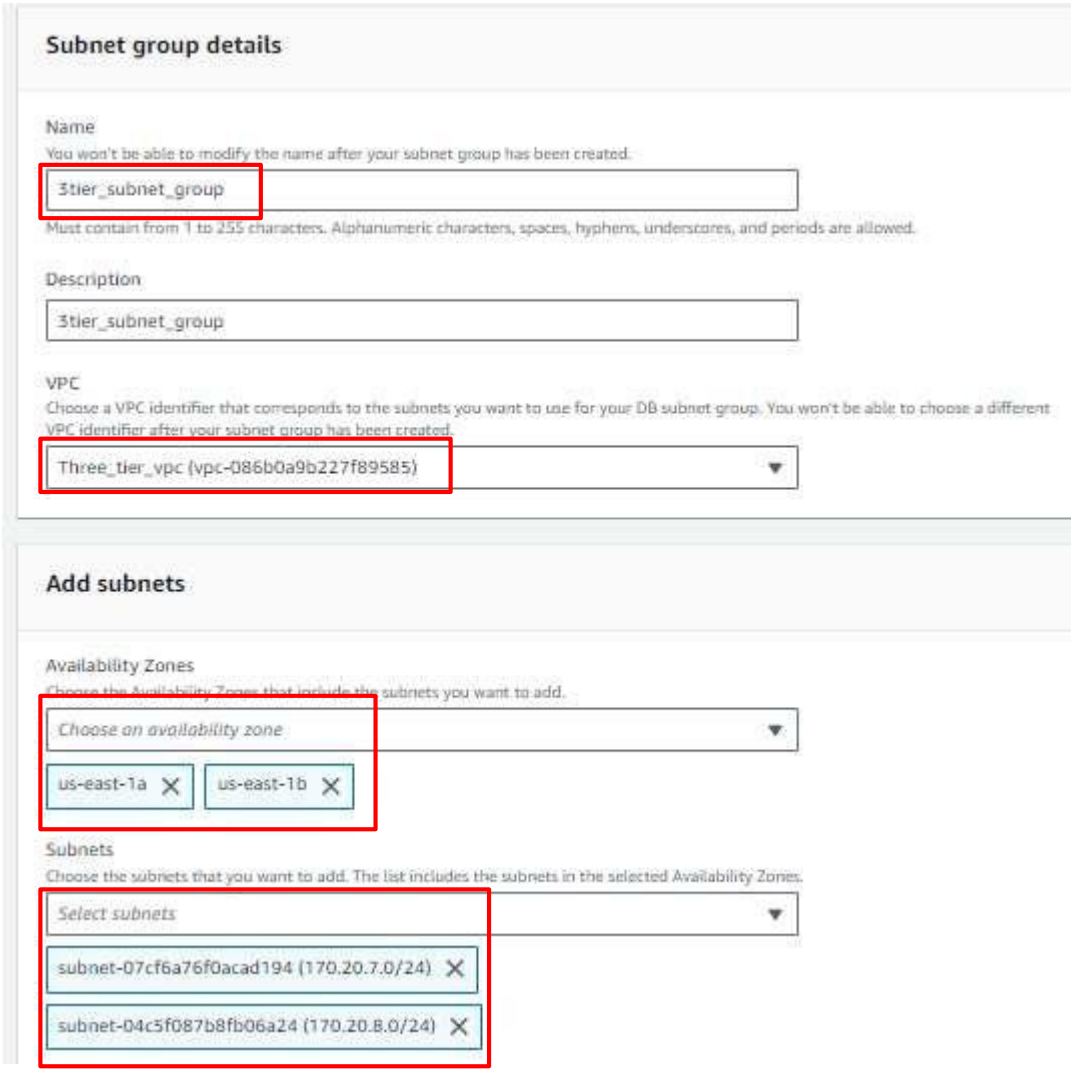
Here we can configure our VPC, subnet, and availability zone.

Give any name to your subnet but make sure you select the correct VPC.

And select Azs **us-east-1a** and **us-east-2b**.

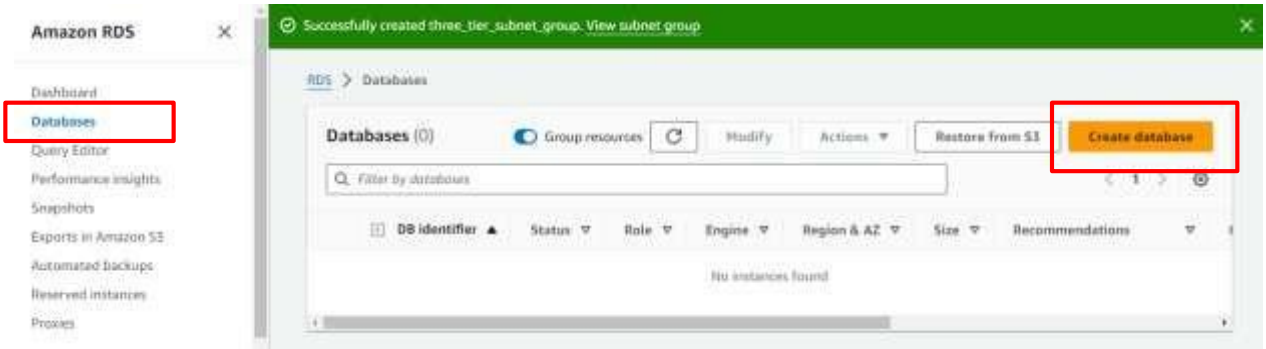
According to the architecture that I have shown you, our database will be in private subnet **pri-sub-7a** and **pri-sub-8b**.

So please select as I have shown in the below figure. And then click on the create button.



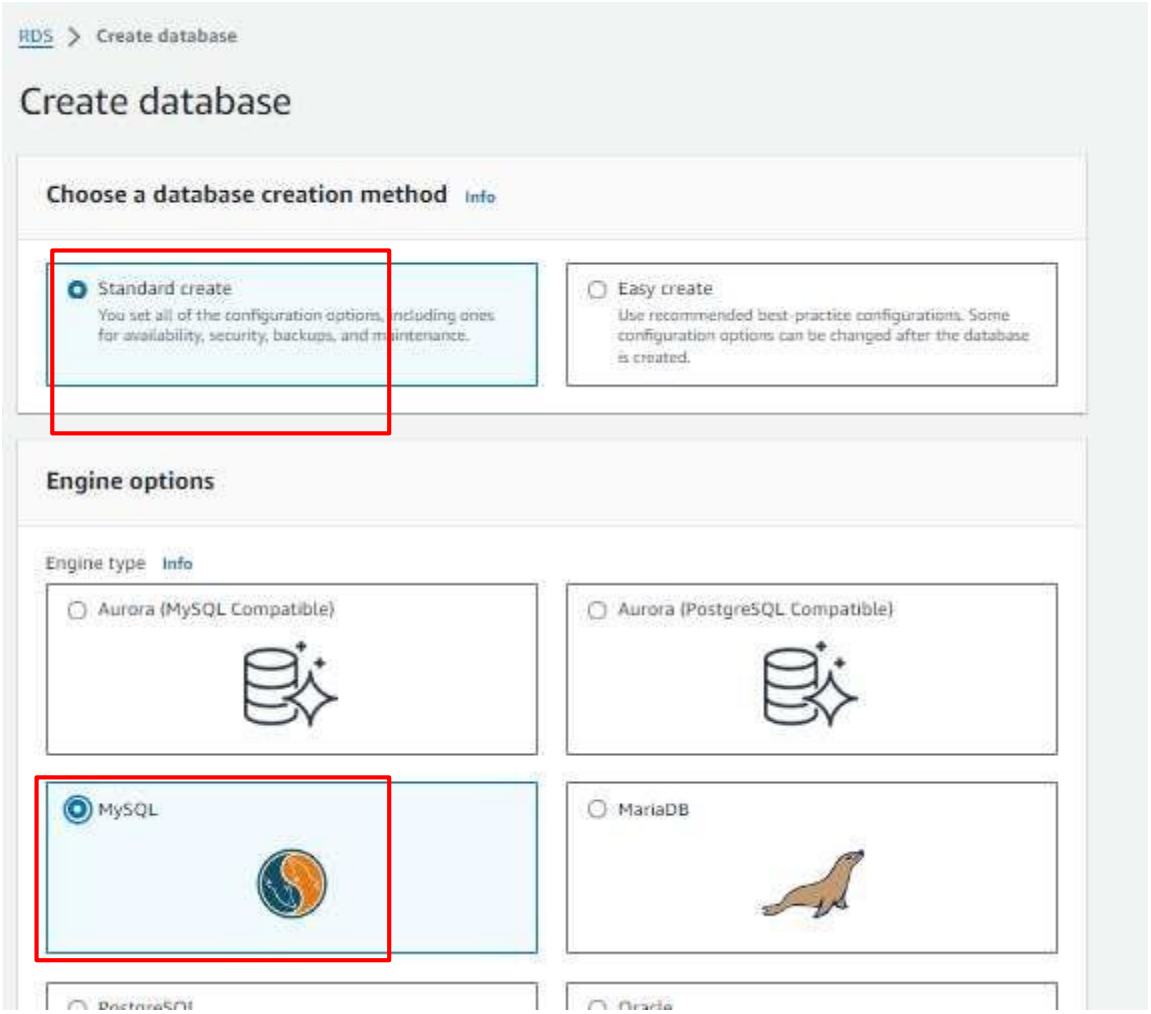
Create subnet group in Oregon region also same like above steps

Create a database. So click on the database button on the left panel and then click on the created database button.



On this page, we can configure our database. Select standard create. Select MySQL in the engine option because our application runs on MySQL database Furthermore, you can select the engine

version my application is compatible with MySQL version.



Scroll down, and select Dev/test as template.

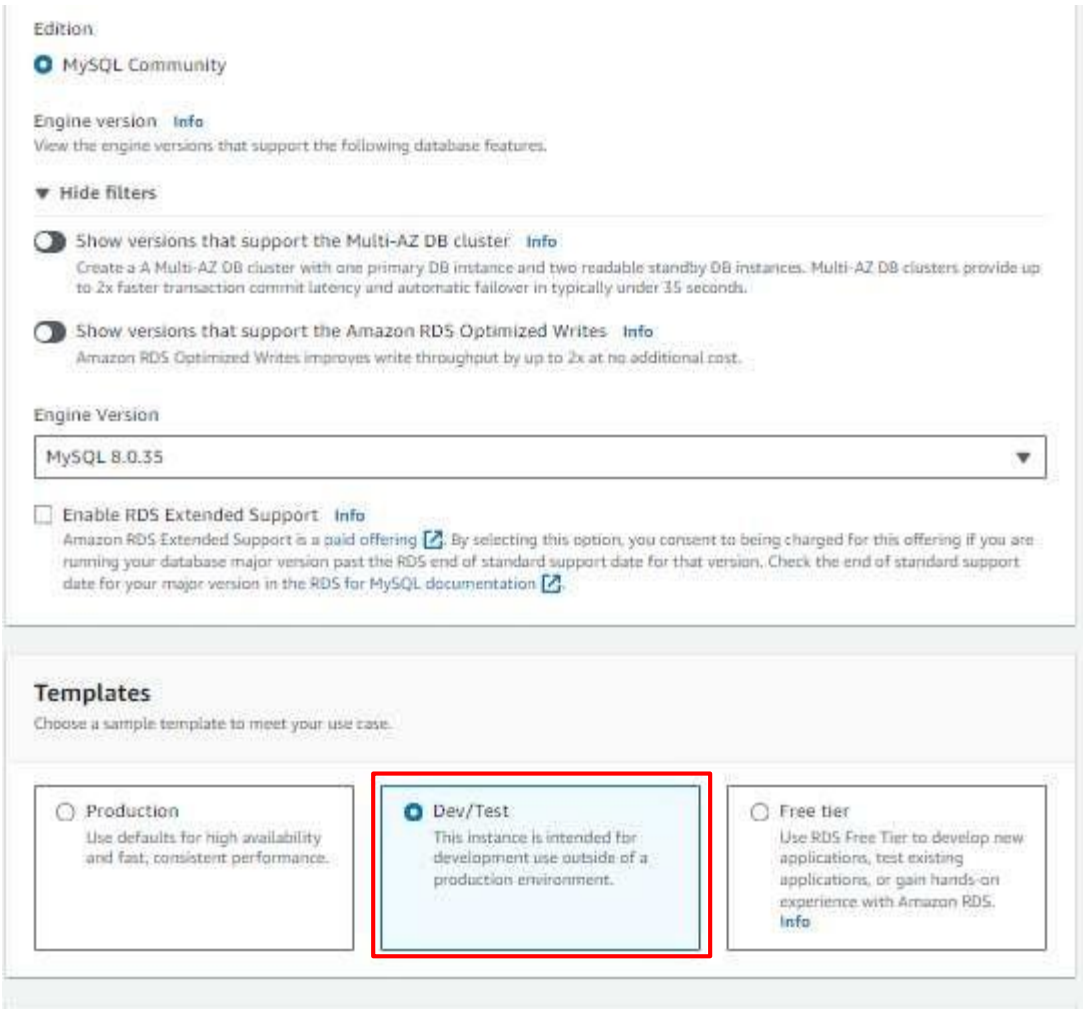
If you select the free tier then you won't be able to deploy RDS in a multi-availability zone.

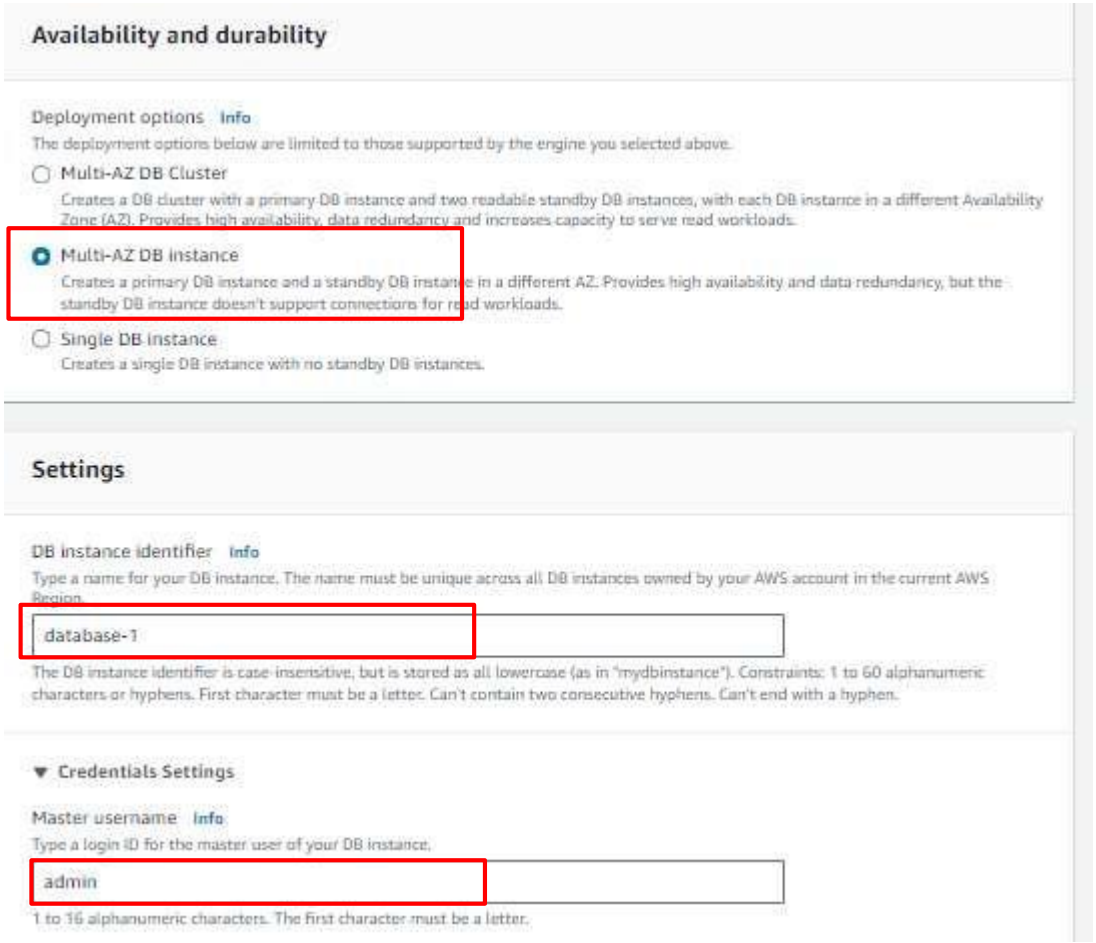
Select Multi-AZ DB instance from availability and durability option.

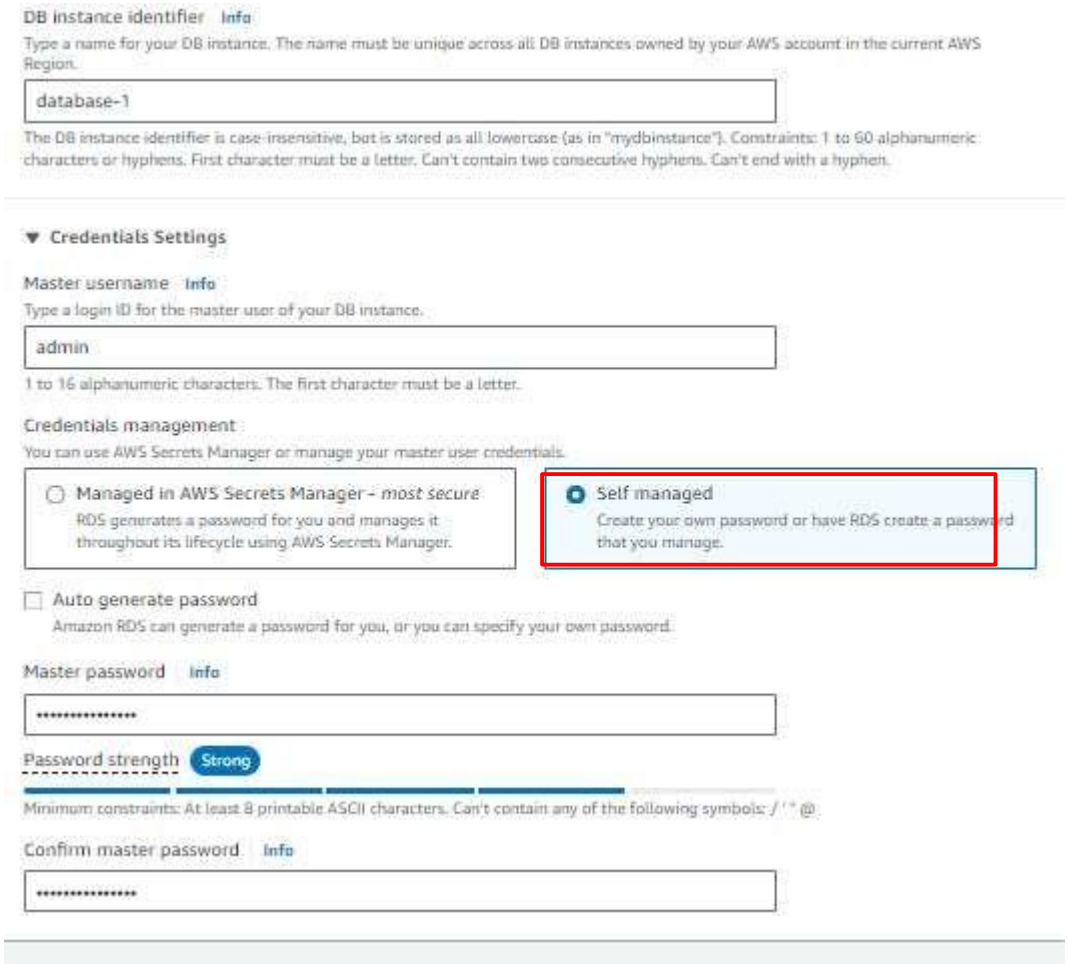
In settings give any name to your database.

In the credential setting give the username of the database in the Master Username field and give the password in the Master password field. And then confirm the password below.

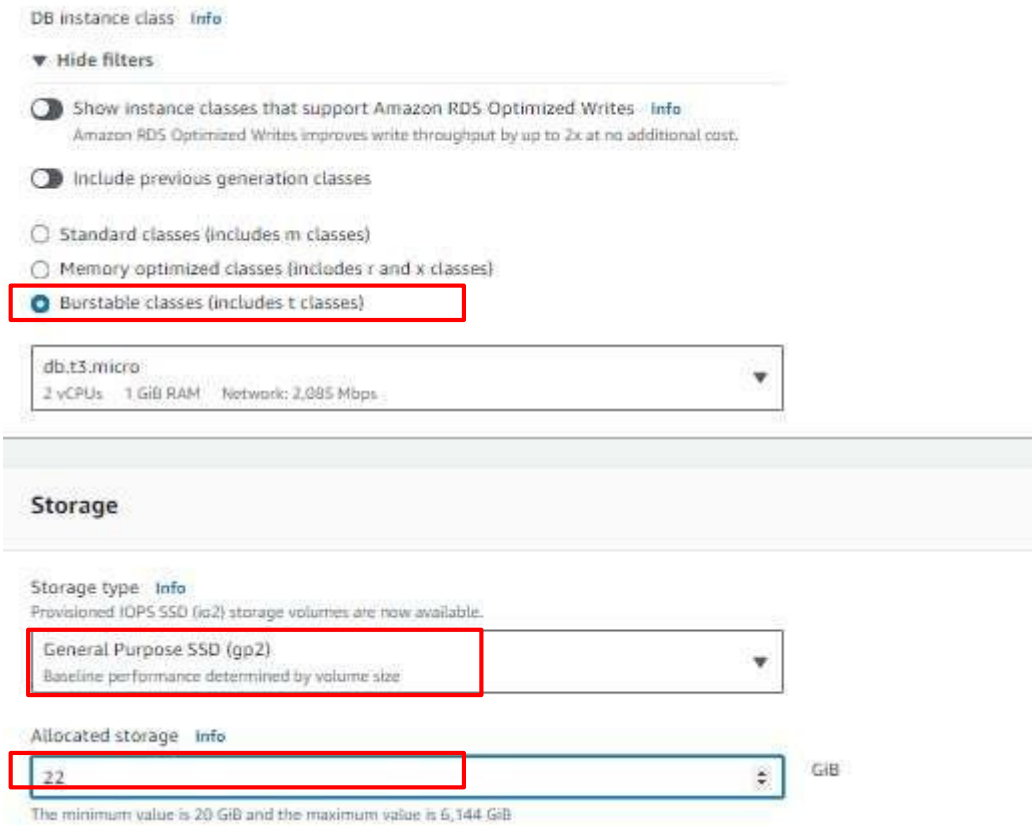
Please do remember your username and password.


Edition
MySQL Community
Engine version Info
View the engine versions that support the following database features.
Hide filters
Show versions that support the Multi-AZ DB cluster Info
Create a A Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.
Show versions that support the Amazon RDS Optimized Writes Info
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.
Engine Version
MySQL 8.0.35
Enable RDS Extended Support Info
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the RDS for MySQL documentation.
Templates
Choose a sample template to meet your use case.
Production
Use defaults for high availability and fast, consistent performance.
Dev/Test
This instance is intended for development use outside of a production environment.
Free tier
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.
Info

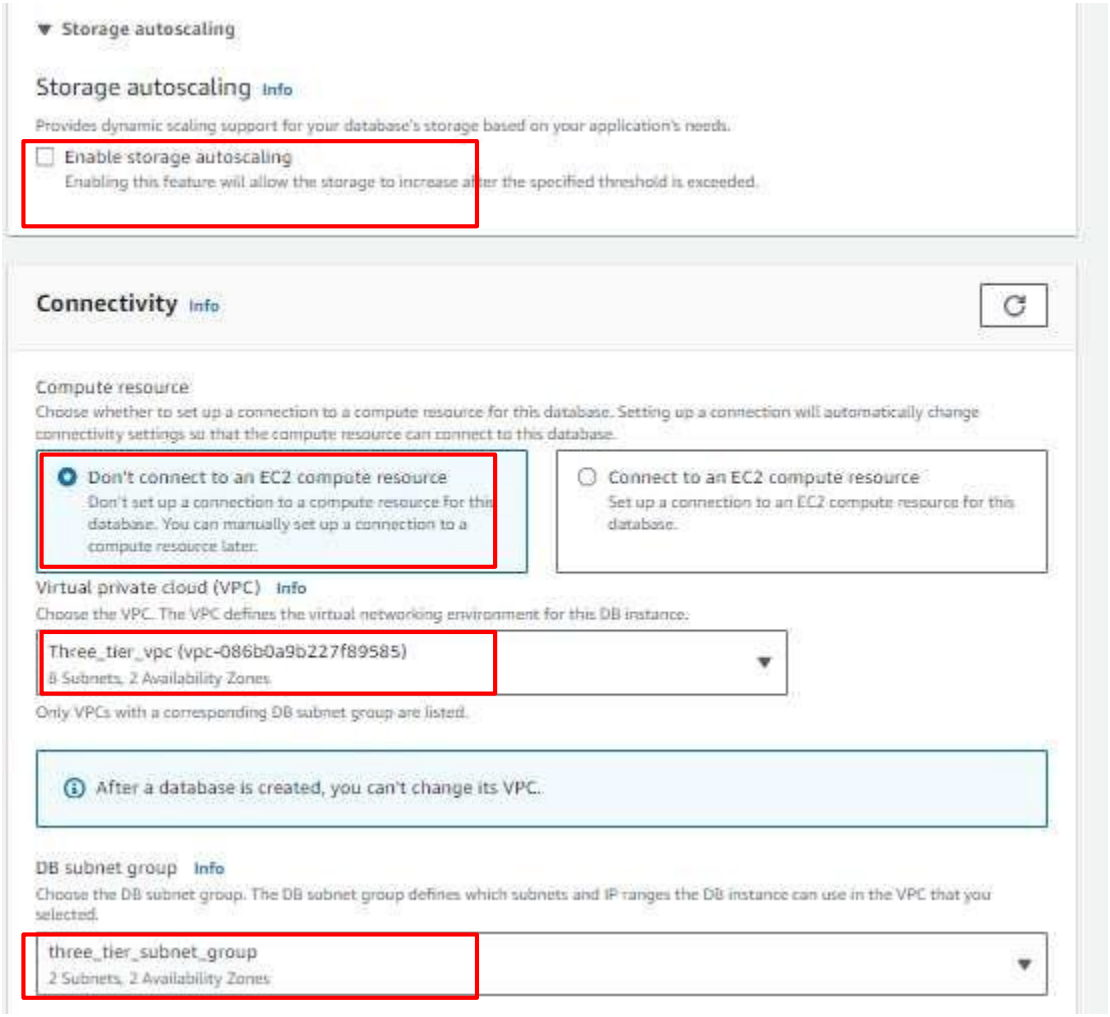

Availability and durability
Deployment options Info
The deployment options below are limited to those supported by the engine you selected above.
Multi-AZ DB Cluster
Creates a DB cluster with a primary DB instance and two readable standby DB instances, with each DB instance in a different Availability Zone (AZ). Provides high availability, data redundancy and increases capacity to serve read workloads.
Multi-AZ DB instance
Creates a primary DB instance and a standby DB instance in a different AZ. Provides high availability and data redundancy, but the standby DB instance doesn't support connections for read workloads.
Single DB instance
Creates a single DB instance with no standby DB instances.
Settings
DB instance identifier Info
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.
database-1
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 60 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.
Credentials Settings
Master username Info
Type a login ID for the master user of your DB instance.
admin
1 to 16 alphanumeric characters. The first character must be a letter.



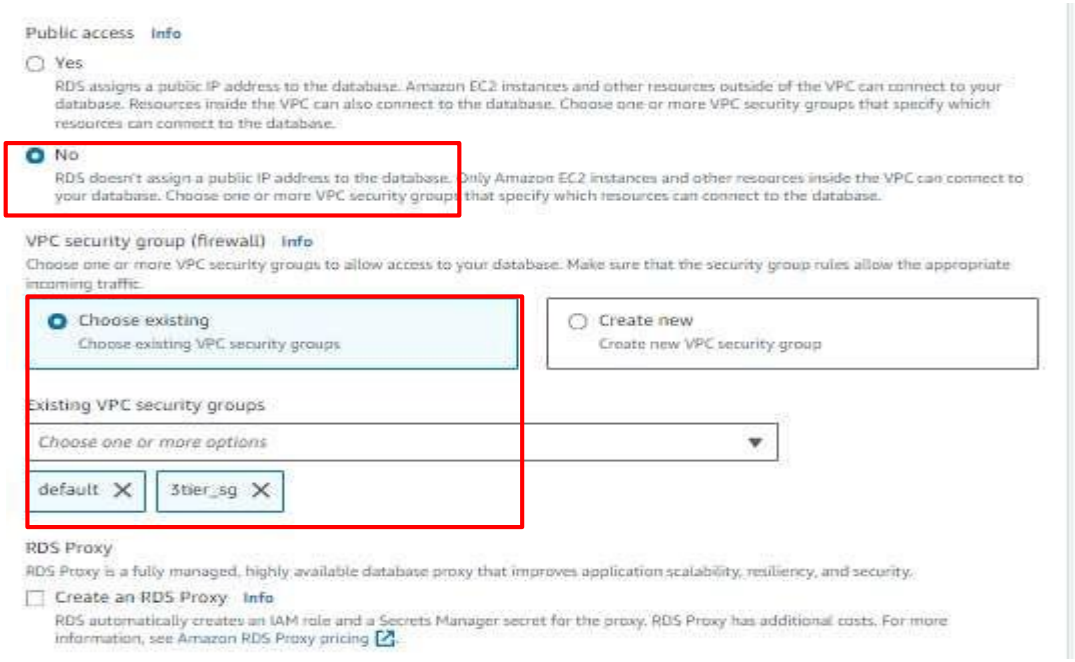
Again scroll down, select Brustable class in the instance setting and select the instance type. Actually, it depends on your application uses. But for learning purposes, I am selecting t3.micro. Now in storage type select General purpose (GP2) and allocate 22 GiB for database. Please uncheck the auto-scaling option to keep our costs low. And In the connectivity option please select the option according below screenshot.



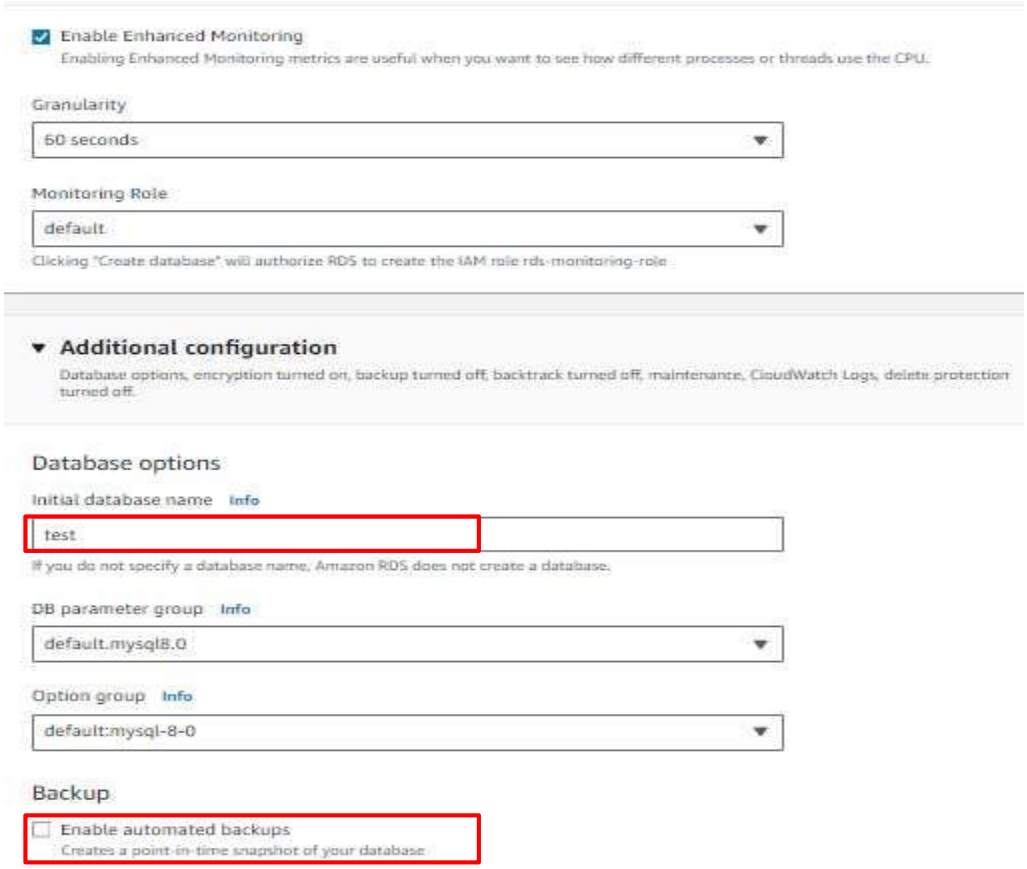
917396627149-Madhukiran
devopstraininghub@gmail.com



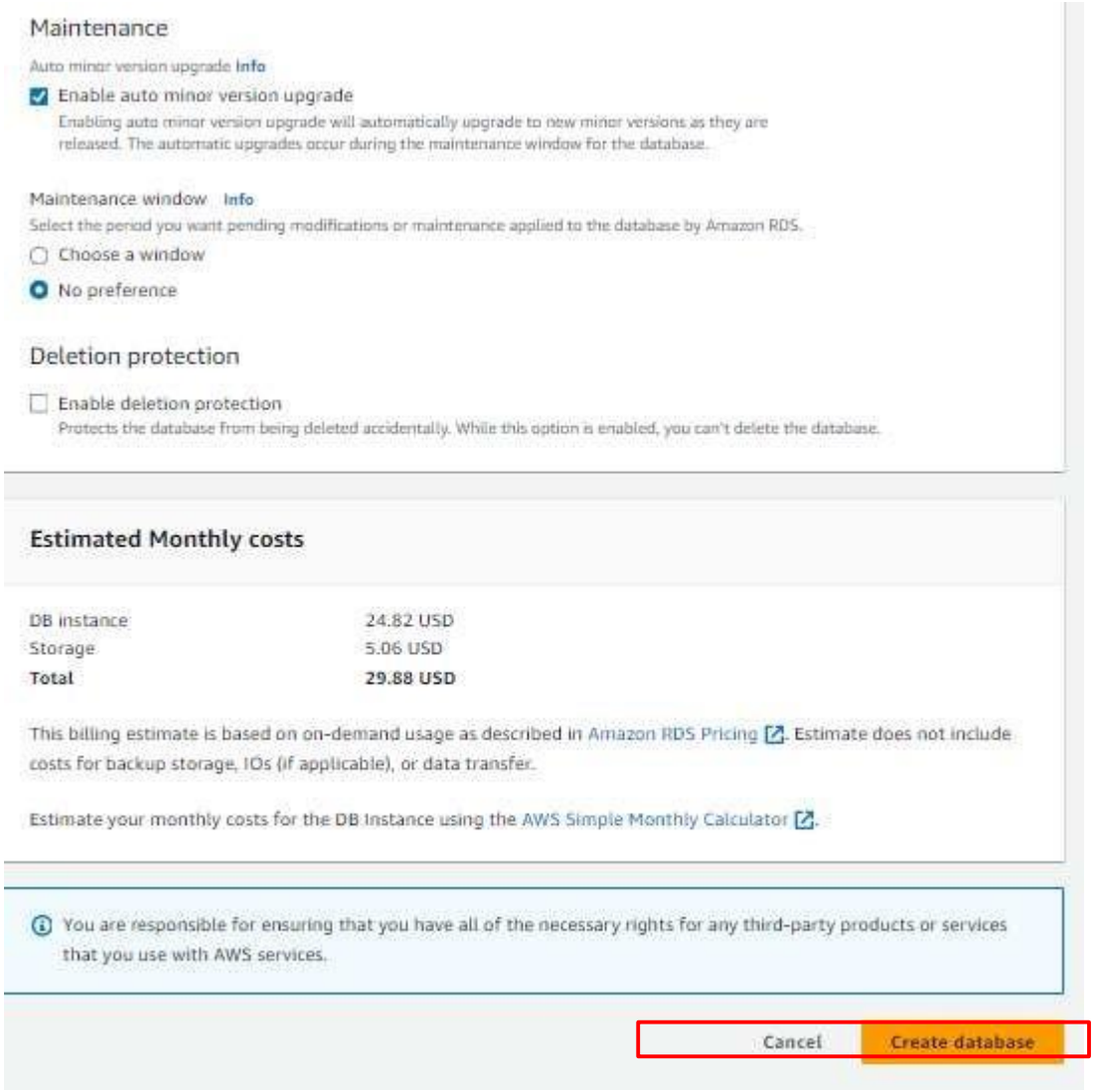
In VPC, select VPC that we created earlier and in DB subnet group select the group that we just created, In the public access option please select No, choose existing security, and select security group **book-rds-db**.



Scroll down, click on Additional Configuration, and in the database option give the name **test** because we need a database with the name of the **test** in the application.

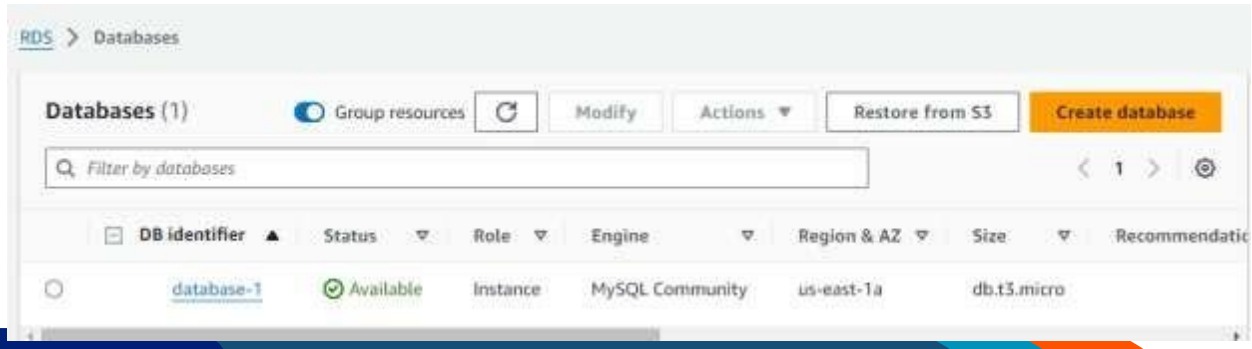


Scroll down, mark on enable encryption checkbox to make the database bit more secure, and click on Create database button below.



Note: *RDS take 15-20 minute because it creates a database*

After your database is completely ready and you see the status Available



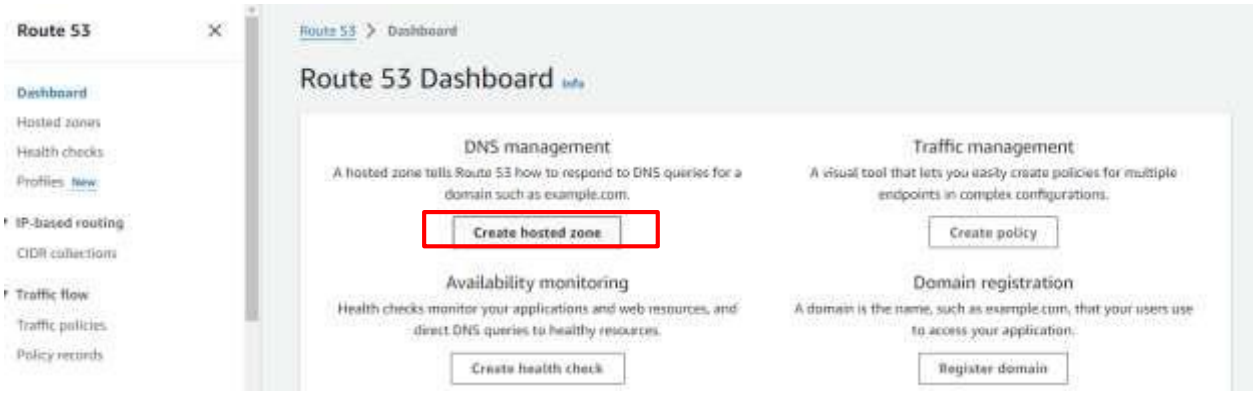
After your database is completely ready and you see the status Available then select the database and click on the Action button. There you see the drop-down list. Please click on created read-replica.
This page is similar to creating a database. In the AWS region select the region where you want to create the read replica. It is **Oregon (us-west-2)**. Give a name to your read replica, and select all the necessary configurations that we did before while creating the database.

Note: we can't write anything into a read replica. It is just read-only database. So when a disaster happens we just have to promote read replica so that it becomes the primary database in that region.

Route 53

Now we are going to utilize route 53 services and create private hosted zone. So head over to Route 53. Type route 53 in the AWS console. And click on the service.

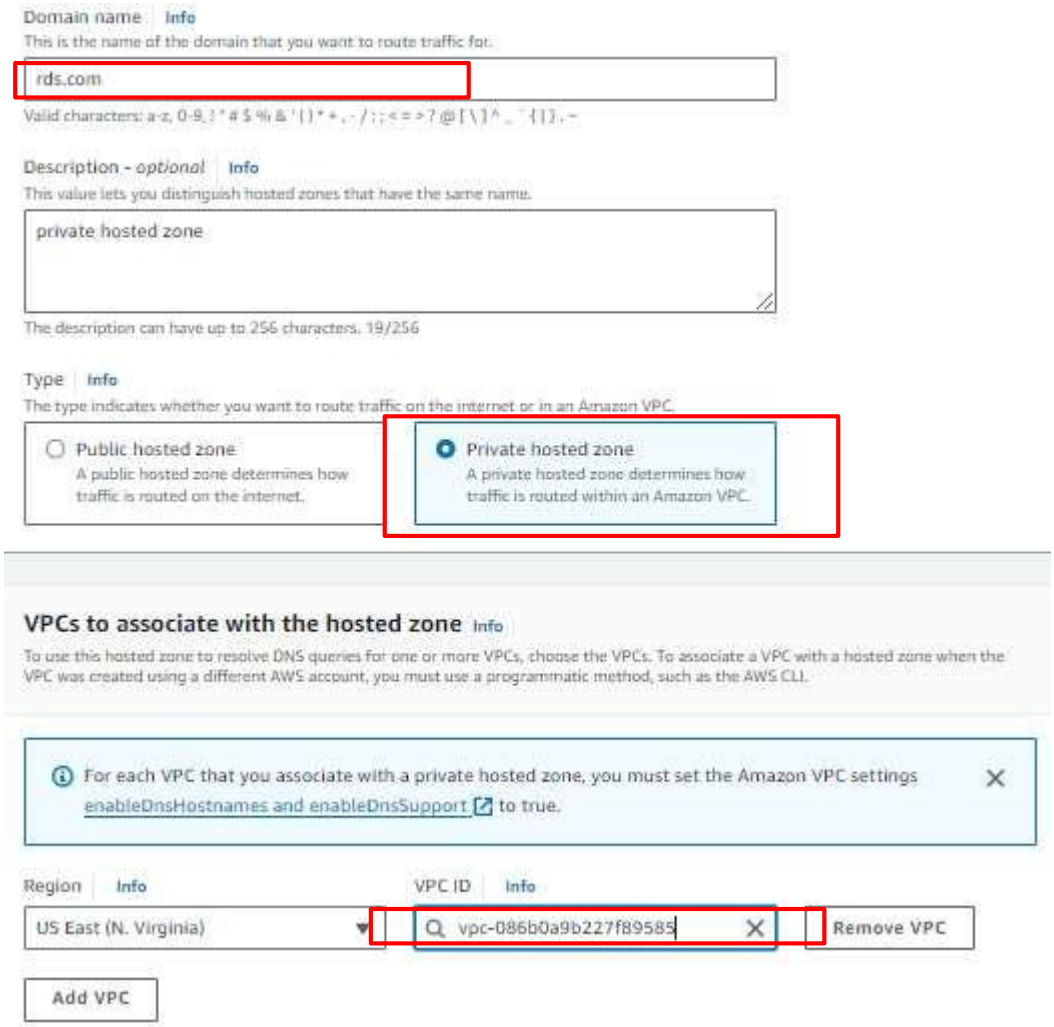
Firstly, we are going create a hosted zone. Click on the created hosted zone



Give any domain name because anyhow it will be private hosted zone but it would be great if you give the name same as mine (**rds.com**).

Please select the private hosted zone and Select the region. In my case, it is **us-east-1**. And then select VPC ID.

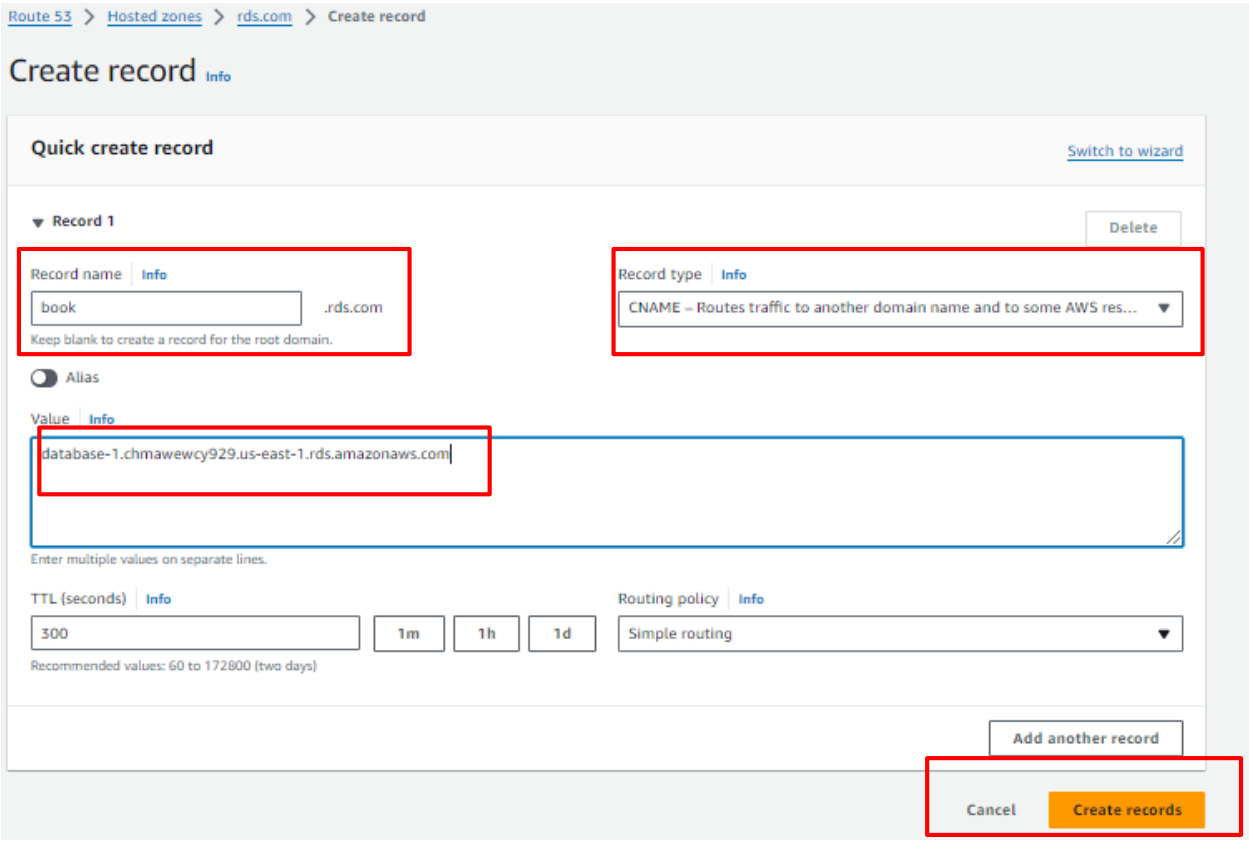
Make sure you select VPC that we created earlier. Because this hosted zone will resolve the record only in specified VPC. And then click on the Create hosted zone.



Now we are going to create a Record that points to our RDS instance which is in **us-east-1**. So click on create record button on the top right corner.

Here type book in the record name field. In the record type select CNAME. In the value field paste **endpoint of the RDS which is in us-east-1**. Then click on the defined record button.

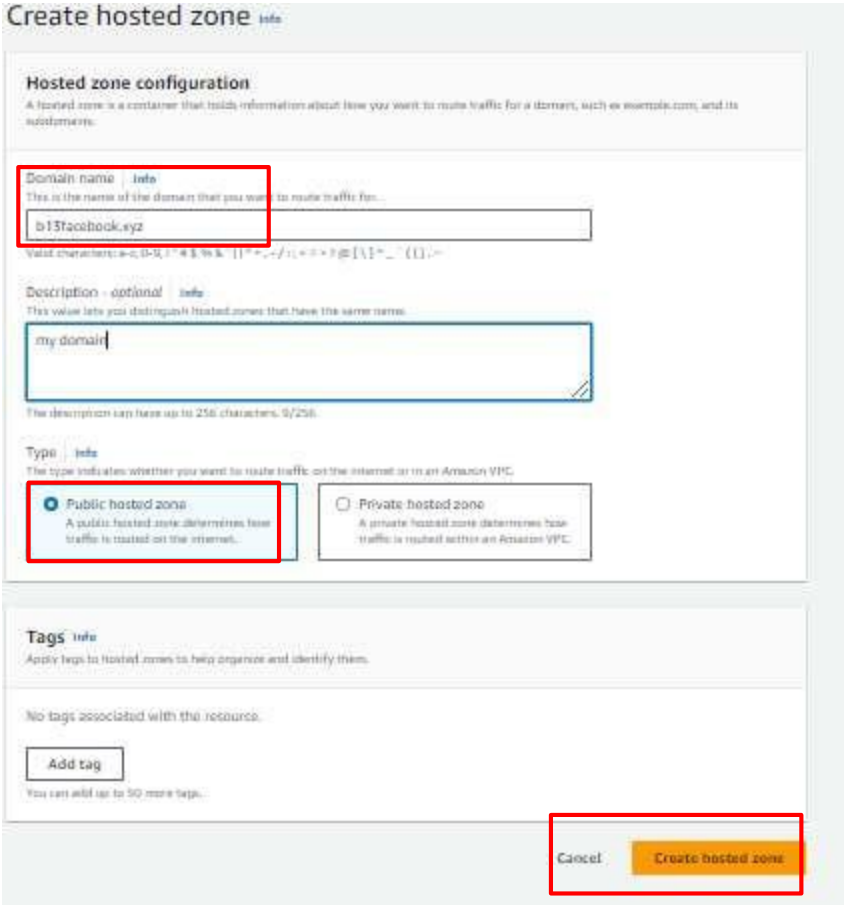
Click on create record button.



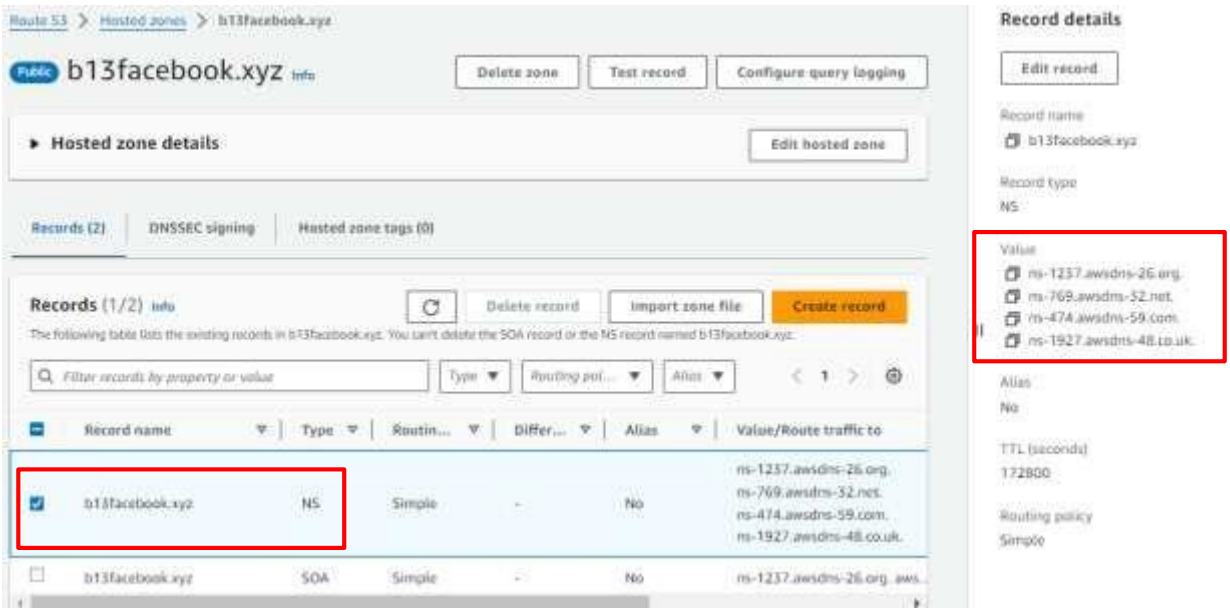
- Now we are going to create a new hosted zone with the same name. but for **disaster recovery region** and that is **us-west-2 (Oregon)**. While creating hosted zone please keep in mind that you need to choose the **us-west-2** region and select VPC that you have created in the **the us-west-2** region.Nad follow same steps

Creating our domain Host:

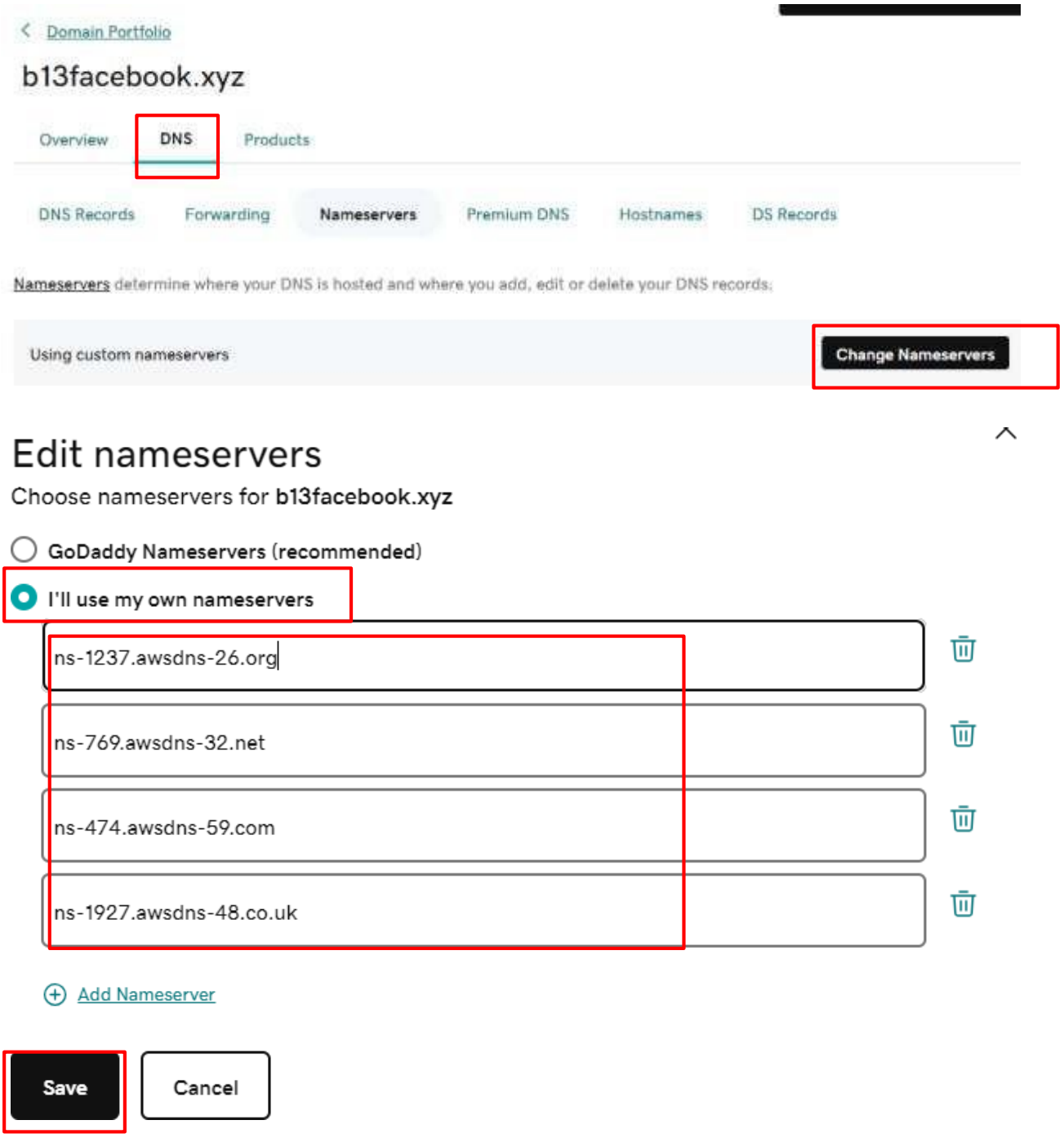
1. **Navigate to Hosted Zones** in the Route 53 dashboard.
2. **Click on "Create Hosted Zone"**.
3. **Enter the Domain Name** (e.g.: **b13facebook.xyz)** you want to associate with the hosted zone.
4. **Select Public Hosted Zone** as the type.
5. **Click on "Create"**.



After creating hosted zone, select the record b13facebook.xyz and copy the values



Add these values to “godaddy” name servers, as shown in below figure.



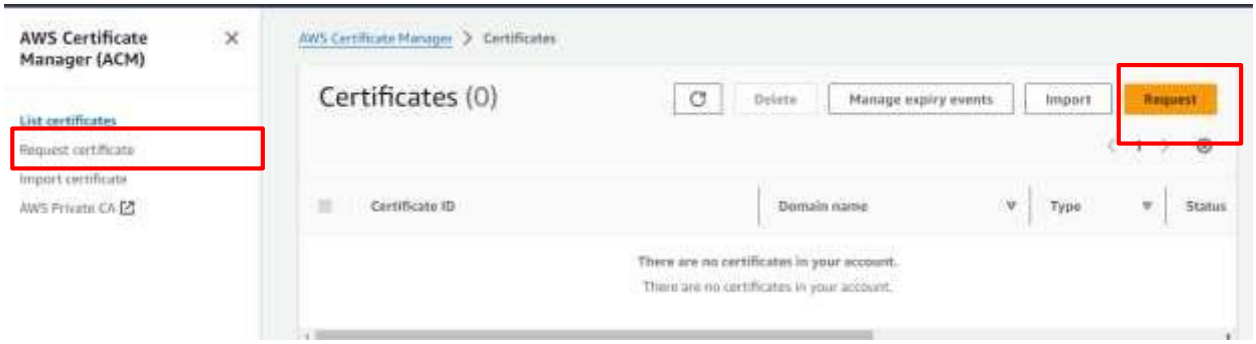
It Takes 5-10mins to create

Certificate Manager

I have the domain name b13facebook.xyz from godaddy in Route 53. Now I am going to use this domain name to create sub domains such as api.b13facebook.xyz and that will resolve **ALB-backend DNS**. Furthermore, we need an SSL certificate so that we can make the connection secure.

So let's head over to ACM (AWS certificate manager).Type certificate manager in the AWS console search bar. And click on the service.

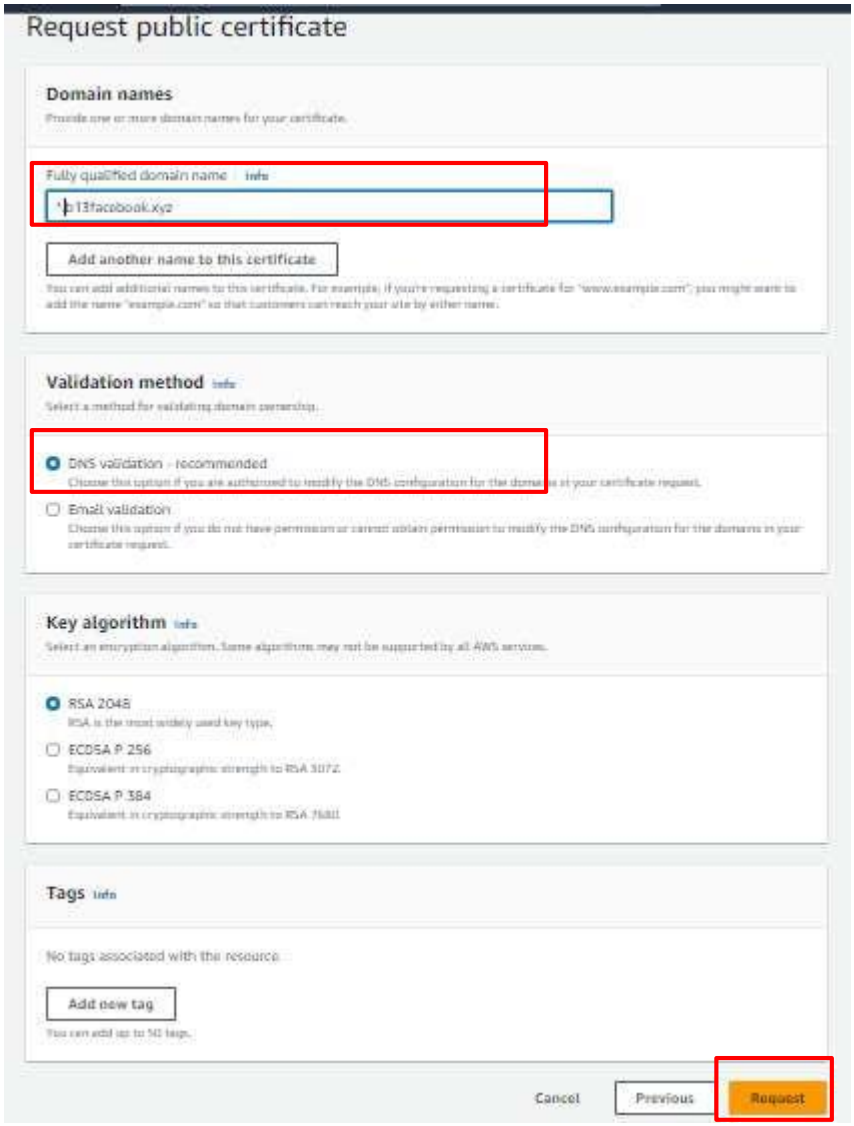
Now click on the list certificates button on the left panel and then click on the request certificate on the top right corner.



Select the option Request the public certificate and click on the next button.

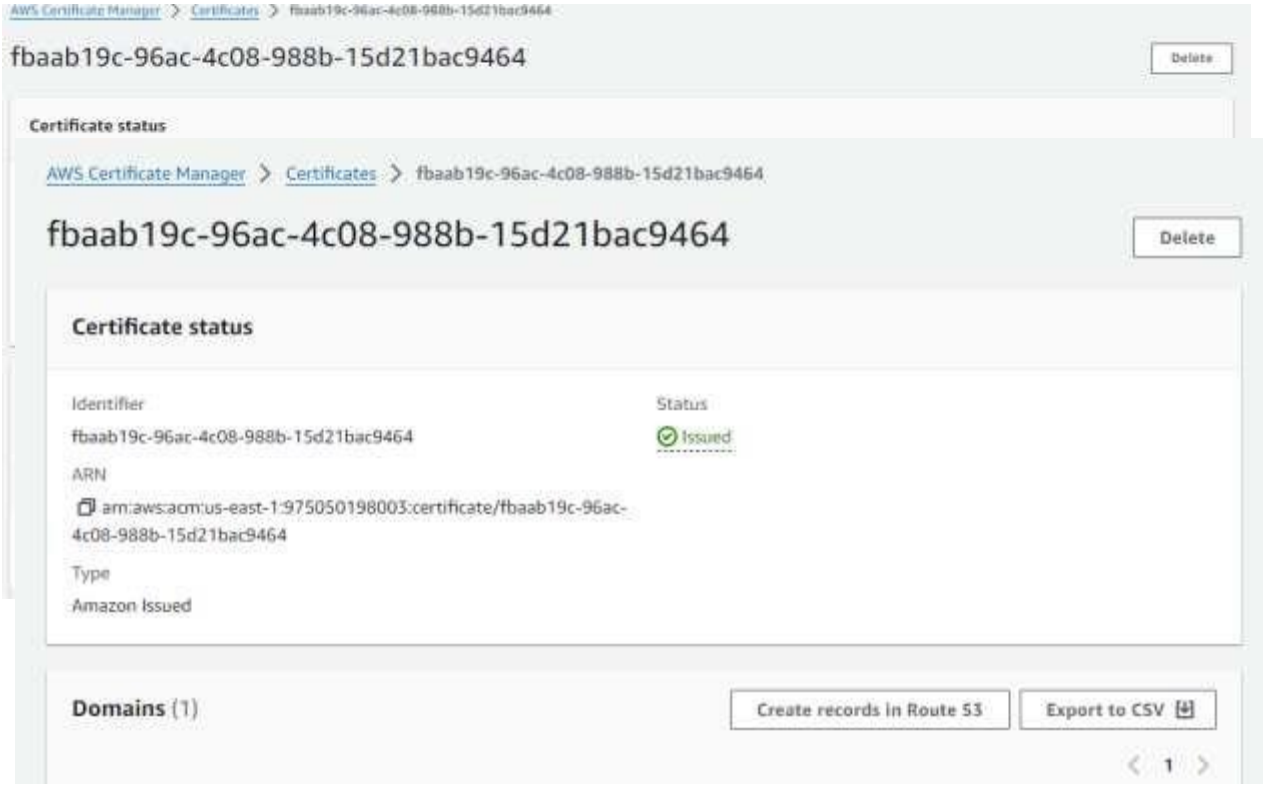
In the domain name field please type ***.Your_Domain_Name.xyz** in my case it is ***.b13facebook.xyz**.

In the validation methods select DNS validation and click on the request certificate.



Here you can see the status pending validation. Now we need to add a **CNAME record** in our domain. If you are not using route 53 then you need to add this CNAME record manually by going to your DOMAIN REREGISTER. And if you are using route 53 then click on the button

create record in route 53 and click on the create record button



And in just a few minutes you will see the status issued.

- Create certificate in another region(us-west-2) also for same domain and follow same steps

917396627149-Madhukiran
devopstraininghub@gmail.com

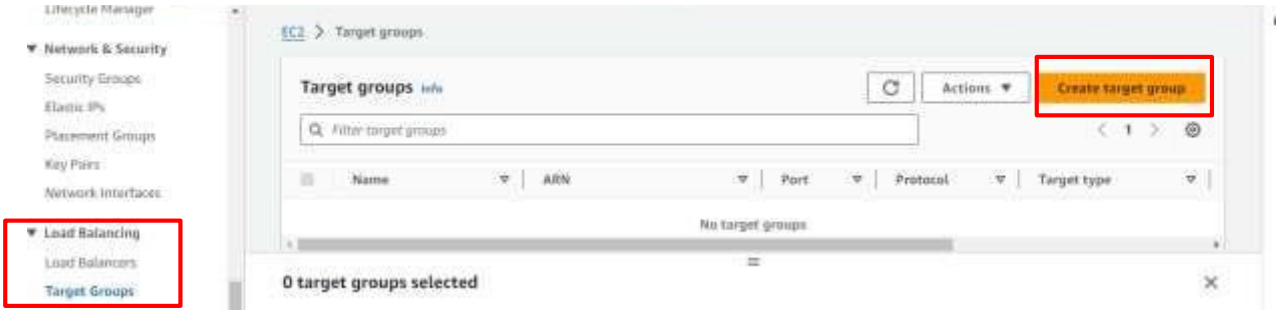
Application Load balancer (ALB) and Route 53

Now it's time to set up an Application load balancer. We need two load balancers, one point to the backend server, and another point to the frontend server.

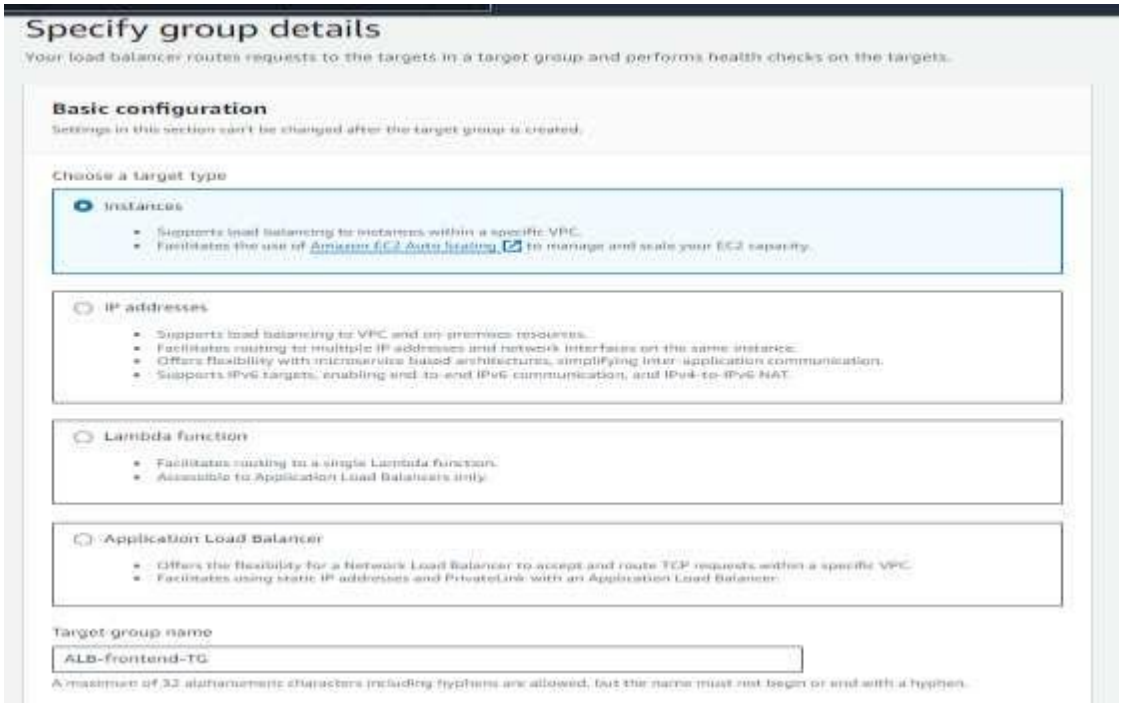
Type ec2 in the AWS console. And click on the EC2 service.

Note: before we created ALB we need to create a Target group (TG). So first we will create TG for ALB-frontend and then create TG for ALB-backend.

Click the target group button on the bottom of the left panel. And click on the create target group button.



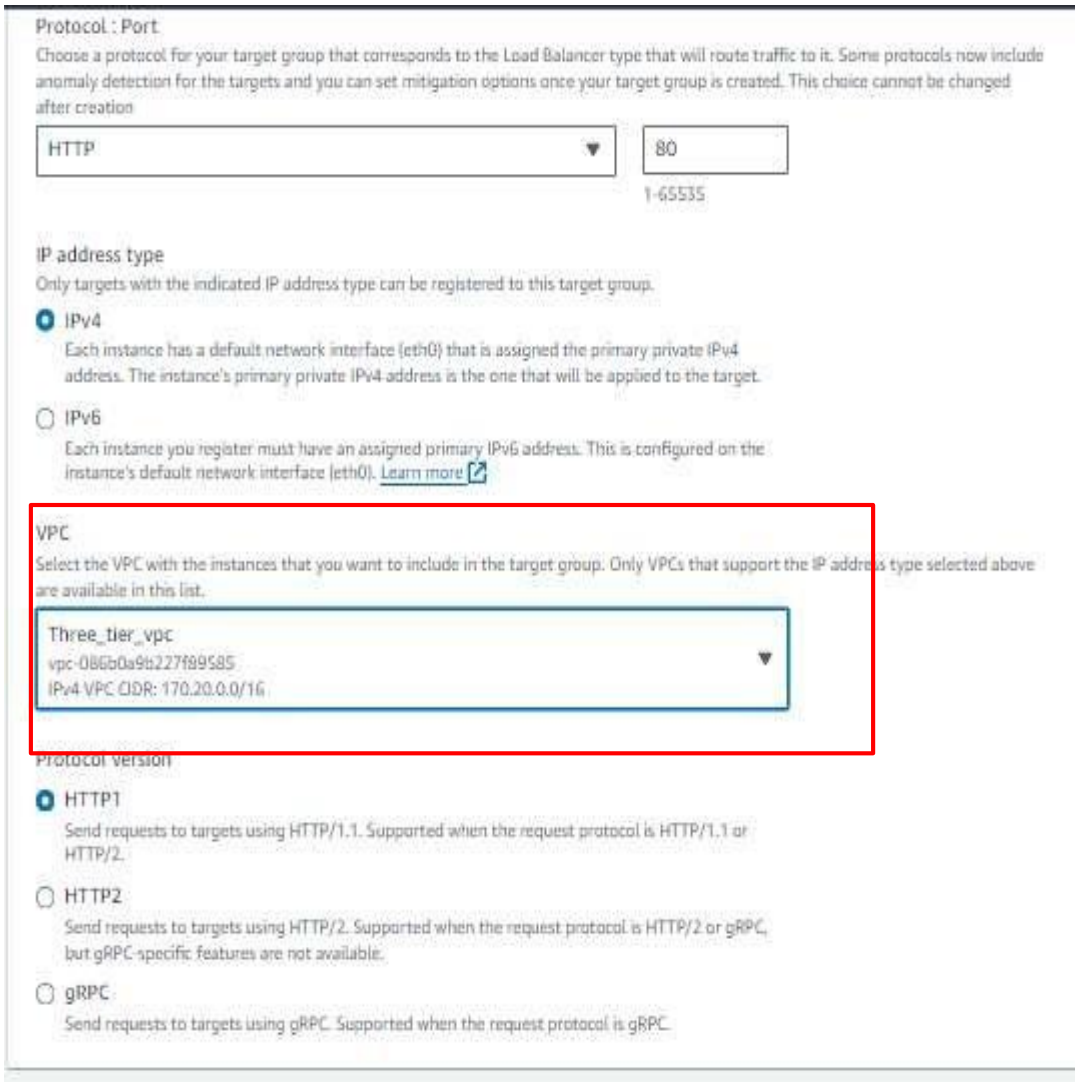
Here we can configure our TG. Select the instance in the target type. You can give any name to TG but try to give some relevant name such as **ALB-frontend-TG** because we are creating TG for ALB-frontend.

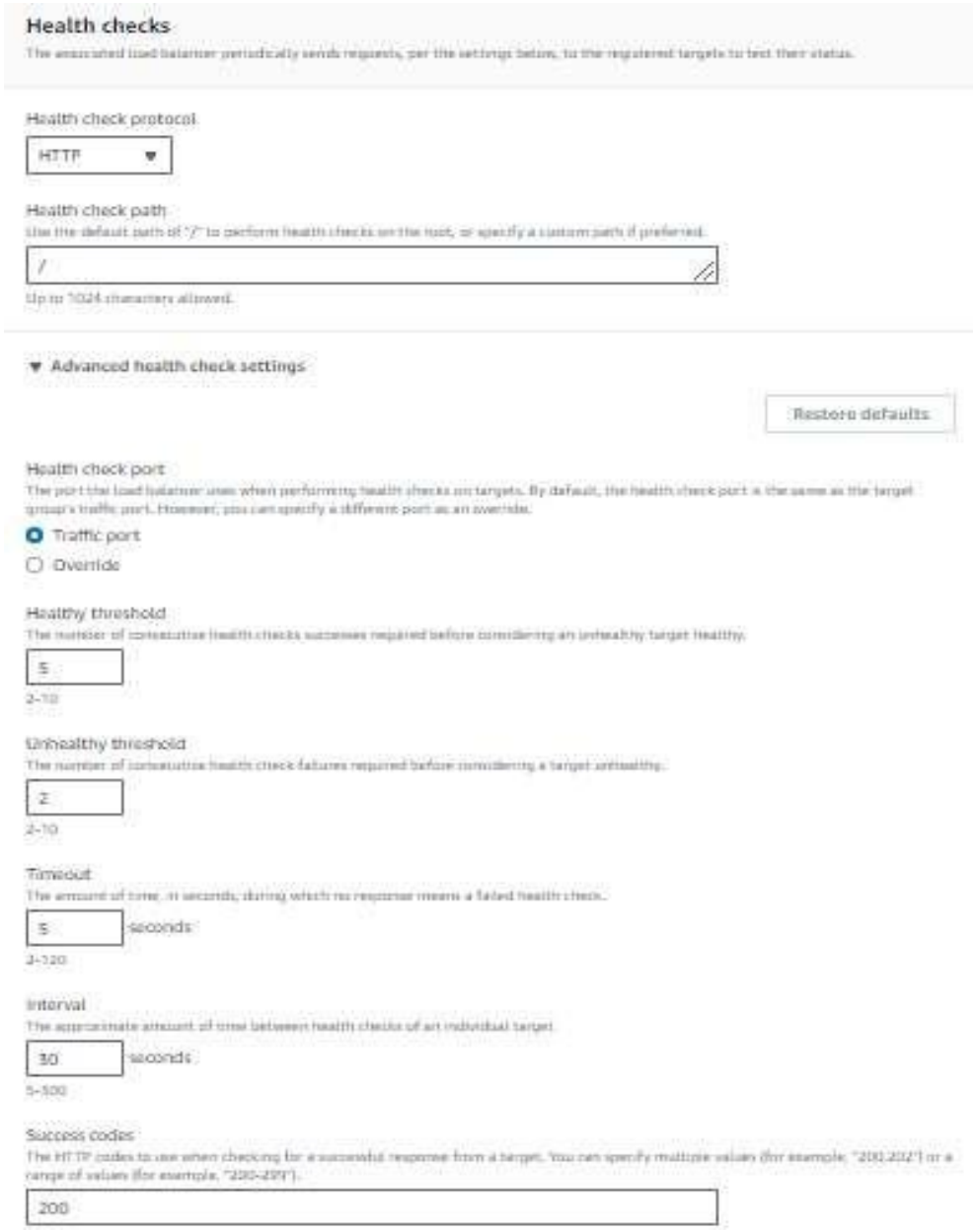


917396627149-Madhukiran
devopstraininghub@gmail.com

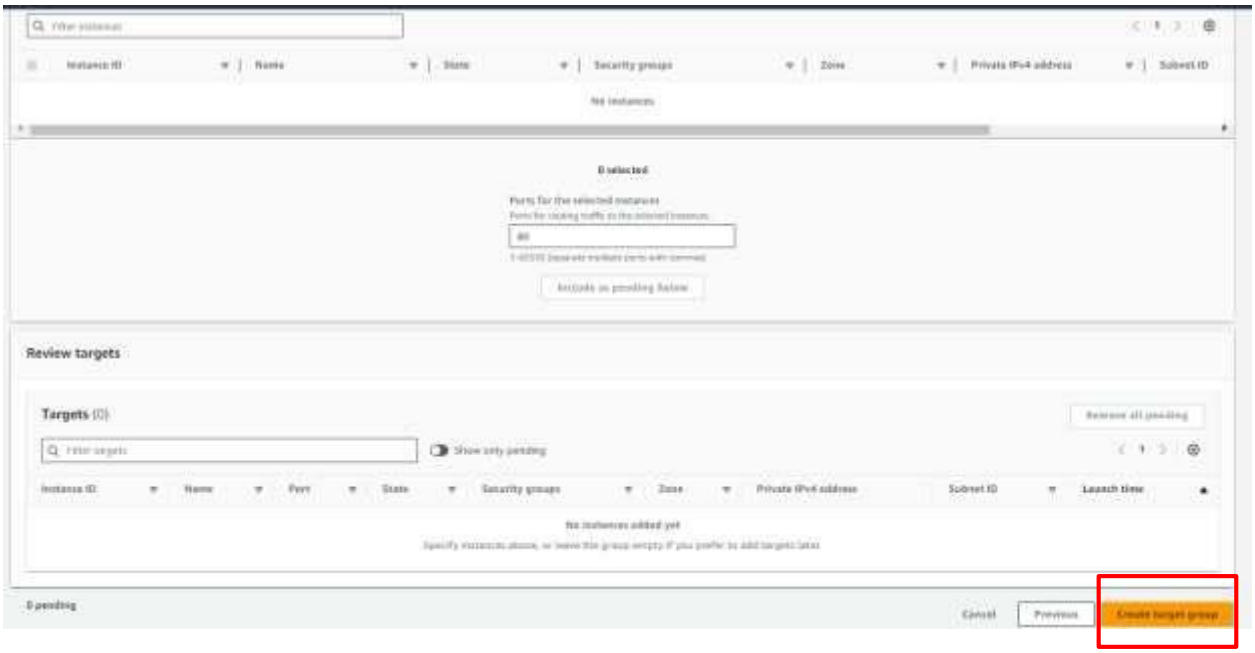
In the VPC section select VPC that we created earlier.

Keep everything as it is, scroll down, and click on the Next button.



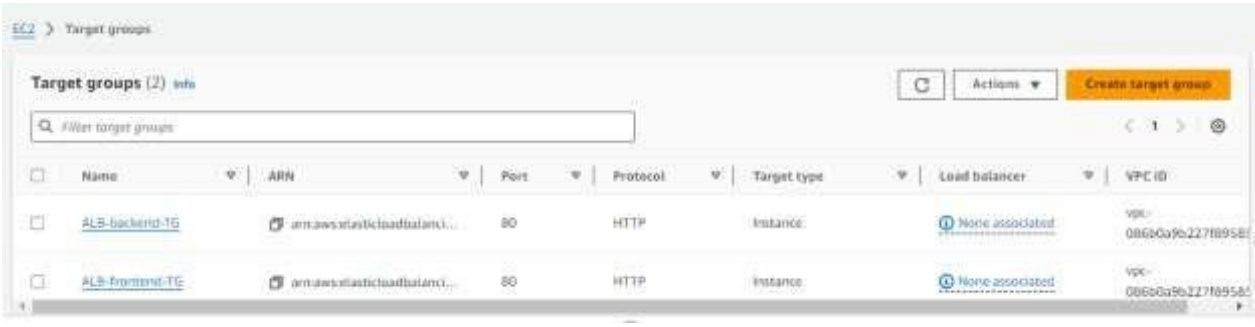


Click on the create target group button.

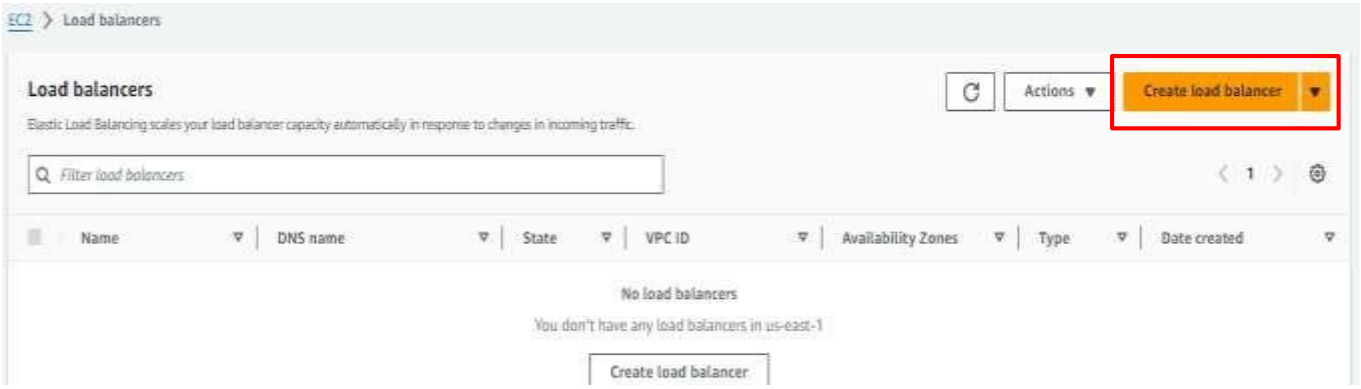


Let's create TG for **ALB-backend**. Follow same as above steps.

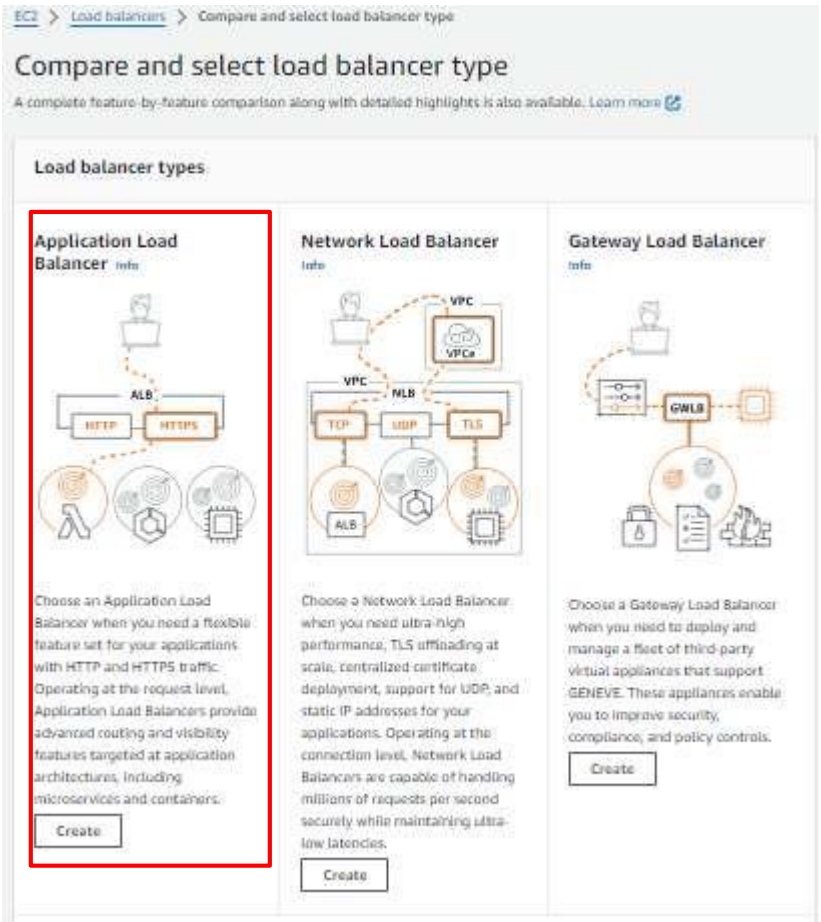
So we have two TG. **ALB-frontend-TG and ALB-backend-TG.**



Now let's associate these TG with the load balancer. So click on the Load Balancer button at the bottom of the left panel and click on the create load balancer button. First, we will create ALB for frontend.

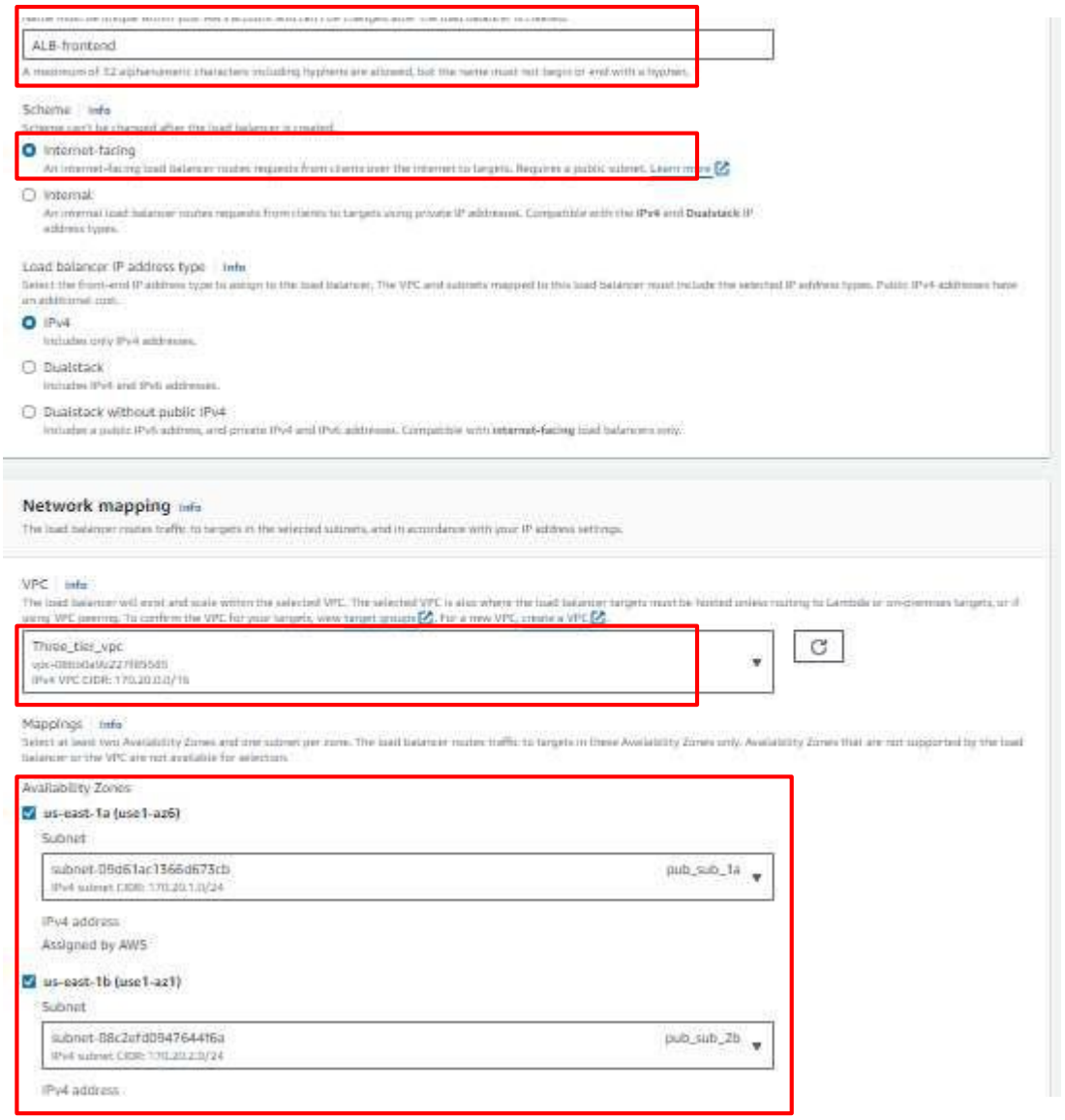


Choose Application load balancer and click on create button.

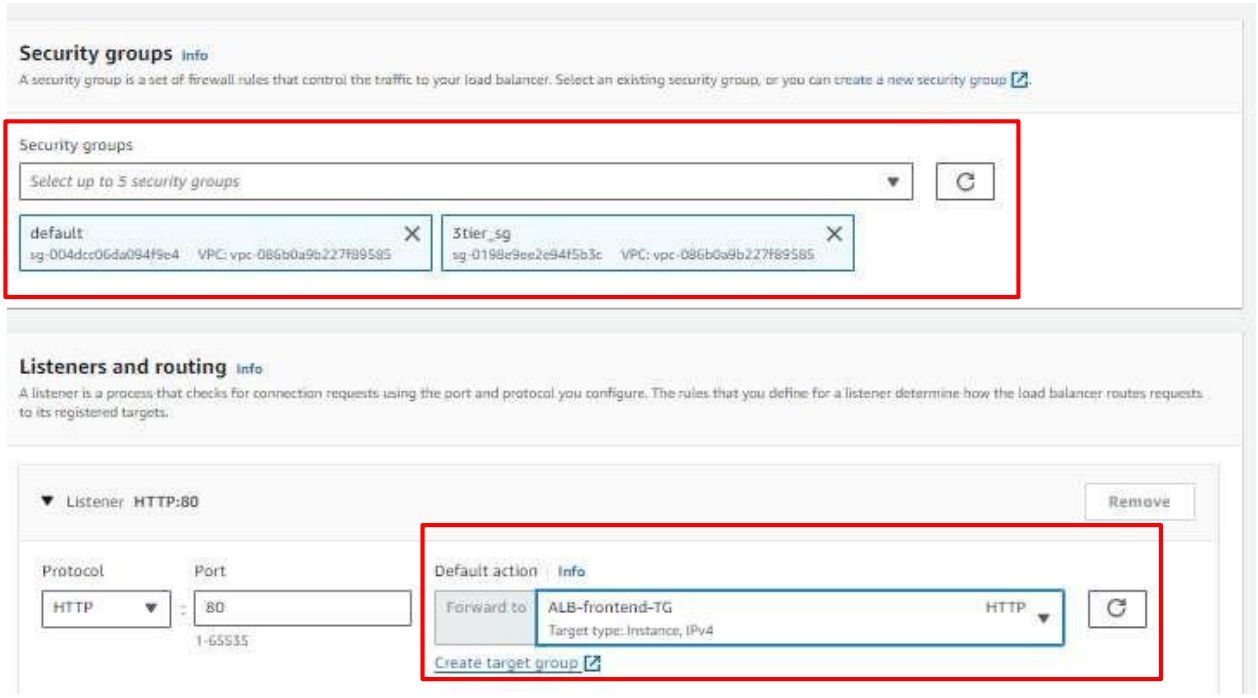


Here we can configure our ALB. First, give the relevant name to ALB such as **ALB-frontend**. Select the internet-facing option. In Network mapping select VPC that we have created. Select

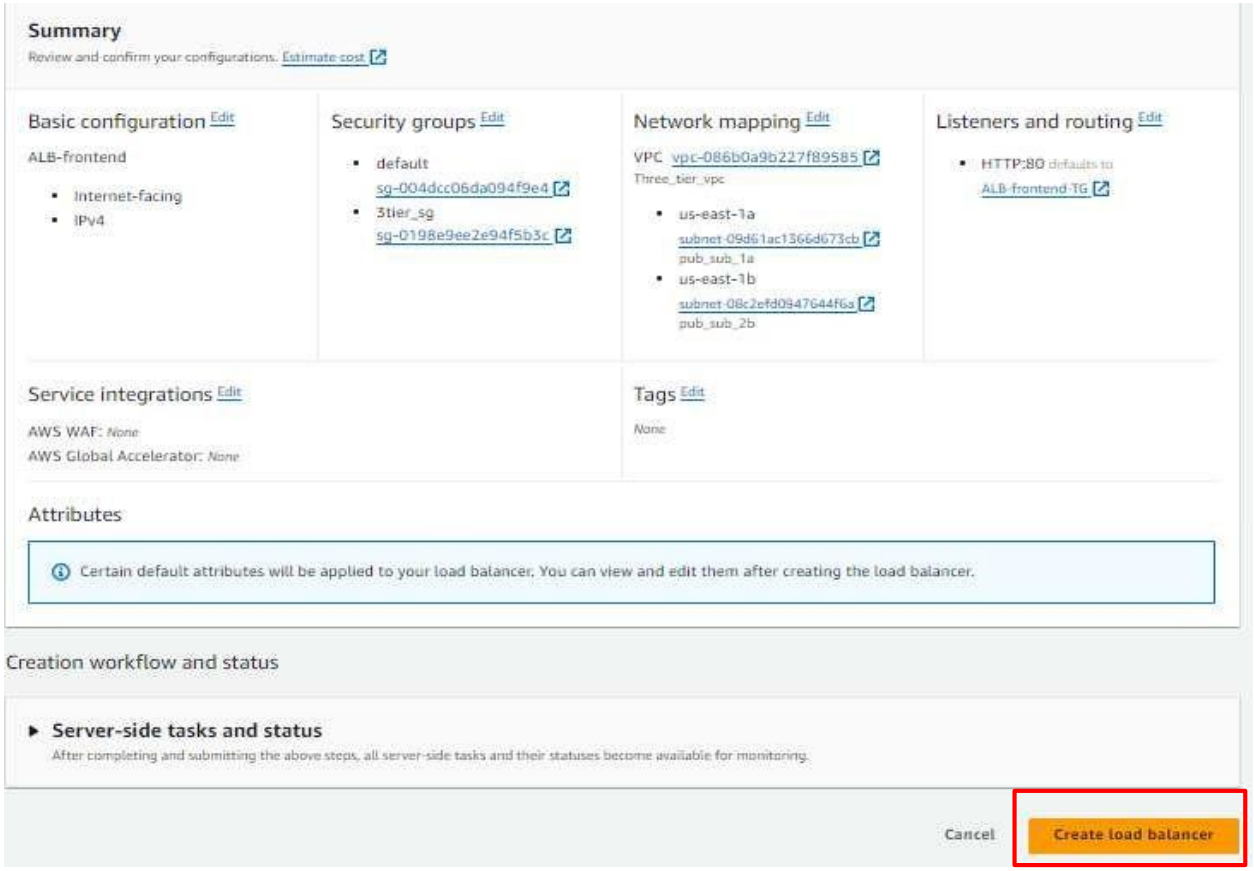
both availability zone **us-east-1a** and **us-east-2b**. And select subnet **pub-sub-1a** and **pub-sub-2b** respectively.



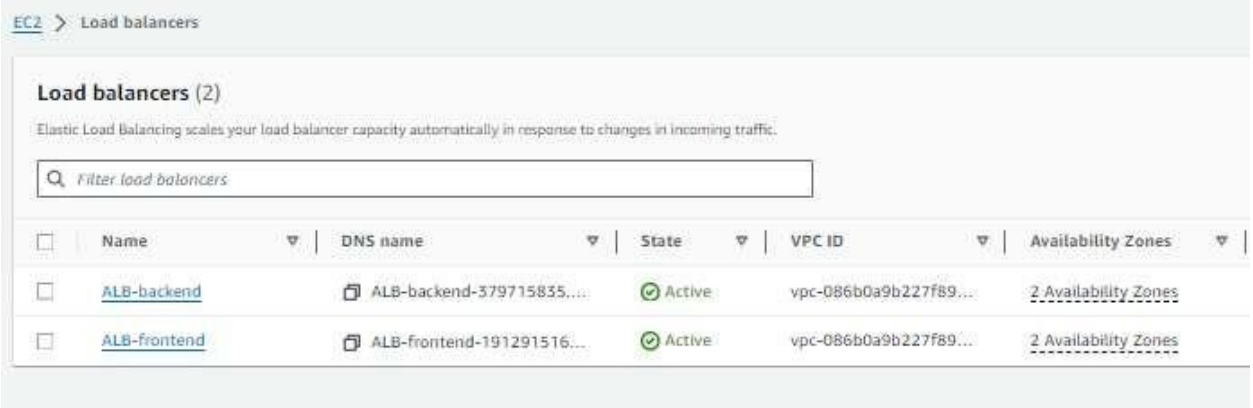
Select security group. In the listener part select TG that we have just created **ALB-frontend-TG**.



Scroll down and click on the create load balancer button.



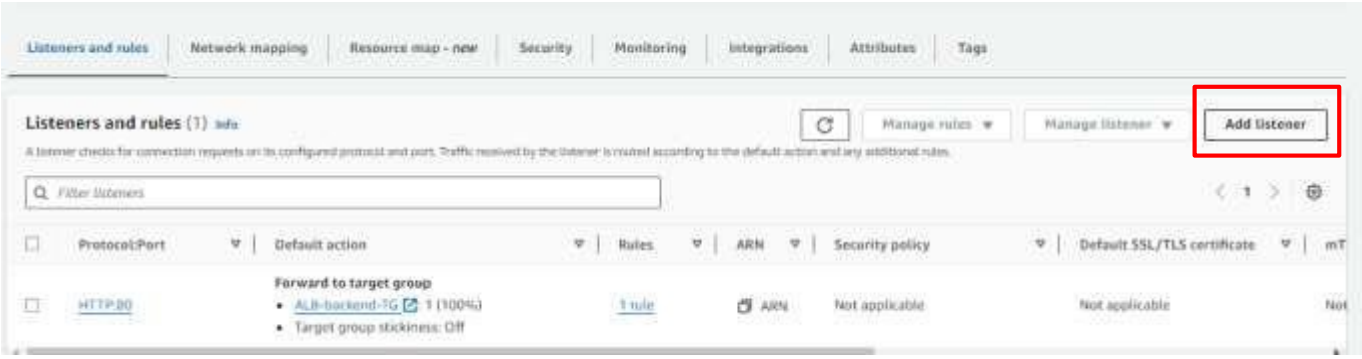
Now, let's create ALB for backend. Follow same above steps, but in the listener part select TG that we just created **ALB-backend-TG**.



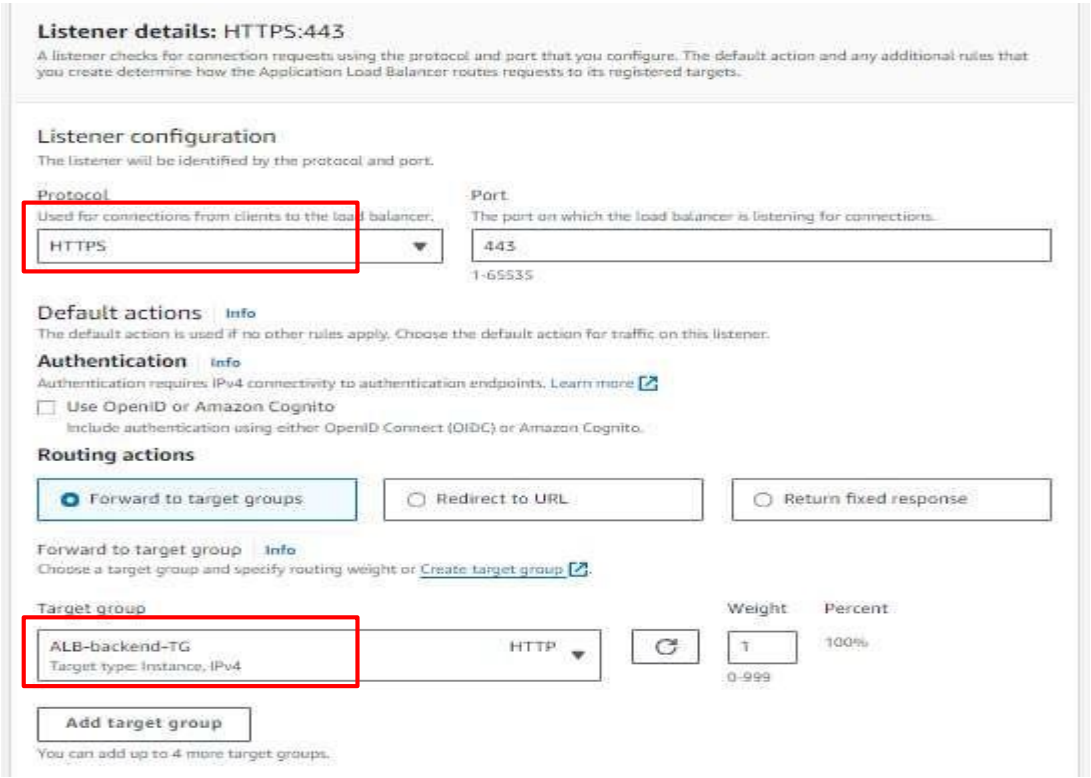
Now we have two load balancers, **ALB-frontend and ALB-backend**. But we need to add one more listener in **ALB-frontend and ALB-backend**.

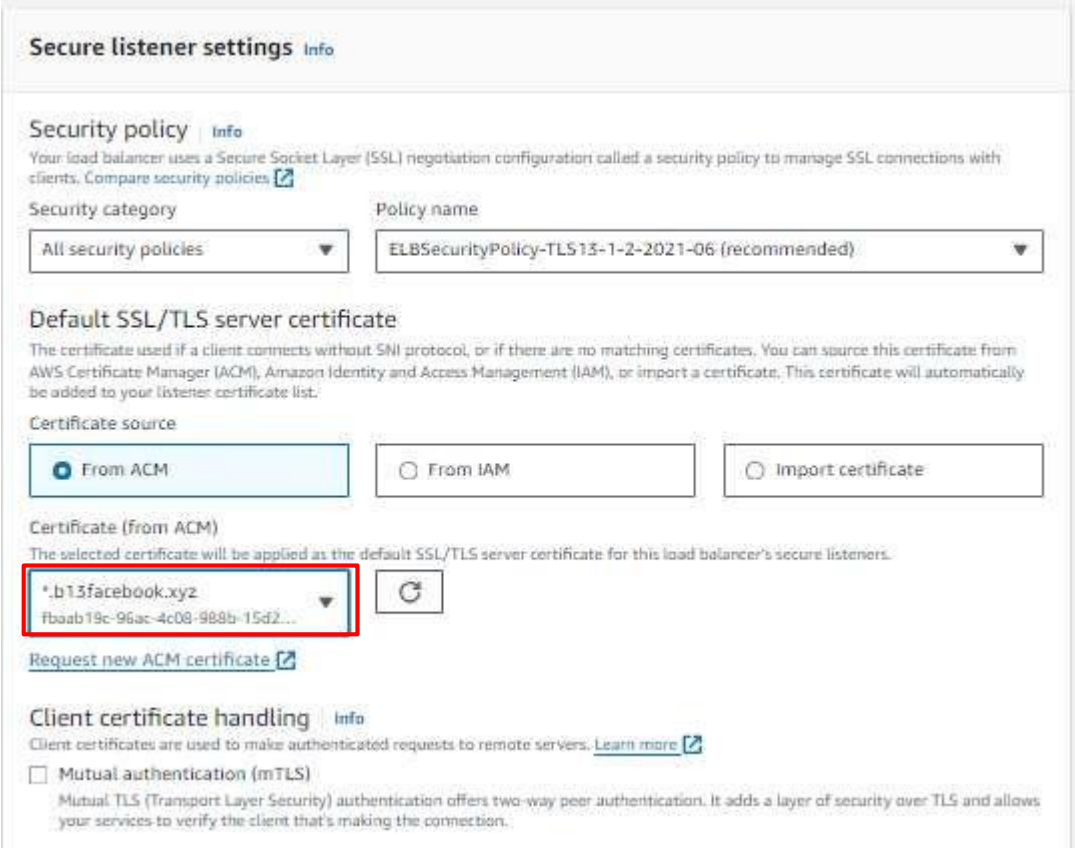
So click on ALB-backend.

Click on add listener the button that is located on the right side.



Here In listener details select HTTPS. Default Action should be forward and select ALB-backend-TG. Now we need to select the certificate that we have created. So in the Secure Listener setting select the certificate. And click on the add button below.


Listener details: HTTPS:443
A listener checks for connection requests using the protocol and port that you configure. The default action and any additional rules that you create determine how the Application Load Balancer routes requests to its registered targets.
Listener configuration
The listener will be identified by the protocol and port.
Protocol
Used for connections from clients to the load balancer.
HTTPS
Port
The port on which the load balancer is listening for connections.
443
1-65535
Default actions Info
The default action is used if no other rules apply. Choose the default action for traffic on this listener.
Authentication Info
Authentication requires IPv4 connectivity to authentication endpoints. Learn more
Use OpenID or Amazon Cognito
Include authentication using either OpenID Connect (OIDC) or Amazon Cognito.
Routing actions
Forward to target groups
Redirect to URL
Return fixed response
Forward to target group Info
Choose a target group and specify routing weight or Create target group.
Target group
ALB-backend-TG
Target type: Instance, IPv4
HTTP
Weight
1
0-999
Percent
100%
Add target group
You can add up to 4 more target groups.


Secure listener settings Info
Security policy Info
Your load balancer uses a Secure Socket Layer (SSL) negotiation configuration called a security policy to manage SSL connections with clients. Compare security policies
Security category
All security policies
Policy name
ELBSecurityPolicy-TLS13-1-2-2021-06 (recommended)
Default SSL/TLS server certificate
The certificate used if a client connects without SNI protocol, or if there are no matching certificates. You can source this certificate from AWS Certificate Manager (ACM), Amazon Identity and Access Management (IAM), or import a certificate. This certificate will automatically be added to your listener certificate list.
Certificate source
From ACM
From IAM
Import certificate
Certificate (from ACM)
The selected certificate will be applied as the default SSL/TLS server certificate for this load balancer's secure listeners.
*.b13facebook.xyz
fbaab19c-96ac-4c08-988b-15d2...
Request new ACM certificate
Client certificate handling Info
Client certificates are used to make authenticated requests to remote servers. Learn more
Mutual authentication (mTLS)
Mutual TLS (Transport Layer Security) authentication offers two-way peer authentication. It adds a layer of security over TLS and allows your services to verify the client that's making the connection.

Follow same steps for **ALB-frontend** also.

So here we successfully completed the ALB setup

- Create target groups and load balancers in **Oregon(us-west-2)** region also with same steps

EC2

Now we are going to create a temporary frontend and backend server to do all the required setup, take snapshots and create Machine images from it. So that we can utilize it in the launch template.

1. First, click on the instance button and then click on the Launch Instance button on the top right corner.
2. First, we are going to set up a frontend server. Give a name to your instance **(temp-frontend-server)**. Select Ubuntu as the operating system. Choose the instance type as t2.micro. Click on Create key pair if you don't have it.
3. Here we are doing a temporary setup so we don't use our OWN VPC. We can use the default VPC given by AWS. In short, keep the Network setting as it is.

We have successfully launched **temp-frontend-server**. So now let's launch a temporary backend server. Give a name to your instance **(temp-backend-server)**.Follow above steps

Please wait for 5-8 minutes so that the instance comes in a running state. And then we will utilize instances for further steps.

Temp-frontend-server:

Select **temp-frontend-server**. Now open Gitbash where you have downloaded your YOUR_KEY.pem file. And type the command.

ssh -i <name_of_key>.pem ubuntu@<Public_IP_add_of_Instance>

Now you are successfully logged your remote **temp-frontend-server**. Now our first task is to install some packages and second task is to clone git repo.

```
#packages for our frontend server
#!/bin/bash

sudo apt update -y

sudo apt install apache2 -y

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - &&\
sudo apt-get install -y nodejs -y
```

```
sudo apt update -y

sudo npm install -g corepack -y

corepack enable

corepack prepare yarn@stable --activate

sudo yarn global add pm2
```

```
ubuntu@ip-172-31-35-128:~$ sudo apt update -y

sudo apt install apache2 -y

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - &&\
sudo apt-get install -y nodejs -y

sudo apt update -y

sudo npm install -g corepack -y

corepack enable

corepack prepare yarn@stable --activate --yes

sudo yarn global add pm2
```

The Github repository link is https://github.com/jadalaramani/aws_three_tier_code.git

```
git clone https://github.com/jadalaramani/aws_three_tier_code.git
```

Go inside the directory.

```
cd aws_three_tier_code/client
```

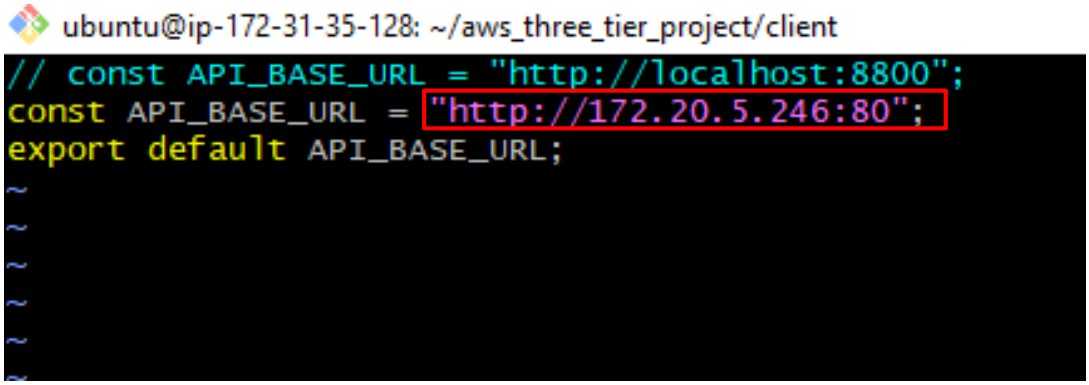
```
ubuntu@ip-172-31-35-128:~$ git clone https://github.com/Ramani-github/aws_three_tier_project.git
Cloning into 'aws_three_tier_project'...
remote: Enumerating objects: 51, done.
remote: Counting objects: 100% (51/51), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 51 (delta 9), reused 28 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (51/51), 317.74 KiB | 14.44 MiB/s, done.
Resolving deltas: 100% (9/9), done.
ubuntu@ip-172-31-35-128:~$ cd aws_three_tier_project/client/
ubuntu@ip-172-31-35-128:~/aws_three_tier_project/client$
```

Now, we need to change just one line in our frontend application that is built in React. So type the command

```
vim src/pages/config.js
```

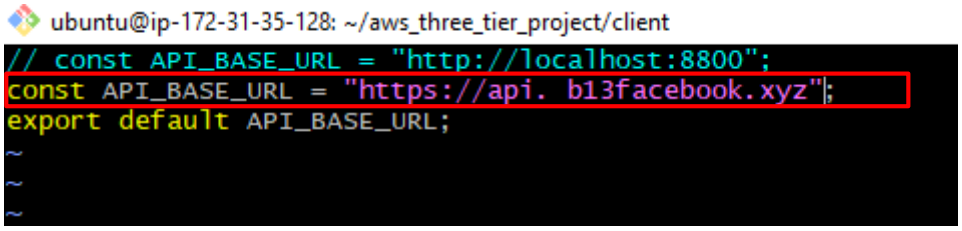

917396627149-Madhukiran
devopstraininghub@gmail.com

The above command opens the file in a text editor. Now press esc + I the button on your keyboard to edit the file. In this file, we have to change API_BASE_URL. So remove whatever is present in the API_BASE_URL variable.



And add **https://api. b13facebook.xyz**, in my case I have added this URL but in your case it is different. This means you need to use your OWN domain name. So your API_BASE_URL should be like https://api.<YOUR_DOMAIN_NAME>.XYZ. After updating the variable press ESC key on your keyboard and then type: wq and hit the Enter button.

API_BASE_URL = https://api.b13facebook.xyz



After making these changes our frontend of the application will send all the API calls on the domain name https://api.b13facebook.xyz and lastly, that will point to our backend server.

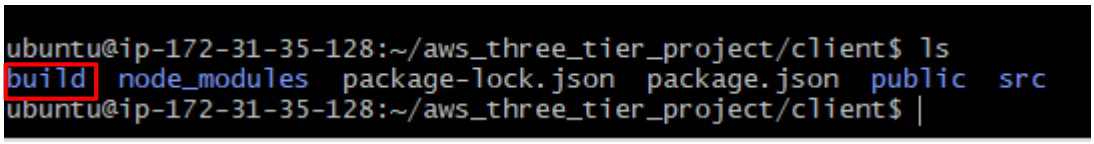
Now type the command npm install in the terminal to install all the required packages.

```
npm install
```

Type the command npm run build to create the optimize static pages.

```
npm run build
```

Now you have one more folder in the directory called build. You can verify that by tying ls command0



Now type the very essential command sudo cp -r build/* /var/ww/html/

```
sudo cp -r build/* /var/www/html
```

The above command takes all the static files from the build folder and stores them in /var/www/html so that Apache can serve them.

Here our temp-frontend-server configuration is completed.

Temp-backend-server

Now let's set up the temp-backend-server. So select the temp-backend-server and copy the IP address of the instance. Again please open Git bash in the same directory where your stored key.pem file. And type the below command

ssh -i name_of_your_key>.pem ubuntu@<Public_IP_add>

We are successfully logged in inside the backend server. First, install packages and we will clone the repo.

```
#!/bin/bash

sudo apt update -y

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - &&\
sudo apt-get install -y nodejs -y

sudo apt update -y

sudo npm install -g corepack -y

corepack enable

corepack prepare yarn@stable --activate

sudo yarn global add pm2
```

```
ubuntu@ip-172-31-47-128:~$

sudo apt update -y

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - &&\
sudo apt-get install -y nodejs -y

sudo apt update -y

sudo npm install -g corepack -y

corepack enable

corepack prepare yarn@stable --activate --yes
```

git clone https://github.com/jadalaramani/aws_three_tier_code.git

go inside the aws_three_tier_code/backend

cd aws_three_tier_project/backend

```
ubuntu@ip-172-31-47-128:~$ git clone https://github.com/Ramani-github/aws_three_tier_project.git
Cloning into 'aws_three_tier_project'...
remote: Enumerating objects: 51, done.
remote: Counting objects: 100% (51/51), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 51 (delta 9), reused 28 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (51/51), 317.74 KiB | 12.71 MiB/s, done.
Resolving deltas: 100% (9/9), done.
ubuntu@ip-172-31-47-128:~$ cd aws_three_tier_project/backend/
ubuntu@ip-172-31-47-128:~/aws_three_tier_project/backend$
```

Here we are going to create one file with the name .env

vim .env

Press the esc I button on your keyboard. And copy the code given below and paste the snippet into the code editor. This code contains information about the RDS instance. Please change your username and password according to whatever you kept while creating a database. And then click on the ESC button and type: wq and hit the enter button

DB_HOST=book.rds.com

DB_USERNAME=admin

DB_PASSWORD="Mindcircuit1234"

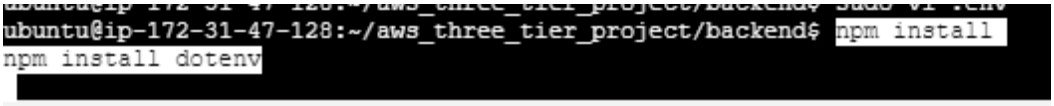
PORT=3306

```
DB_HOST=book.rds.com
DB_USERNAME=admin
DB_PASSWORD="Mindcircuit1234"
PORT=3306

~
~
```

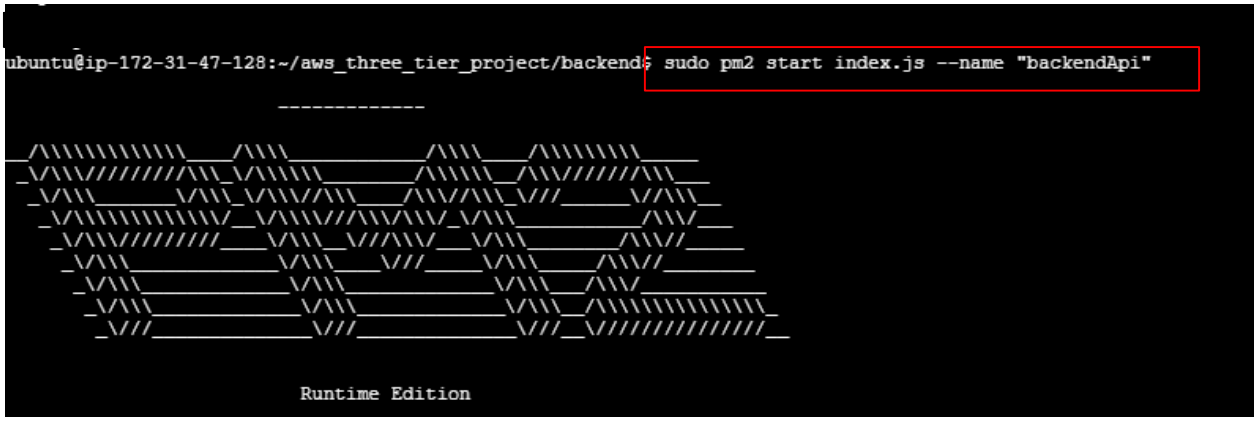
Now type the below commands in terminal

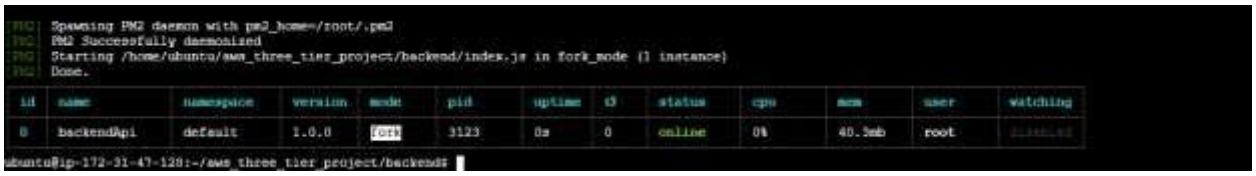
```
npm install
npm install dotenv
npm install mysql2
```



Now, let's start the backend server. (***Very IMP***)

```
sudo pm2 start index.js --name "backendApi"
```





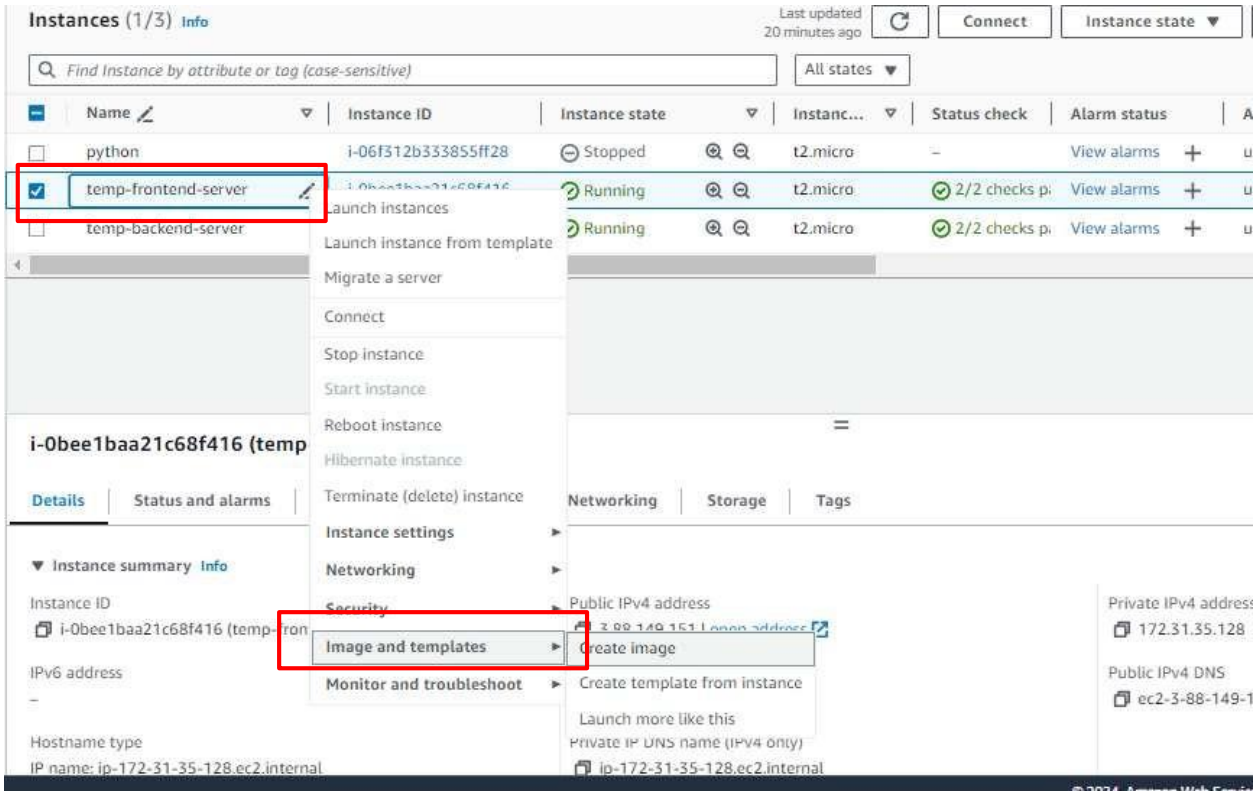
You can verify that by typing the command

```
sudo pm2 list
```

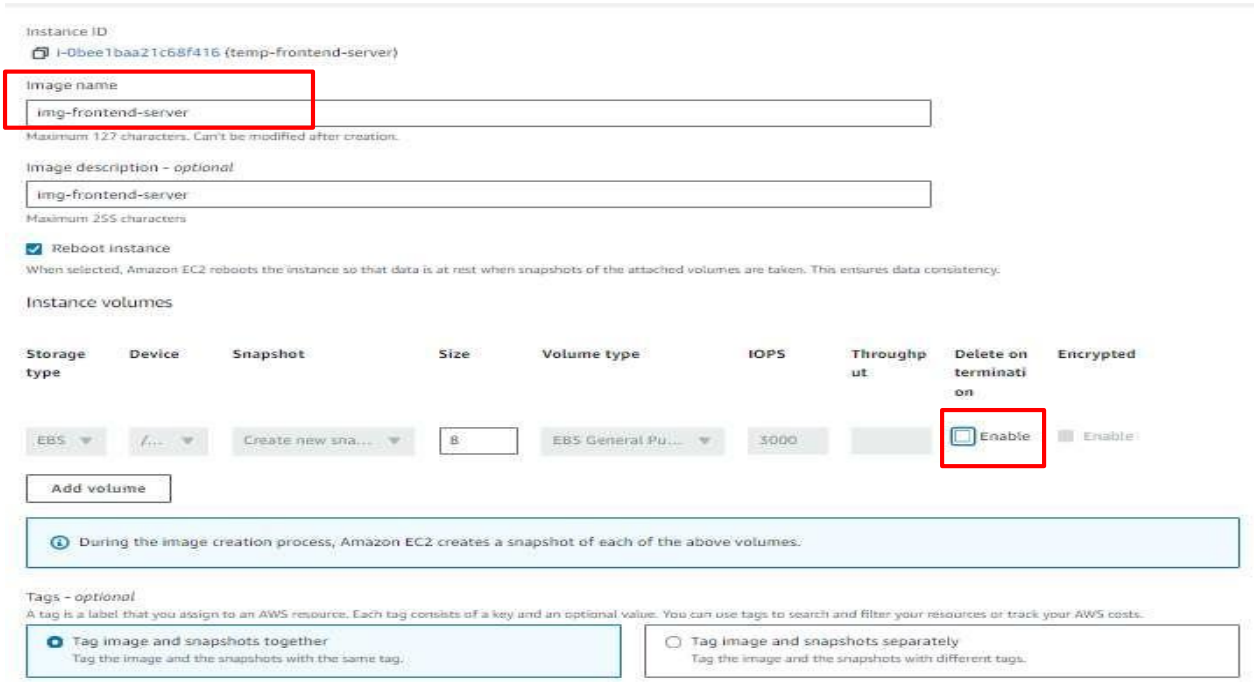

917396627149-Madhukiran devopstraininghub@gmail.com

Now we have to create Machine images of these servers so that we can create a launch template.

So please select temp-frontend-server and click on the Action button in the top right corner. One drop-down menu will open. You have to select the images and template option and that will give one more drop-down menu from which we need to click on create image button.



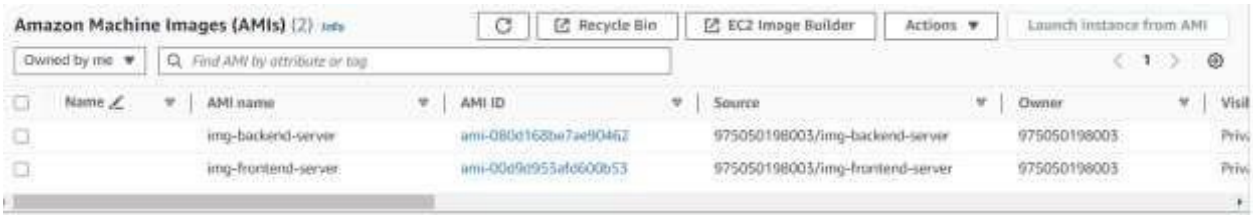
Give the name you your image (**img-frontend-server**). Just deselect that delete on the termination button and click on the create image button.



917396627149-Madhukiran
devopstraininghub@gmail.com

You have to do the same thing for the temp-backend-server as well.

After a couple of minutes (10-15) you can see those images. Click on the AMIs button on the left panel and you can see both images here.



Backup service

We create machine images in **N.virginia (us-east-1)** region and copy these images to the **Oregon region (us-west-2)** using backup service

1: Open AWS Backup Service

- Go to the **AWS Console**.
- In the search bar, type **"Backup"** and click on the **Backup** service.
- Make sure your **region is set to N. Virginia (us-east-1)**.

2: Create a Backup Vault

- On the left panel, click **"Backup vaults"**.
- Click **"Create Backup Vault"** (top right).
- Enter a **Vault name** (e.g., `my-backup-vault`).
- Click **"Create Backup Vault"**.

3: Create a Backup Plan

- On the left panel, click **"Backup plans"**.
- Click **"Create Backup Plan"** (top right).

Inside Create Plan:

- Choose **"Build a new plan"**.
- Enter a **Backup Plan Name** (e.g., `daily-ec2-backup-plan`).

Under Backup Rule Configuration:

- **Rule Name**: (e.g., `daily-backup`)
- **Backup Vault**: Select the vault you just created (e.g., `my-backup-vault`).
- **Frequency**: Daily or as needed.
- **Backup Window Start Time**: Select a time **10 minutes ahead of current UTC time**.
- **Lifecycle**: Optional - set retention if needed.

Cross-Region Copy:

- Scroll down to **"Copy to destination"**.
- **Destination Region**: Select `us-west-2` (Oregon).
- Use **default backup vault** in Oregon or create a new one.
- Leave other settings as default unless instructed.
- Click **"Create Plan"**.

4: Assign Resources to Backup Plan

- After creating the plan, click **"Assign resources"**.

Inside Assign Resources:

- **Assignment Name**: (e.g., `backup-temp-servers`)
- **IAM Role**: Use default or create a new one if required.
- **Resource Selection**: Choose **"Include specific resource types"**.
- **Resource Type**: Select **EC2**.
- **Choose Resources**: Select **Instance IDs** of:
  - `temp-frontend-server`
  - `temp-backend-server`
- Click **"Assign Resources"**.

5: Monitor Backup Job

- On the left menu, click **"Jobs"** > **"Backup Jobs"**.
- Wait ~20 minutes for the job to initiate.
- Once completed, you'll see **status: Completed**.
- Your **backup AMIs are now available in us-east-1**.
  - Go to **EC2 > AMIs** in **N. Virginia** to verify.

6: Monitor Copy Job to Oregon

- In AWS Backup, go to **"Copy Jobs"** on the left panel.
- A new copy job will initiate after backup completion.

- Wait for the **copy job to finish** (~few more minutes).

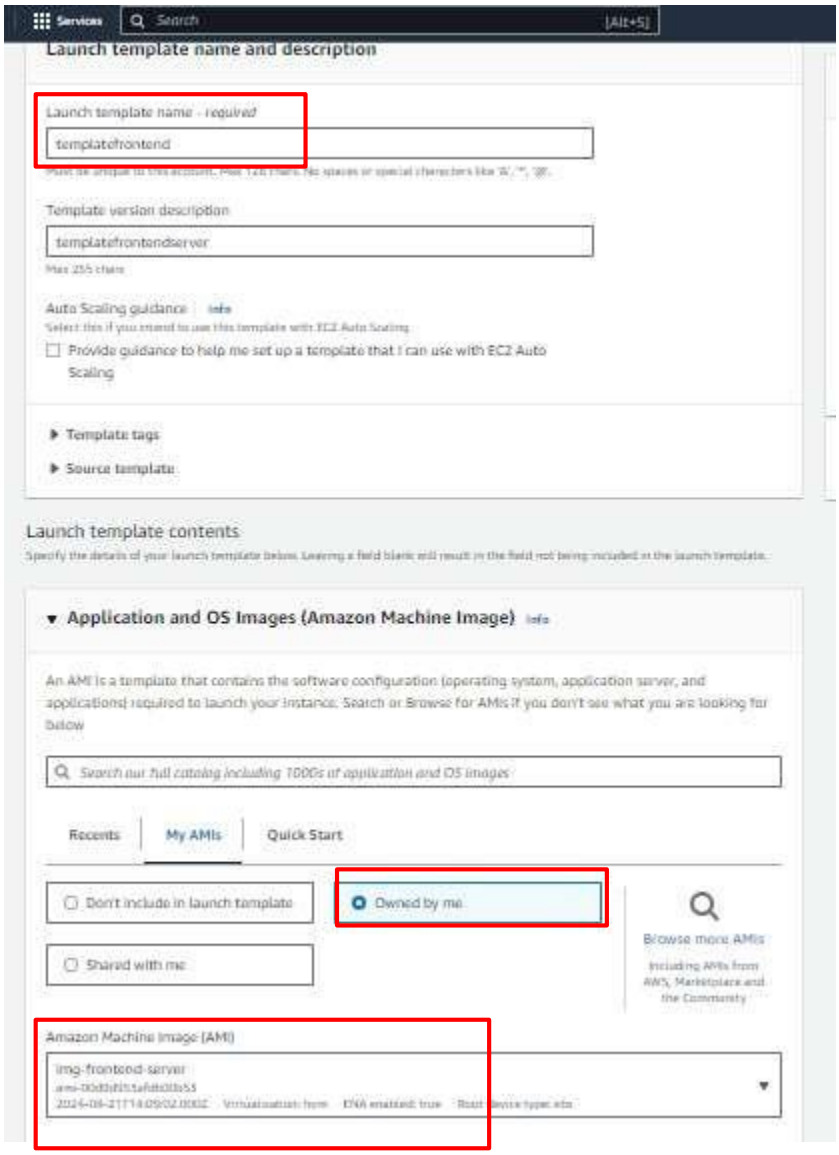
7: Verify AMIs in Oregon (us-west-2)

- Switch AWS region to **us-west-2 (Oregon)**.
- Go to **EC2 > AMIs**.
- You should now see **the copied backup images** (AMIs).

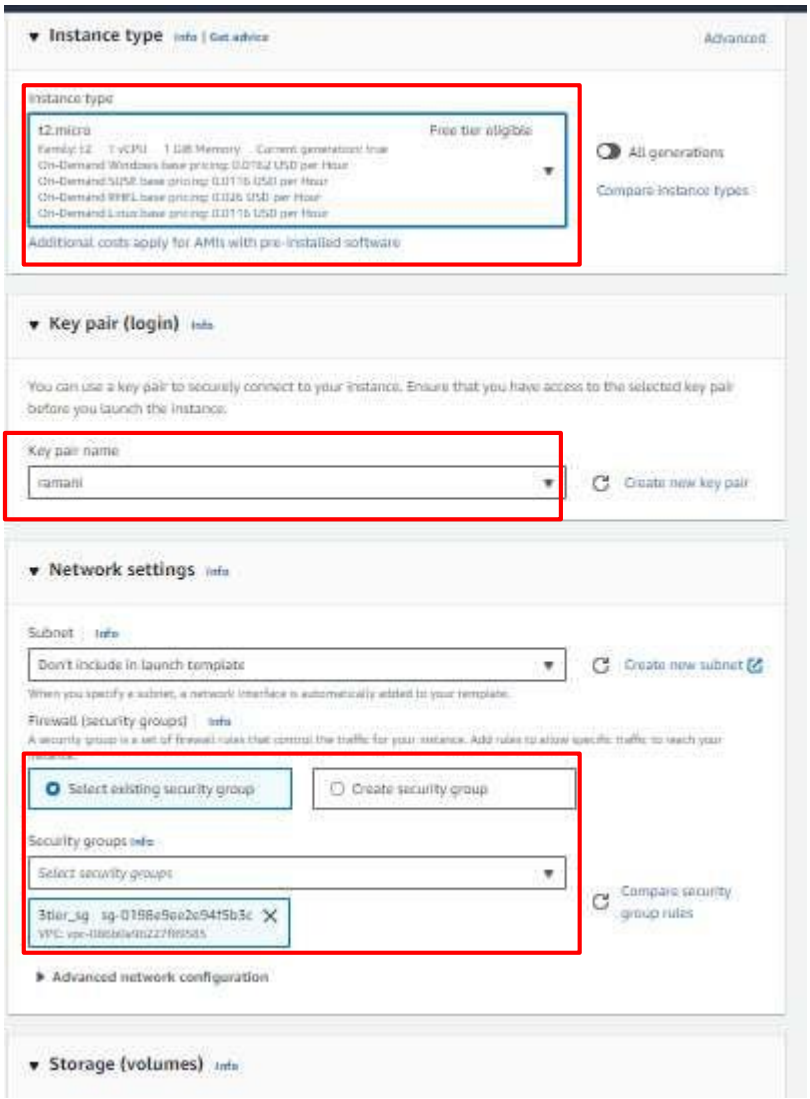
Launch Template

Create a launch template, so click on the launch template button on the left panel and click on the create launch template button.

Give the name to your launch template such as template-frontend-server as we are creating a launch template for frontend-server. Here we need to select AMI so click on My AMIs tab and select the option owned by me. So now it will show you all the images that are present in your current region. Here you have to select the image that contains the frontend application. Select instance type t2.micro



Scroll down, attach the key pair, and in the network setting just select the security group that we created for the frontend server.



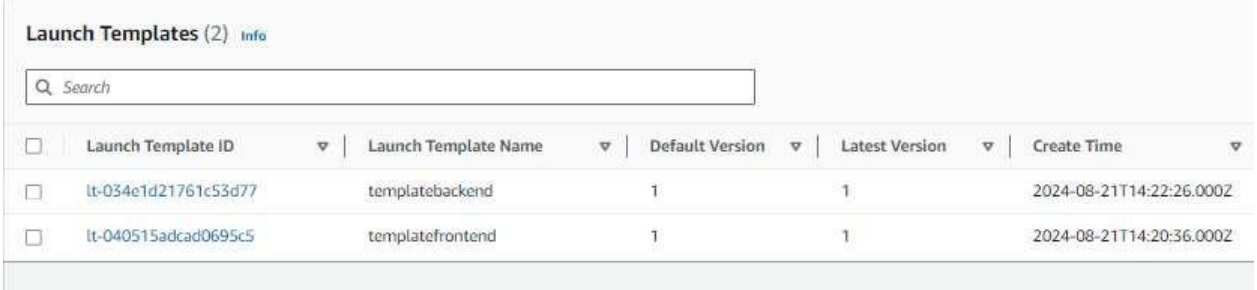
We successfully created a launch template for the frontend-server. Now let's create a launch template for the backend server.

Give a name to your launch template (template-backend-server). Give version 1 in the version field, but make you select the correct AMI that holding your backend application. And Select an instance type t2.micro

Select the key pair, and in the network setting just select the security group that we have created.

And then click on the Create launch template button.
We have created two launch templates, template-frontend-server and template-backend-server

917396627149-Madhukiran
devopstraininghub@gmail.com



- Create Launch template in **oregon(us-west-2**) region. Everything is completely similar just you have to change AMIs. Please select the correct AMI for the frontend and backend. If you have difficulties finding AMIs you can compare the instance_id with temp-frontend-server and temp-backend-server. this will definitely help you.

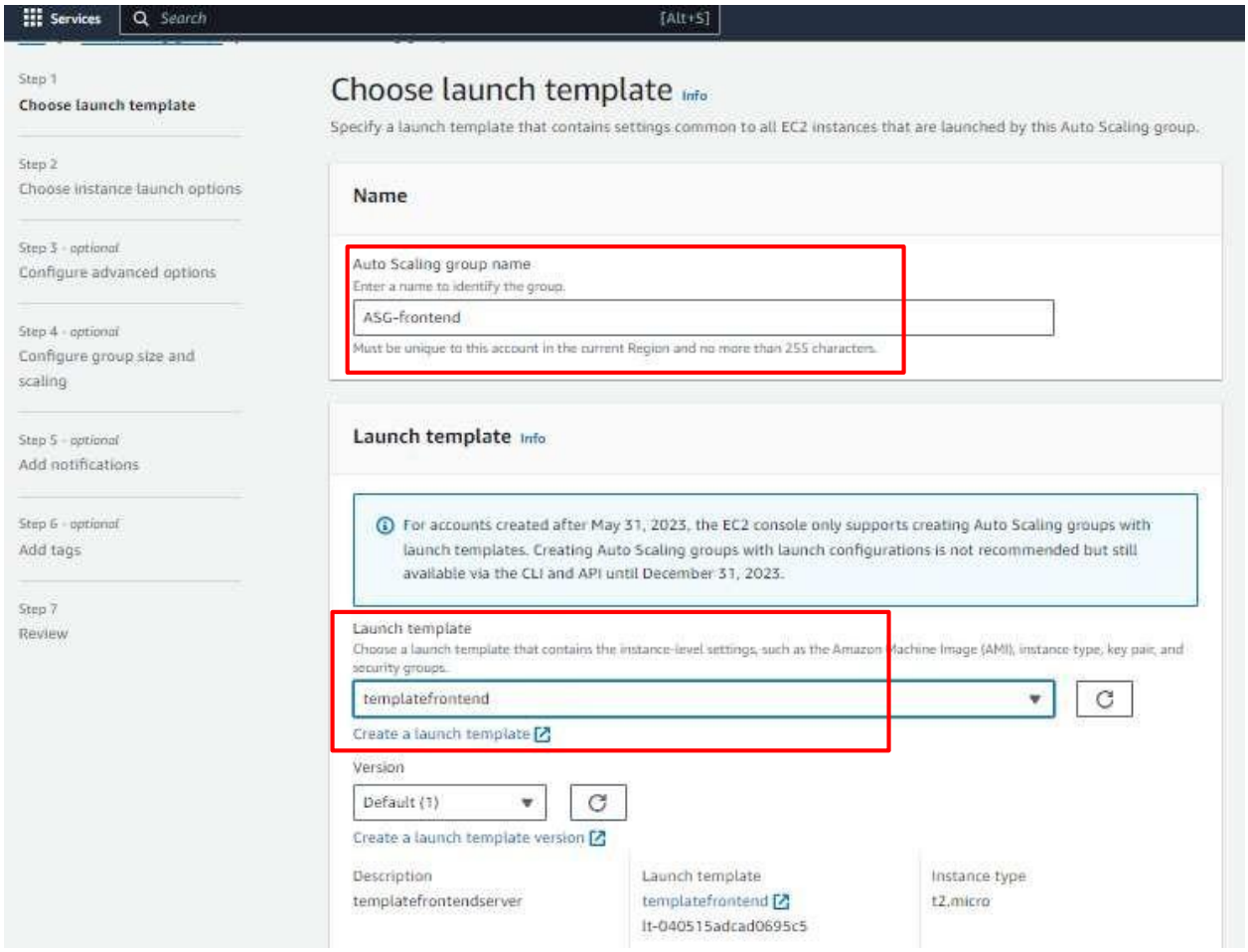
Auto scaling group (ASG)

- **we need to set up an Auto Scaling group in both regions us-east-1 (N.virginia) and us-west-2 (Oregon, Disaster recovery region).**

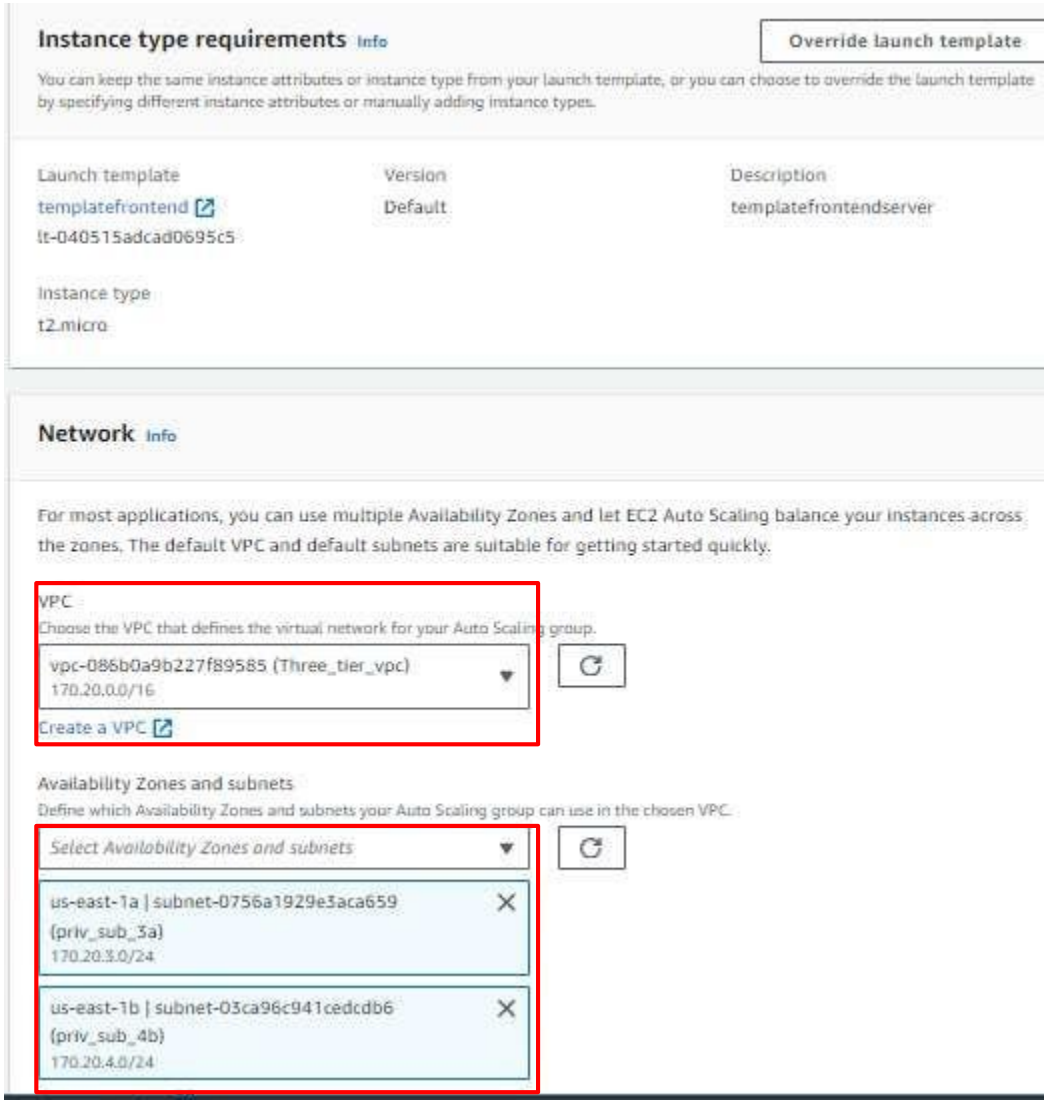
The auto-scaling group is the functionality of EC2 service that launches instances depending on your network traffic or CPU utilization or parameter that you set. It launches instances from the launch template.

Click on the Auto scaling group's button which is located at the bottom of the left panel. And then click on the Create auto scaling group button.

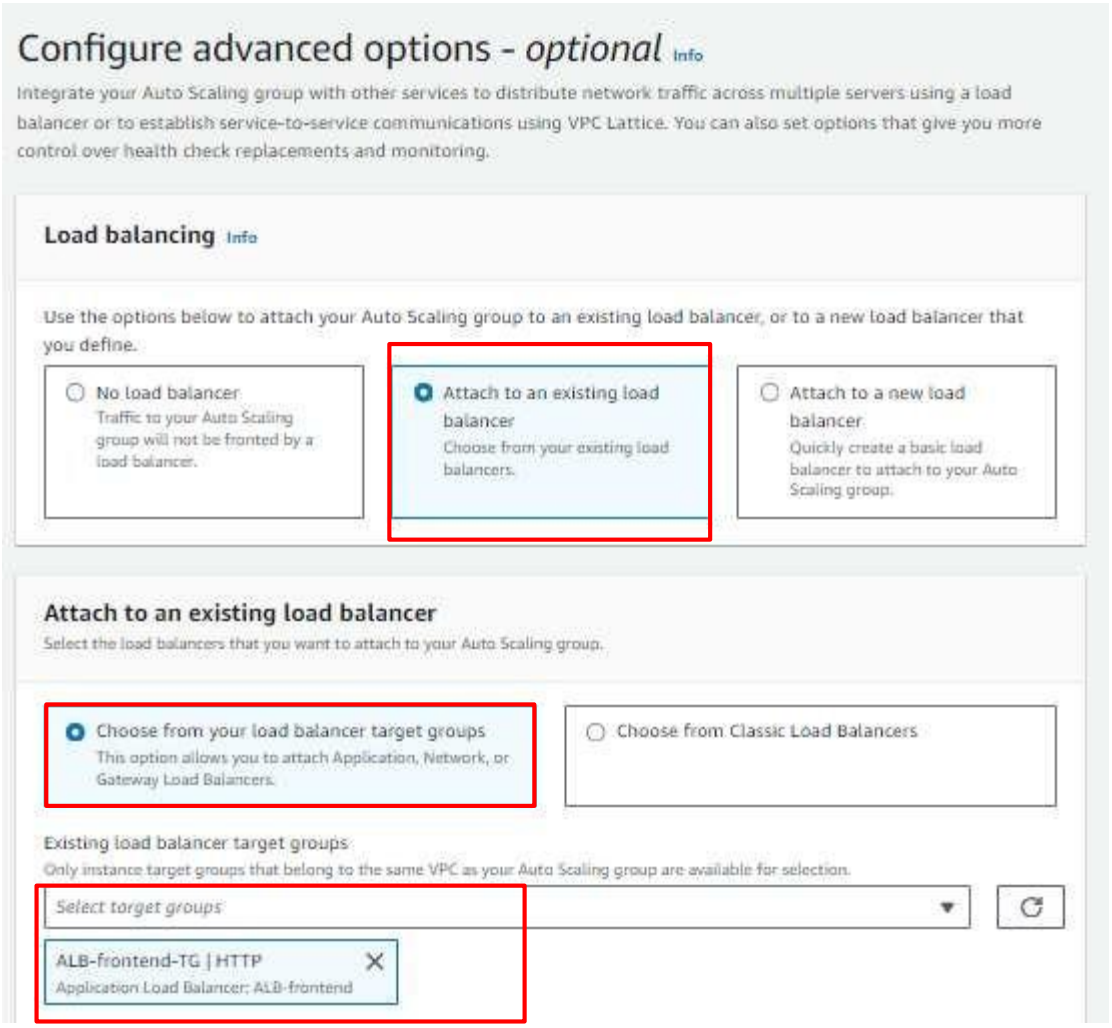
Give a name to your ASG. E.g. ASG-frontend. And select the launch template that we have created for frontend (e.g. templatefrontend) in the launch template field. And click on the next button.



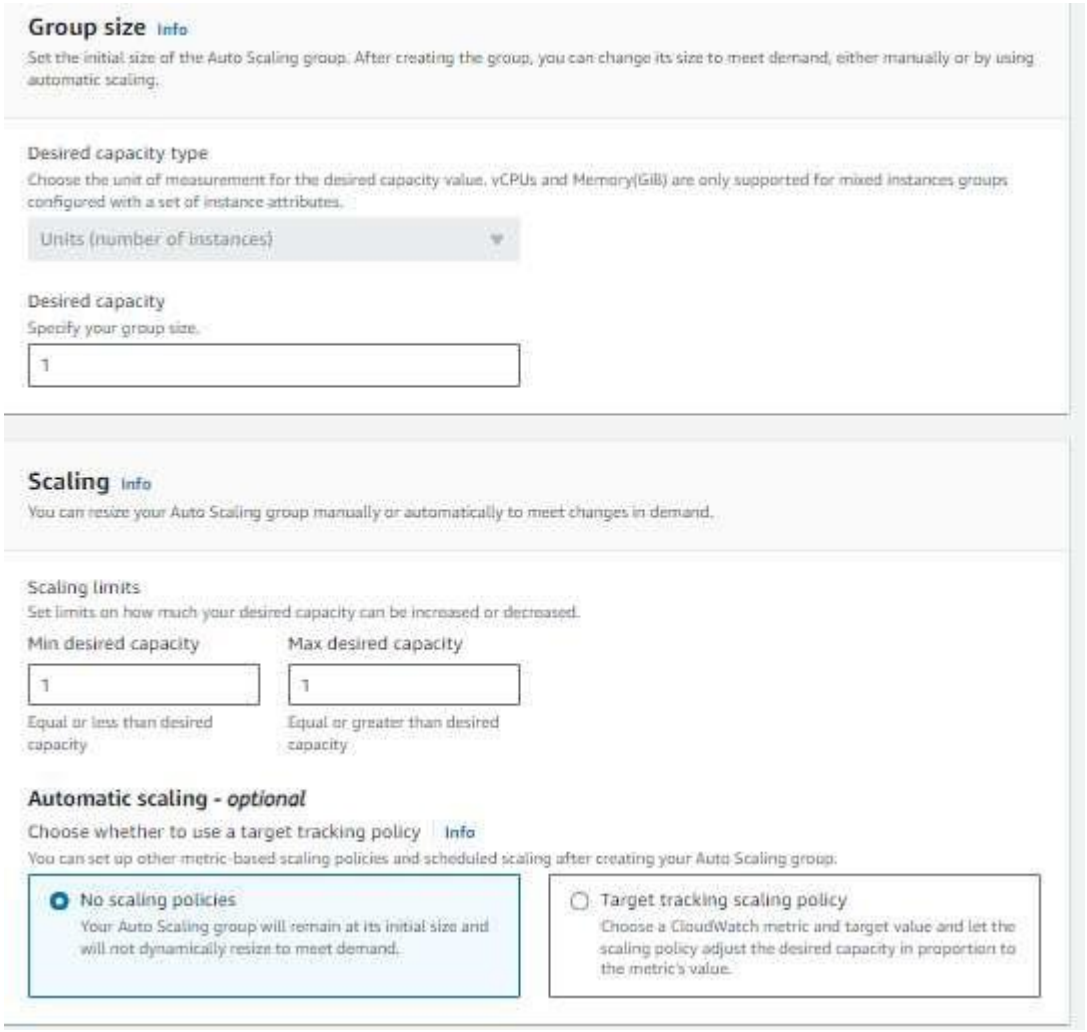
In the network field, you have to choose VPC that we created earlier. And in AZs and subnet filed choose pri-sub-3a and pri-sub-4b. These subnets we have created for frontend servers. And click on the next button.



On this page we need to attach ASG with ALB so select the Attach existing ALB option and select TG that we have created for frontend e.g. ALB-frontend-TG. And then scroll down and click on the NEXT button



Here you can set the capacity and scaling policy but now I am keeping 1, 1, and 1 to save cost but in real projects, it depends on the traffic. Click on the NEXT->next->next-> and create ASG button.

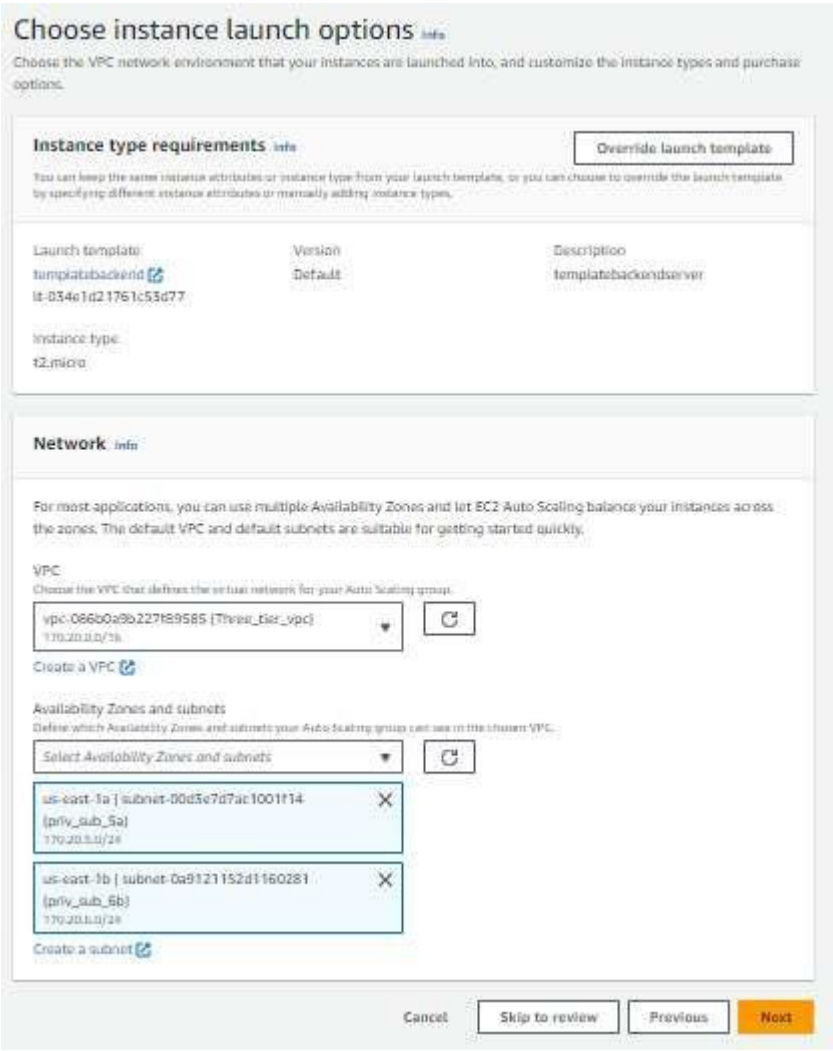


Let's set up ASG for the backend.

Give a name to your ASG. E.g. backendasg. And select the launch template that we have created for the backend (e.g. templatebackend) in the launch template field. And click on the next button.

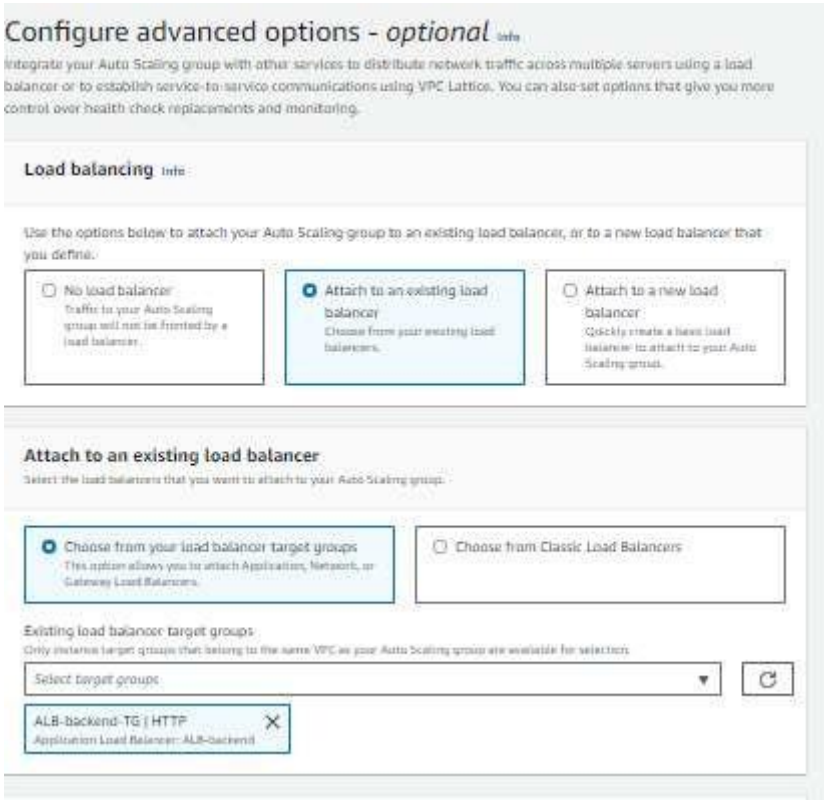
In the network field, you have to choose VPC that we created earlier. And in AZ and subnet field choose pri-sub-5a and pri-sub-6b. These subnets we have created for backend servers. And click on the next button.

917396627149-Madhukiran
devopstraininghub@gmail.com



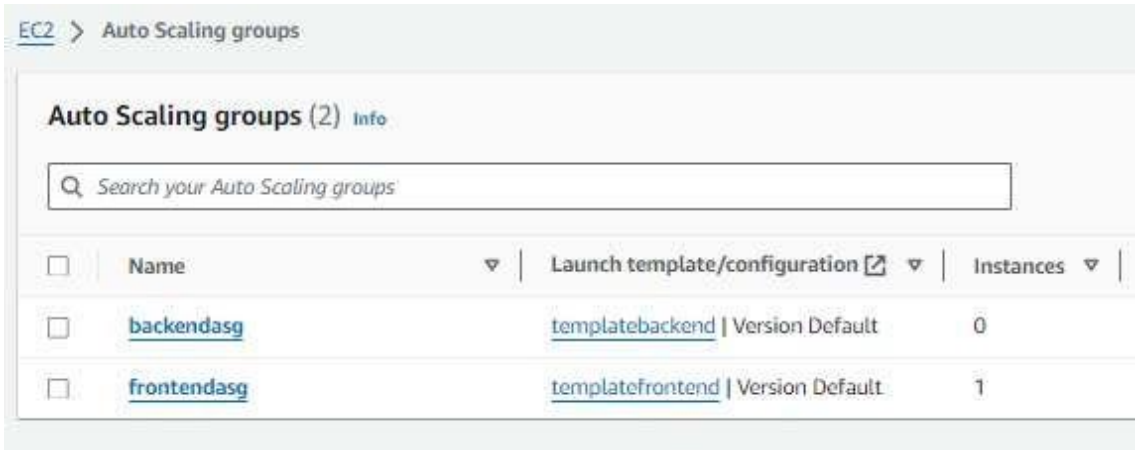
On this page we need to attach ASG with ALB so select the Attach existing ALB option and select TG that we have created for the backend e.g. ALB-backend-TG. And then scroll down and click on the NEXT button.

917396627149-Madhukiran
devopstraininghub@gmail.com



Here you can set the capacity and scaling policy but I'm keeping 1, 1, 1 to save cost but in real projects, it depends on the traffic. Click on the NEXT->next->next-> and create ASG button.

Now, We have two ASGs, ASG-frontend will launch frontend servers and ASG-backend will launch backend servers.



We need to initialize our database and need to create some tables. But we can't access the RDS instance or backend server directly coz they are in a private subnet. So we need to launch an instance in the same VPC but in the public subnet that instance is called bastion host or jump-server. And through that instance, we will log in to the backend server, and from the backend server we will initialize our database.

Click on the instance button on the left panel and click on the launch instance button in the top right corner.

Give a name to the instance (bastion-jump-server). Select Ubuntu as OS, instance typet2.micro, and select Key pair. In all the instance and launch template we have used only **one key** so it will be easy to login in any instance. And then click on the Edit button of the Network setting.

In the network setting select VPC that we have created and in the subnet select pub-sub-1a, you can select any public subnet from the VPC. And then select security group. And click on the launch instance.

Once the instance becomes healthy, we can SSH into it. So select the instance and copy its public IP. Open Git bash or terminal in which folder your key.pem file is present and connect

Now copy the pem file to ubuntu server and give permission to connect the backend server which is in private subnet

Now type the below command to login into the Bastion host. And copy the public IP of the Bastion host.

ssh -i <name_of_your_key>.pem ubuntu@<Public_IP_add_of_instance>

We are successfully logged in inside the bastion host.

Create a file of your pem

Give permissions: chmod 400 keypair.pem

Ssh keypair.pem ubuntu@<frontend/backend server private ip>

```
ubuntu@ip-170-20-1-208:~$ vi ramani.pem
ubuntu@ip-170-20-1-208:~$ chmod 400 ramani.pem
ubuntu@ip-170-20-1-208:~$ ssh -i "ramani.pem" ubuntu@170.20.4.70
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1012-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Wed Aug 21 14:46:41 UTC 2024
```

Now we you have login into the frontend server

Run below script:

```
#!/bin/bash

sudo apt update -y

sleep 90

sudo systemctl start apache2.service
```

Type the below command to log in to the backend server.

ssh -i key.pem ubuntu@<Private_IP_add_backend_server>

```
-r-------- 1 ubuntu ubuntu 1675 Aug 21 14:45 ramani.pem
ubuntu@bastion-server:~$ ssh -i "ramani.pem" ubuntu@170.20.6.22
The authenticity of host '170.20.6.22 (170.20.6.22)' can't be established.
ED25519 key fingerprint is SHA256:cWno2yvu2DT6Ab91QmoAUkgny/s/2mzufQ9ZNz4yw3I.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '170.20.6.22' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1012-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Aug 21 14:54:39 UTC 2024
```

Now we are logged in inside the backend server. Just go into aws_three_tier_project/backend directory

```
cd  aws_three_tier_project/backend
```

```
ubuntu@ip-170-20-6-22:~$ cd aws_three_tier_project/backend/
ubuntu@ip-170-20-6-22:~/aws_three_tier_project/backend$ pwd
/home/ubuntu/aws_three_tier_project/backend
ubuntu@ip-170-20-6-22:~/aws_three_tier_project/backend$
```

We need to install one package type below the command

```
#!/bin/bash

sudo apt update -y
```


917396627149-Madhukiran
devopstraininghub@gmail.com

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash - &&\
sudo apt-get install -y nodejs -y
sudo apt update -y
sudo npm install -g corepack -y
corepack enable
corepack prepare yarn@stable --activate
sudo npm install -g pm2
sudo apt install mysql-server -y
cd /home/ubuntu/aws_three_tier_code/backend
sudo pm2 start index.js --name "backendapi"
mysql -h book.rds.com -u admin -psJOMVBzQizbvvmLtqoG8 test < test.sql
```

And type the below command to initialize the database.

mysql -h book.rds.com -u <user_name_of_rds> -p<password_of_rds> test < test.sql

Route 53

Amazon Route 53 is a highly available and scalable Domain Name System (DNS) web service.

If you try to access the web app using ALB-frontend DNS then you won't see the website in functional mode because our frontend or loaded static pages try to call the API from your browser on the domain namehttps://api.<Your_Domain_name>.xyz in my case, https://api.b13facebook.xyz

And that record we didn't add yet in our domain name.

We are going to set up a Health check in route 53. So route

53 checks the health of the backend servers and if it is

unhealthy (hits by disaster) then it will transfer the traffic

to another region's (**disaster recovery region,**

Oregon) backend server.

Head over to route 53 service. And click on the health check button on the left panel.

Click on the create health check button.

On this page, we can configure our Health check. Give a name to your health check, and select the endpoint to monitor it. select HTTP and in the Domain name field give the DNS of the ALB-backend which is in US-EAST-1 because **us-east-1 is our primary region**. And fill in all the details as I have shown you in the below image. And then click on the next button.

Our health check is set up successfully. it takes a few minutes to show whether our specified resource is healthy or not.

In the next few minutes, it starts showing you the health of the ALB-backend (backend-servers).

Now let's create a record in our domain name. click on the hosted zone and select your public hosted zone or your domain. I already have one. And click on the Create record button in the top right corner.

Select failover record. And click on the next button.

Here in the record name field write api so that our record name becomes api.<Your_Domain_name>.xyz. in the

917396627149-Madhukiran
devopstraininghub@gmail.com

record type field select “A” and then click on the define failover record button.

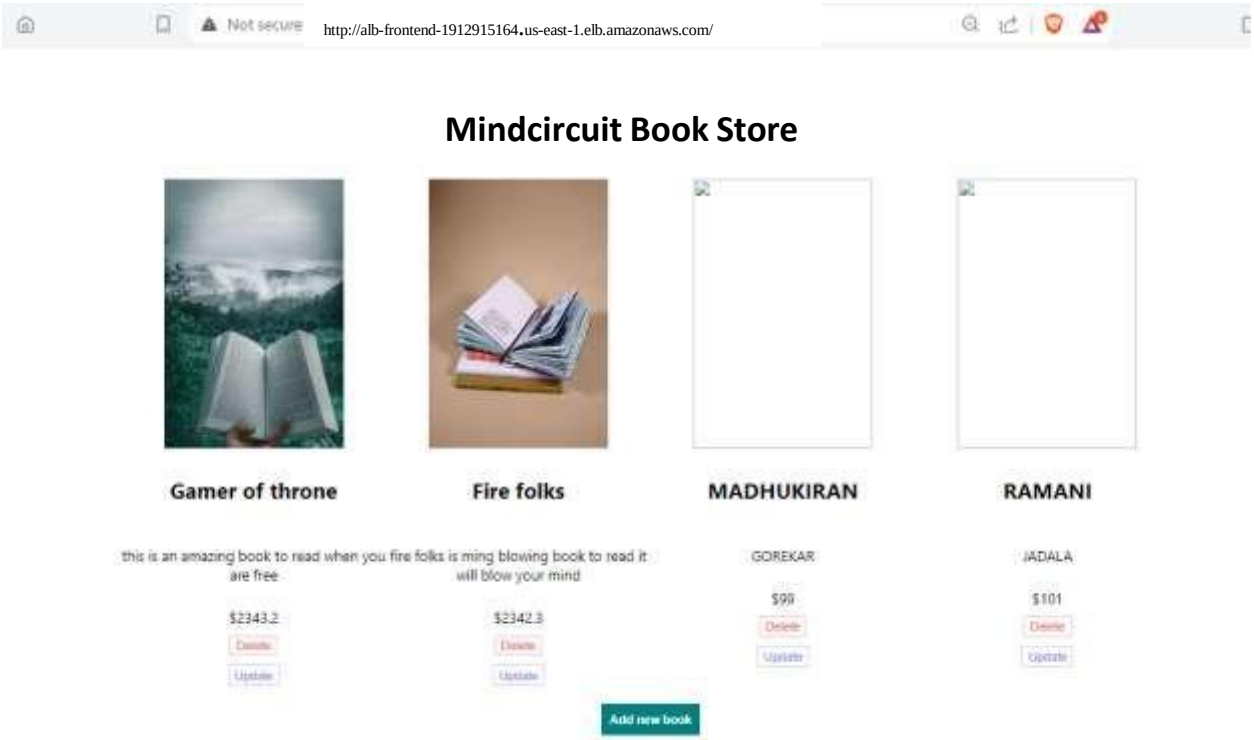
Firstly Select Alias to application and classic Load balancer from the drop-down list, secondly, select us-east-1 as a region. And in the below drop-down list select DNS of the ALB-backend. As you know that **us-east-1 is our primary region** so select primary in failover type. And in the health check ID select the health check that we have created just now. And click on the Define failover record button

Click on create record button.

Now we need to set up one more failover record with the same domain name but for a secondary region. Firstly Select Alias to application and classic Load balancer from the drop-down list, secondly, select us-west-2 as the region. And in the below drop-down list select the DNS of the ALB-backend. As you know that **us-west-2** is our secondary region so select secondary in failover type. **Make sure you don’t select anything in health check ID**. And click on the Define failover record button.

So we have two failover type records with the same name but one is pointing to the backend load balancer which is in **us-east-1 region** and the second one is pointing to the backend load balancer of the **secondary region Oregon(us-west-1)**.

Now it’s time to access our website so take the DNS of the **frontend load balancer** (ALB-frontend) and paste it into the browser. I am sure that you will see the website in fully functional mode. You can add and remove books



- **CloudFront**

AWS CloudFront is a CDN service provided and fully managed by AWS. By utilizing CloudFront we can do the caching of our website at every edge location of the world. The user of the website faces less latency and gets high performance.

let's create Cloudfront distribution for our website. Head over to CloudFront.

Click on the distribution button on the left panel and then click on the create distribution button on top right corner.

In the origin name field select ALB-frontend (**us-east-1 primary region**).

Select Match Viewer in the protocol field. And scroll down

In viewer policy select Redirect **HTTP to HTTPS** and allow all the methods. But please make sure that you select CashingDisabled and in cache policy and select AllViewr in origin request policy.

Click on the add item button and add an alternative domain name (threetier.<yourdomainname>) and select the certificate that we have created in the Custom SSL certificate field.

Scroll down and click button create distribution.

Now, click on the distribution that we have created just now and click on the Origin tab. Here you need to select create origin the button in the top right corner.

Click on the origin domain field and select the ALB-frontend (**us-west-2 secondary region)**, select math view in protocol and the rest of the parameters are all the same so click on the create origin button.

So now we have two Origins one is points to ALB-frontend which is in us-east-1 and the second one is pointing to ALB-frontend which is in the secondary region Oregon (us-west-2). Now click on the create origin group button.

Here, In the Origins field select the first origin that is associated with us-east-1 and click on the add button. And again click on the origin field and select the origin that is associated with us-west-2 and click on the add button. Give any name to the origin group (frontend_failover_handler) and select all the failover

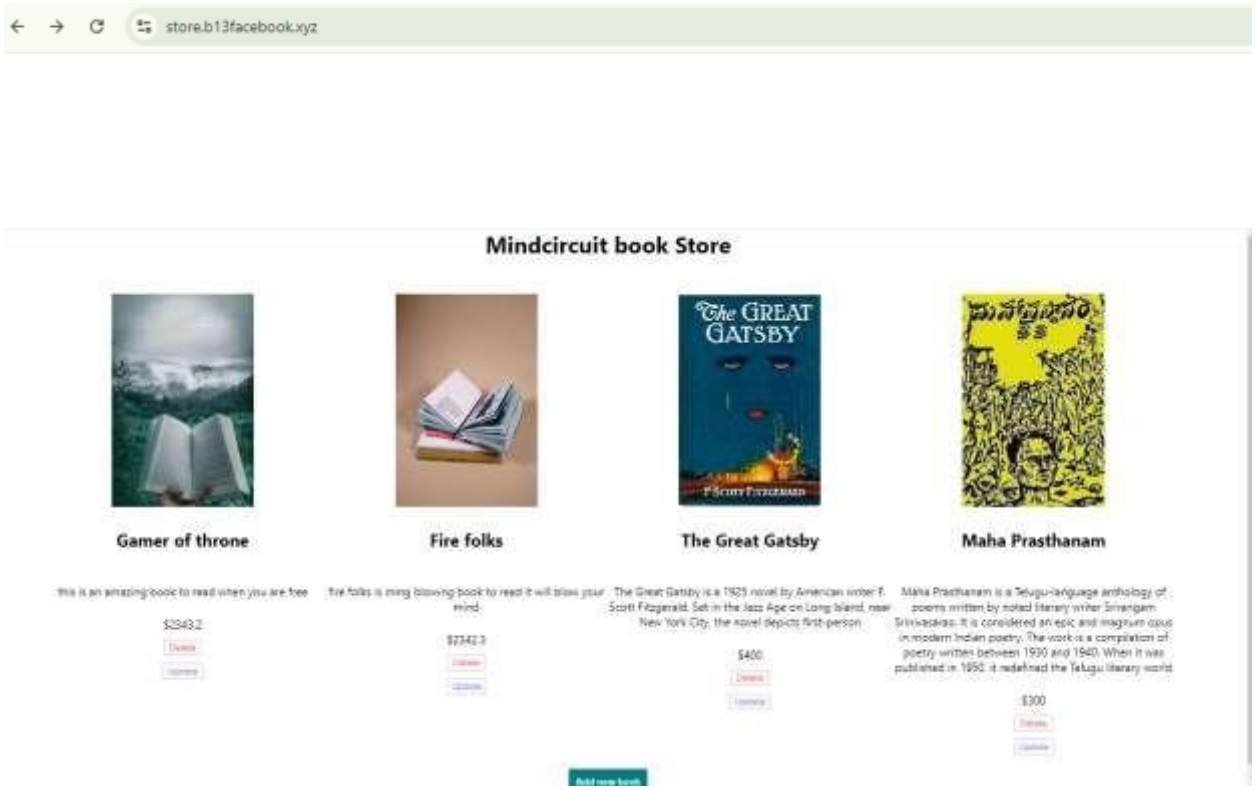
Now, click on the behavior tab. And select the behavior and click on the edit button.

Here we need to change the origin and origin group. Select the origin group that we have just created (frontend_failover_handler). Scroll down and click on the save button.

We need to wait till the distribution become available. It takes around 5-8 minutes. And then we can access our website through the DNS name generated by CloudFront. But we want to access the web app custom domain name. so again head over to Route 53 and select your public hosted zone. or your domain name hosted zone. and click on create record button.

Select simple record, and click on the button defined record. In the record name, add name threetier so our domain name becomes threetier.<Your_Domain_name>.XYZ , Select record type “A”. Select Alias to CloudFront distribution from the drop-down list in value/route traffic to field. And select the distribution that we have created just now. Lastly, hit the define simple record button. Route 53 takes sometime around 5-10 minutes to route traffic on the newly created record so please wait.

Now, let's check the final endpoint. Please hit the record name that you have set up.



Resource cleanup

✓ **RDS**

- RDS instance (takes a lot of time)

✓ **Route 53**

- Delete private hosted zone (rds.com)
- Delete all records in the public hosted zone

✓ **EC2**

- Delete ASG
- Terminate Bastion host
- Delete ALB
- Delete TG
- Delete the Launch template
- Deregister AMIs which are created manually

- ✓ **ACM**
  - Delete the certificate
- ✓ **VPC**
  - Delete NAT gateways (takes around 5 minutes)
  - Release the Elastic IP
  - Delete VPC in both regions (17 resources will be deleted on one click)